



**INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
FLUMINENSE
Campus Campos-Centro**



Controladores Lógicos Programáveis

Curso Técnico em Automação

Elaboração/Edição:

Leonardo Tavares, Msc / Natália Nogueira Monteiro

Aprovação/Revisão Final:

Leonardo Tavares, Msc

1ª edição / Rev. 0

CAMPOS DOS GOYTACAZES – RJ

2017



SUMÁRIO

1. INTRODUÇÃO	6
1.1. IMPORTÂNCIA DA AUTOMAÇÃO	6
1.2. HISTÓRICO E EVOLUÇÃO.....	7
2. A ESTRUTURA DA AUTOMAÇÃO	9
3. PROJETO DE AUTOMAÇÃO INDUSTRIAL	13
3.1. DOCUMENTOS BÁSICOS E ESSENCIAIS	13
3.1.1. P&ID	14
3.1.1.1. Lista de I/O	14
3.1.1.2. Descritivo Operacional	15
3.1.1.3. Planta Baixa e localização dos Instrumentos e equipamentos.....	15
3.1.1.4. Arquitetura geral de Automação.....	16
4. ESTUDO E DEFINIÇÕES GERAIS SOBRE CLP.....	16
4.1. DEFINIÇÃO.....	16
4.2. CARACTERÍSTICAS GERAIS	16
POSSIBILIDADE DE INTEGRAÇÃO COM REDES DE CHÃO DE FÁBRICA.	17
4.3. APLICAÇÕES.....	17
4.4. COMPONENTES E ESTRUTURA.....	17
4.4.1. MICROPROCESSADOR	18
4.5. MEMÓRIA	18
4.5.1. TIPOS DE DADOS	20
4.5.1.1. REPRESENTAÇÃO NUMÉRICA E NÚMEROS NEGATIVOS.....	20
4.5.1.2. Norma IEEE-754	23
4.5.1.3. DADOS ESTRUTURADOS	24
4.5.2. Estrutura Interna e Mapa de Memória	25
4.5.3. Cálculo de Memória	27
4.5.4. Arquitetura de Memória	28
4.5.5. Estrutura do Mapa de Memória do CLP	29
4.6. CARTÕES DE ENTRADA E SAÍDA	30
4.6.1. Cartões de Entrada Digital	31
4.6.2. Cartões de Saída Digital	35
4.6.3. Cartões de Entrada e Saída Analógica	41
4.7. CÁLCULO DE RESOLUÇÃO.....	42
4.7.1. INSTALAÇÃO E TROCA DOS CARTÕES	44
4.8. CLASSIFICAÇÃO QUANTO AO PORTE	44
4.8.4. Redundância por Software e Hardware	45
4.9. SOFTWARE DE PROGRAMAÇÃO	47
5. PRINCÍPIO DE FUNCIONAMENTO DE UM CLP	48
5.1. FUNCIONAMENTO	48
5.2. ESTADOS DE OPERAÇÃO	51
6. LINGUAGENS DE PROGRAMAÇÃO PARA CLP	51
6.1. CLASSIFICAÇÃO	51
6.1.1. Texto Estruturado (ST).....	52



6.1.2.	Diagrama de Blocos (FBD)	53
6.1.3.	Lista de Instruções (IL)	53
6.1.4.	Passos ou Steps (SFC – Sequential Function Chart)	54
6.1.5.	Ladder (LD – Ladder Diagram)	54
6.2.	PROCESSAMENTO DO PROGRAMA	55
6.2.1.	CARACTERÍSTICAS GERAIS DAS TASKS	55
6.3.	ALOCÇÃO DINÂMICA DE DADOS EM MEMÓRIA	56
6.4.	TIPAGEM DE DADOS	56
6.5.	ESTRUTURAÇÃO, MODULARIZAÇÃO, REUTILIZAÇÃO E MODELO DE SOFTWARE	56
7.	NORMA IEC 61131	58
7.1.	IEC 61131-1 – INFORMAÇÕES GERAIS	58
7.2.	IEC 61131-2 - REQUISITOS DE EQUIPAMENTOS E TESTES	59
7.3.	IEC 61131-3 – LINGUAGENS DE PROGRAMAÇÃO	59
7.4.	IEC 61131-4 – ORIENTAÇÕES PARA O USUÁRIO	61
7.5.	IEC 61131-5 - COMUNICAÇÕES	61
7.6.	IEC 61131-7 – PROGRAMAÇÃO DE CONTROLE FUZZY	62
7.7.	IEC 61131-8 – IMPLEMENTAÇÃO DAS LINGUAGENS	62
8.	PRÁTICA – PROGRAMAÇÃO	63
8.1.	INTRODUÇÃO	63
8.2.	INSTRUÇÕES E BLOCOS DE FUNÇÃO	63
8.2.1.	Booleanas	64
8.2.1.1.	XIC (Examine If Closed)	64
8.2.1.2.	XIO (Examine If Opened)	65
8.2.1.3.	OTE (Output Energize - Bobina Simples)	66
8.2.1.4.	OTL (Output Latch)	66
8.2.1.5.	OTU (Output Unlatch)	67
8.2.1.6.	ONS (One Shot)	67
8.2.1.7.	OSR (One Shot Rising)	69
8.2.1.8.	OSF (One Shot Falling)	70
8.2.2.	Timer/Counter	72
8.2.2.1.	TON (Timer On Delay)	72
8.2.2.2.	TOF (Timer Off Delay)	73
8.2.2.3.	RTO (Retentive Timer On)	74
8.2.2.4.	CTU (Count Up)	76
8.2.2.5.	CTD (Counter Down)	77
8.2.2.6.	RES	77
8.2.3.	Comparadores	78
8.2.3.1.	Equal (EQU)	78
8.2.3.2.	Not Equal To (NEQ)	79
8.2.3.3.	Less Than (LES)	80
8.2.3.4.	Greater Than (GRT)	81
8.2.3.5.	Less Than or Equal To (LEQ)	82
8.2.3.6.	Greater Than or Equal To (GEQ)	83
8.2.3.7.	Compare (CMP)	84
8.2.4.	Matemáticos	86
8.2.4.1.	ADD	86
8.2.4.2.	SUB	87



8.2.4.3.	MUL.....	88
8.2.4.4.	DIV.....	89
8.2.4.5.	MOD.....	90
		90
8.2.4.6.	SQR.....	91
8.2.4.7.	Negate (NEG).....	92
8.2.4.8.	ABS.....	93
8.2.4.9.	Compute (CPT).....	94
8.2.5.	<i>Operadores Lógicos</i>	95
8.2.5.1.	MOV.....	95
8.2.5.2.	Bitwise AND (AND).....	95
		95
8.2.5.3.	Bitwise OR (OR).....	97
8.2.5.4.	XOR.....	98
8.2.5.5.	Bitwise NOT (NOT).....	100
8.2.5.6.	CLR.....	101
8.2.6.	<i>Movimentação de Bits</i>	101
8.2.6.1.	Bit Shift Left (BSL).....	101
8.2.6.2.	Bit Shift Right (BSR).....	102
8.2.7.	<i>JSR</i>	103
REFERÊNCIAS		104



ÍNDICE DE FIGURAS

Figura 1 – Arquitetura tradicional e simplificada de Automação	10
Figura 2 - Níveis da Pirâmide da Automação	11
Figura 3 – P&ID Exemplo	14
Figura 4 – Lista de I/O Exemplo	15
Figura 5 – Exemplo de Arquitetura de Automação (Fonte: Catálogo WEG)	16
Figura 6 - Diagrama em Blocos com os componentes básicos de um CLP.	18
Figura 7 - Memória eletrônica típica com os barramentos indicados	26
Figura 8 - Cartão de Entrada Digital com Alimentação em Corrente Contínua	31
Figura 9 - Cartão de Entrada Digital com Alimentação em Corrente Alternada	31
Figura 10 - Cartão de Entrada Digital do tipo PNP ou Sourcing	32
Figura 11 - Cartão de Entrada Digital do tipo NPN ou Sinking	32
Figura 12 – Cartão de Entrada Digital com Diagnóstico de Fio Partido	33
Figura 13 - Cartão de Saída Digital com Alimentação em Corrente Contínua - 24Vcc	35
Figura 14 - Cartão de Saída Digital em Corrente Contínua do tipo Transistor PNP	35
Figura 15 - Cartão de Saída Digital em Corrente Contínua do tipo Transistor NPN	36
Figura 16 - Circuito Interno de Cartão de Saída Digital do Tipo TRIAC	36
Figura 17 - Demonstração de disparo de pulso no Gate em momentos diferentes do semiciclo	37
Figura 18 - Demonstração de atraso no desligamento da saída vs sinal lógico	38
Figura 19 - Circuito Interno de Cartão de Saída do Tipo Relé	38
Figura 20 - Cartão de saída com Diagnóstico de Fio Partido	39
Figura 21 - Demonstração de acionamento de carga indutiva em saídas digitais sem proteção	40
Figura 22 - Diodo inversamente polarizado em paralelo com carga indutiva	40
Figura 23 - Estrutura Básica e circuito de Conversão A/D	41
Figura 24 - Estrutura Básica e circuito de conversão D/A	42
Figura 25 - Exemplo de Redundância de Hardware	46
Figura 26 - Diagrama de fluxo que descreve o processo de redundância por Software	47
Figura 27 – Ciclo de operação do CLP	49
Figura 28 – Ciclo de operação do CLP	50
Figura 29 - Partes da Norma IEC 61131	58



1. INTRODUÇÃO

A humanidade vem se transformando ao longo dos séculos, evoluindo, se adequando e se adaptando às mudanças decorrentes desses novos tempos, demos um verdadeiro salto na evolução, saindo da idade da pedra para dias atuais, vivendo a era digital.

Desde a primeira Revolução Industrial, o homem vem buscando cada vez mais meios de se obter um menor esforço, mais rapidez e eficiência na execução e finalização de tarefas, principalmente quando se trata de fabricação de produtos, o que se potencializou com o surgimento das máquinas e motores a vapor nos meados do século XIX.

Estamos sempre evoluindo, buscando algo novo, melhorando o que já está bom, mas quando menos esperamos, algo novo surge e ainda assim continuamos o processo.

Chegamos a um momento onde tudo praticamente funciona com uma enorme dependência das tecnologias, usamos computadores, eletrodomésticos digitais, carros computadorizados, tablets e celulares nas nossas tarefas diárias, sejam elas profissionais ou pessoais, enfim, estamos num caminho sem volta, mas isso faz parte de nossa evolução.

Todavia, diante de tal cenário, cabe observar que nada disso seria possível, sem que também houvesse uma mudança nos processos produtivos no contexto industrial, máquinas e processos industriais ficaram muito eficientes, mais rápidos, mais seguros e mais inteligentes, e esse é o momento onde realmente percebemos a presença da Automação Industrial.

1.1. Importância da Automação

A inserção da automação na indústria está diretamente associada ao aumento de velocidade do fluxo e processamento de informações e, portanto, há uma maior produtividade nos processos operacionais. Além disso, com um sistema de automação é possível atuar de diversas maneiras proporcionando ganhos de uma forma geral através do aumento da segurança, redução no consumo de energia, matéria-prima e no uso da força de trabalho, proporcionando um produto final de melhor qualidade e menor custo.

Um dos grandes passos de relevante importância na área da automação foi, na década de 60, o surgimento do Controlador Lógico Programável, ou CLP, que tornou todo processo industrial muito mais produtivo e confiável, além de reduzir drasticamente os custos para as empresas. Assim, desde então, essa tecnologia vem revolucionando a área de comandos e controle nas indústrias, que anteriormente era feita por relés eletromagnéticos.



1.2. Histórico e Evolução

O CLP e suas primeiras gerações surgiram em 1969, antes, porém, os sistemas de controle ou hardware era realizados principalmente através de painéis relés, dos mais variados. Contudo tais dispositivos eram as vezes sinônimo de “dor de cabeça”, alguns problemas, como os listados abaixo, geravam paradas de processo e recorrentes intervenções da equipe de manutenção industrial (Souza, 1999):

- Mau contato e/ou desgastes dos contatos;
- Necessidade de instalação de inúmeros relés, execução de fiação entre os inúmeros terminais de contatos e de bobinas;
- Complexidade na introdução de alteração na sequência;
- Necessidade de manutenções periódicas.

Apesar de apresentarem algumas desvantagens acima citadas, os relés eram uma excelente solução, em razão de não haver, na ocasião, uma tecnologia que pudessem substituí-los.

No final da década de 60, iniciou-se o desenvolvimento de microcomputadores, utilizando-se o circuito integrado (CI), e a universalidade da capacidade de processamento dos mesmos tornou-se o centro das atenções, aguardando-se com enorme expectativa o surgimento do hardware para controle dotado de grande versatilidade de processamento.

Quase que ao mesmo tempo, inicia-se a era da produção em grande escala, e os assuntos, como automação, incremento da produtividade, uniformidade na qualidade e outros, transformam-se em temas principais indústria de forma geral, e a solução desses problemas era exigida também pelo lado da tecnologia de controle de sequência. Na época, a General Motors (GM – empresa automobilística americana), na Hydromic Division, sob a liderança do engenheiro Richard Morley, em 1968, devido à grande dificuldade de se mudar a lógica de controle em painéis de comando a cada mudança na linha de montagem (era necessário parar a produção para troca de todos os quadros elétricos), o que gerava altos gastos de tempo e dinheiro, anunciou 10 itens relativos às condições que um novo dispositivo eletrônico de controle de sequência deveria atender para que pudesse substituir os tradicionais relés, conforme abaixo: (Souza, 1999).

- a. Facilidade de programação, de alteração do programa, inclusive nas estações de trabalho;
- b. Facilidade na manutenção, desejável que fosse totalmente do tipo de encaixar (plug-in unit);
- c. A confiabilidade na estação de trabalho deverá ser superior em relação ao painel de controle do tipo com relés;
- d. Deverá ser mais compacto que o painel de controle do tipo com relés;



- e. Possibilitar o envio direto de dados à unidade central de processamento de dados;
- f. Deverá ser economicamente competitivo com o painel de controle do tipo com relés;
- g. Possibilitar entradas com níveis de tensão alternada da ordem de 115[V];
- h. As saídas deverão ser em 115[V] C.A. com capacidade superior a 2[A] de intensidade de corrente; deverá ainda possibilitar a operação das válvulas solenoides, comando para partida de motores e outros;
- i. Com um mínimo de alteração, possibilitar a ampliação do sistema básico;

A evolução dos CLPs está ligada diretamente ao desenvolvimento tecnológico da informática em suas características de software e hardware. Por exemplo, o que antes era executado com componentes discretos, hoje utilizam-se microprocessadores e microcontroladores de última geração, com técnicas de processamento paralelo, inteligência artificial, redes de comunicação, etc, que são muito mais complexos, uma vez que os processos industriais geralmente necessitam de controle para malhas analógicas além de somente realizar lógicas discretas.

Hoje, os CLPs oferecem um considerável número de benefícios para aplicações industriais, que podem resultar em economia, que excede bastante o custo do CLP, tornando o seu uso muito viável.

Dentre suas vantagens, temos: menor ocupação de espaço; potência elétrica requerida menor; grande número de contatos; reprogramação, se ocorrerem mudanças de requisitos de controle; confiabilidade maior com diagnóstico de falhas; manutenção mais fácil; maior flexibilidade e capacidade de processamento, satisfazendo um maior número de aplicações; permite a interface através de rede de comunicação com outros CLPs e microcomputadores; velocidade de operação, entre outras.

Todas estas considerações mostram a evolução de tecnologia, tanto de hardware (CPU, fonte de alimentação, etc) quanto de software, o que permite o seu acesso a um maior número de pessoas tanto nos projetos de aplicação de controladores programáveis quanto na sua programação.



2. A ESTRUTURA DA AUTOMAÇÃO

A automação industrial é reflexo das necessidades dos processos industriais e seus mais diversos segmentos, tais necessidades estão associadas ao aumento da velocidade de processamento das informações, uma vez que as operações estão cada vez mais complexas e variáveis, necessitando de um grande número de controles e mecanismos de regulação para permitir decisões mais ágeis e, portanto, aumentar os níveis de produtividade e eficiência do processo produtivo dentro das premissas da excelência operacional.

A automação permite economias de energia, força de trabalho e matérias-primas, um melhor controle de qualidade do produto, maior utilização da planta, aumenta a produtividade e a segurança operacional. Em essência, a automação nas indústrias permite elevar os níveis de continuidade e de controle global do processo com maior eficiência, aproximar ao máximo a produção real à capacidade nominal da planta, ao reduzir ao mínimo possível as horas paradas, de manutenção corretiva e a falta de matéria-prima.

Além disso, com o advento dos sistemas de automação baseados em redes de campo e tecnologia digital, podem-se ter vários benefícios em termos de manutenção e aumentar a disponibilidade e segurança operacional, permitindo que os sistemas de automação e controle, equipamentos de campo, controladores, possam assumir funções antes inimagináveis, como o controle de contínuo e discreto, tempos de varreduras menores, arquiteturas redundantes, gerenciamento e tráfego de informação, disponibilidade de informações para IHMs, Internet, geração de relatórios, gerenciamento de ativos, altos níveis de segurança, tudo isso, aliado à confiabilidade industrial tanto de hardware quanto de software.

Quanto mais informação, melhor uma planta pode ser operada e sendo assim, ou seja, menores perdas, melhores produtos e matéria prima, portanto mais eficiente.

A tecnologia da informação tem sido determinante no desenvolvimento de diversos conceitos e meios de comunicação para a operação de dispositivos, isso já está presente nos em nossas vidas, tudo está conectado, desde celulares, Tvs, aparelhos de Ar Condicionado, Geladeiras e muitos outros, entramos definitivamente na era IoT (Internet of Things), internet das coisas, onde as máquinas se falam. (Internet of Things (IoT), s.d.)

Toda essa tendência também está presente na área profissional e industrial, o IoT também se destaca na indústria, o IIoT (Industrial Internet of Things), não exatamente da mesma forma e com a mesma velocidade, mas usando o mesmo conceito, onde as máquinas se falam e geram mais informações, todo esse conceito tem promovido mudanças nas tecnologias da automação, alterando hierarquias e estruturas nos mais

diversos ambientes industriais, desde as indústrias de processo e manufatura até prédios e sistemas logísticos. A capacidade de comunicação entre dispositivos e o uso de mecanismos padronizados, abertos e transparentes são componentes indispensáveis do conceito de automação de hoje. (Sasajima, Ishikuma, & Hayashi, 2015)

Logo, um sistema de automação industrial é composto por vários níveis de controle e trabalho, desde os equipamentos e instrumentos de campo até o gerenciamento corporativo das empresas. A automação, portanto, não é mais apenas uma arquitetura de controle com sensores, atuadores, controladores e sistema de supervisão ou interface operacional (Figura 1), mais que isso, a automação deve proporcionar informação, a partir dos dados de processo, através de sensores e instrumentos e seus respectivos controles, para que, através dessas informações, decisões possam ser tomadas, análises possam ser efetuadas e melhorias contínuas possam ser aplicadas.

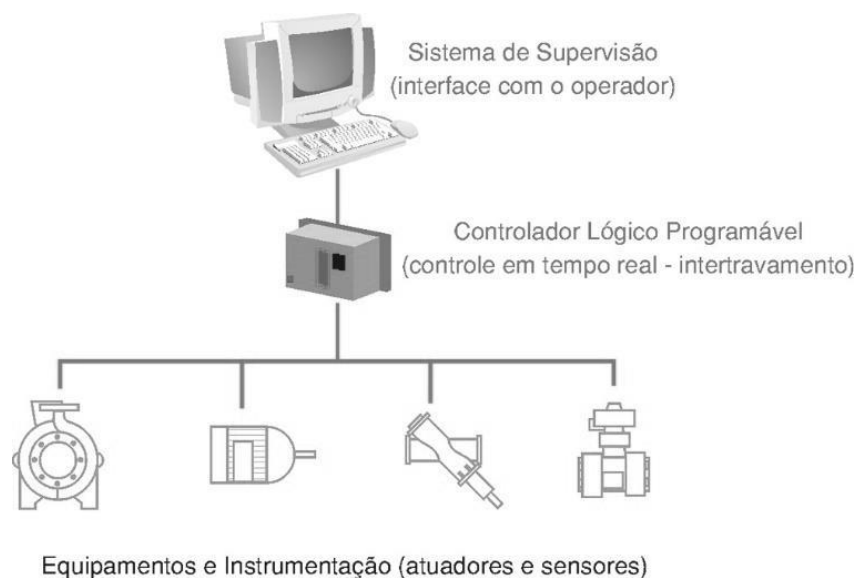


Figura 1 – Arquitetura tradicional e simplificada de Automação

O conceito anterior pode ser representado e estudado de maneira mais detalhada e aprofundada através da Pirâmide da Automação, onde os principais níveis hierárquicos são distribuídos numa sequência vertical de fluxo de dados e informação, caracterizando cada nível de acordo com suas funções e particularidades (Figura 2).

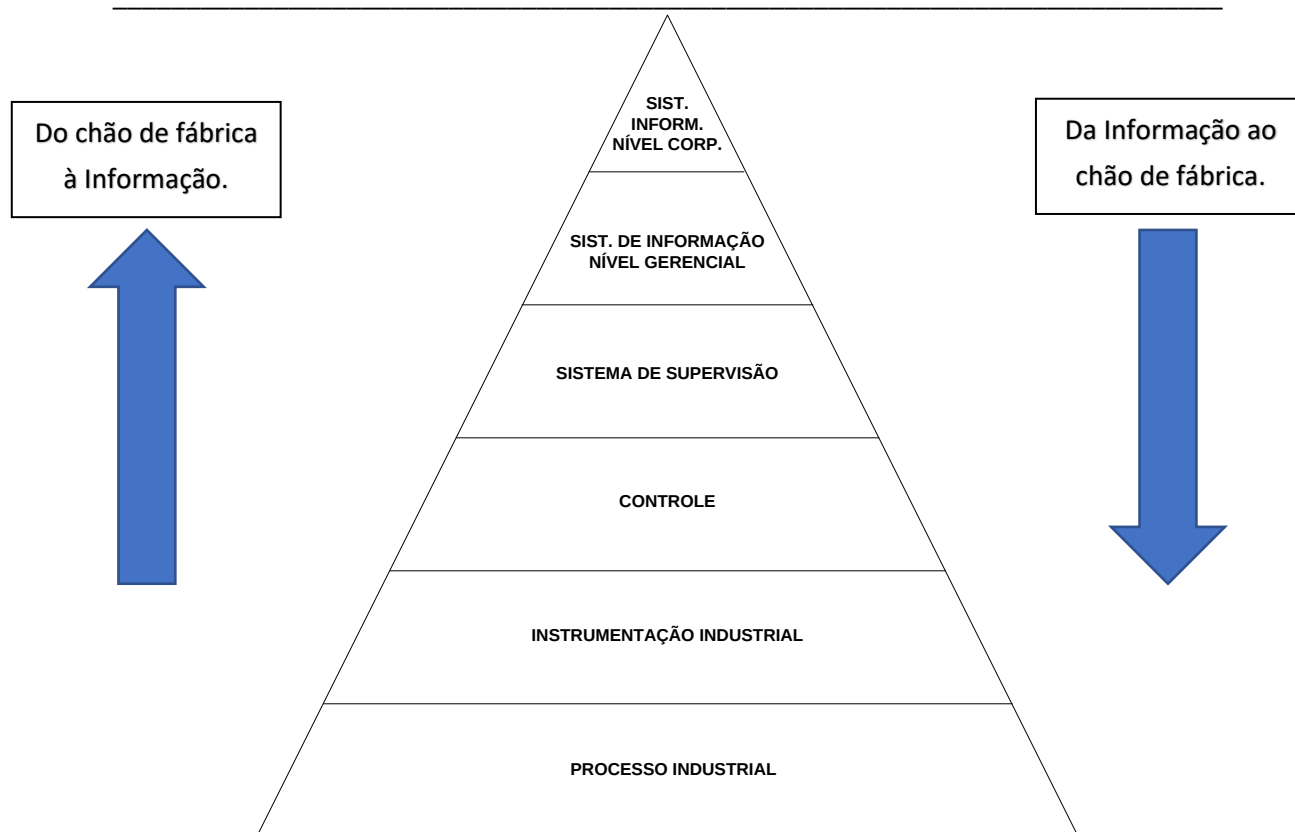


Figura 2 - Níveis da Pirâmide da Automação

Processo Industrial:

Nível de processo, máquinas, equipamentos, dispositivos e componentes da planta. Onde os componentes dessa camada devem, pelo menos, ser conhecidos pelo profissional da área de automação, uma vez que estes, muitas vezes, são o objeto de controle ou elementos finais de controle.

Tipos de bombas e suas respectivas curvas de vazão, caldeiras, trocadores de calor, filtros, evaporadores, são alguns dos tipos de equipamentos industriais encontrados nesse nível.

Instrumentação Industrial:

Este nível se compõe basicamente por dispositivos de campo, como por exemplo atuadores, sensores, transmissores, entre outros, que são diretamente conectados ao processo ou aos equipamentos de um processo. Podem operar como equipamentos ponto a ponto com o sistema de controle ou como elementos microprocessados, inteligentes e sofisticados que geram grande volume de dados. Justamente por isso, as redes de comunicação utilizadas aqui são bastante heterogêneas, mas possuem, em geral, baixa largura de banda, bem como funcionalidades relativamente restritas, tais como as redes CAN, AS-i, Modbus, Profibus PA e Fieldbus Foundation H1 e em raras ocasiões, é possível encontrar equipamentos que se utilizam de tecnologia Ethernet.

Controle do processo

É o nível onde se encontram os equipamentos que fazem o controle automático das atividades da planta, composto por equipamentos que realizam o controle automatizado de processos: CLPs (Controlador Lógico Programável) e SDCDs (Sistema Digital de Controle Distribuído). Para tal, deve ser dotada de equipamentos e dispositivos com um nível considerável de inteligência, de modo que possam ser implantadas as diretrizes de automação do sistema de controle, tais como lógicas e condições de acionamento. Desse modo, as exigências com relação à comunicação de dados são maiores e as redes neste nível apresentam um maior grau de sofisticação, lançando mão de protocolos tais como Profibus DP, DeviceNet, ControlNet, Modbus TCP e EtherNet IP, os quais dispõem de velocidade superior e dotam os sistemas de controle da capacidade de responder às informações de campo em tempo real.

Supervisão do processo

É o nível no qual se insere a supervisão e monitoração de processos e, geralmente, possui um banco de dados no qual estão todas as informações do processo. Nestes tipos de sistema, os operadores têm acesso aos dados de processo em tempo real, dados instantâneos com os quais ele deve ter atenção para manter o processo em níveis de controle adequados, embora exista o controle automático para tal função. As soluções implementadas neste nível demandam um grande e constante volume de dados e, assim, a utilização de protocolos que lançam mão da tecnologia Ethernet é generalizada. Também é muito comum encontrar conversores de protocolo que utilizam a tecnologia OPC neste nível, de modo a garantir interoperabilidade entre diversos sistemas.

Gerenciamento da planta

Neste nível, os dados são tratados com o intuito de transformar tais dados em informação, estas informações já permitem e norteiam decisões em níveis gerenciais, tais como planejamento detalhado da produção, auxiliando a parte de controle e também a logística de suprimentos, além de otimizar os custos, também a manutenção de equipamentos podem usufruir de tais informações para a tomada de decisões quanto à manutenção preditiva. Neste nível, por meio de ferramentas como o MES e o PIMS, que geram relatórios mais avançados que os disponibilizados pelas ferramentas de supervisão, devido à grande massa de dados trabalhados pelas soluções de software disponíveis, todas as informações trafegam em redes de alto desempenho baseadas em Ethernet, bem como lançam mão de bancos de dados para armazenamento histórico das informações.

Gerenciamento corporativo

Última camada da pirâmide da automação, neste nível, se encontram softwares voltados para gestão mais corporativa, gestão financeira e *Business Intelligence* (BI), ou



seja, é responsável pela administração de recursos da empresa o gerenciamento corporativo se desprende das ferramentas fabris e foca na administração dos recursos da empresa, com um enfoque nos resultados financeiros do processo produtivo. Assim, essas redes dificilmente possuem acesso a informações e relatórios técnicos disponíveis nos níveis inferiores e são totalmente baseadas em tecnologia Ethernet.

3. PROJETO DE AUTOMAÇÃO INDUSTRIAL

A elaboração de um projeto de automação industrial é um processo longo e que passa por várias etapas de desenvolvimento, todavia sempre mantendo a atenção especial com as funcionalidades do processo e com a informação para os níveis de tomadas de decisão.

Todo projeto, seja ele básico, detalhado ou executivo, tem que ter como premissa uma boa documentação e que esta seja consistente, capaz de orientar e definir bem os conceitos da automação.

O estudo sobre projetos de automação é extenso e não será abordado de forma aprofundada nesta apostila, contudo serão apresentadas algumas atividades iniciais para um melhor entendimento.

3.1. Documentos Básicos e Essenciais

Conforme citado, um projeto consiste na elaboração e uso de diversos documentos que irão direcionar a execução e o posterior trabalhos de implementação do sistema, contudo para usarmos como base e ponto de partida levaremos em consideração apenas os seguintes documentos:

- P&ID (Fluxograma);
 - Lista de IO;
 - Descritivo Operacional;
- Planta Baixa de Instrumentação;
 - Arquitetura de Automação.

Os dois principais acima são base para a elaboração dos outros dois documentos, conforme estudaremos a seguir, nos próximos tópicos.

3.1.1. P&ID

Também chamado de Fluxograma, P&ID é um diagrama de engenharia de processo onde são representados os equipamentos gerais, tubulação e instrumentação, mostra o fluxo físico de processos (Figura 3). É geralmente usado no campo da engenharia.

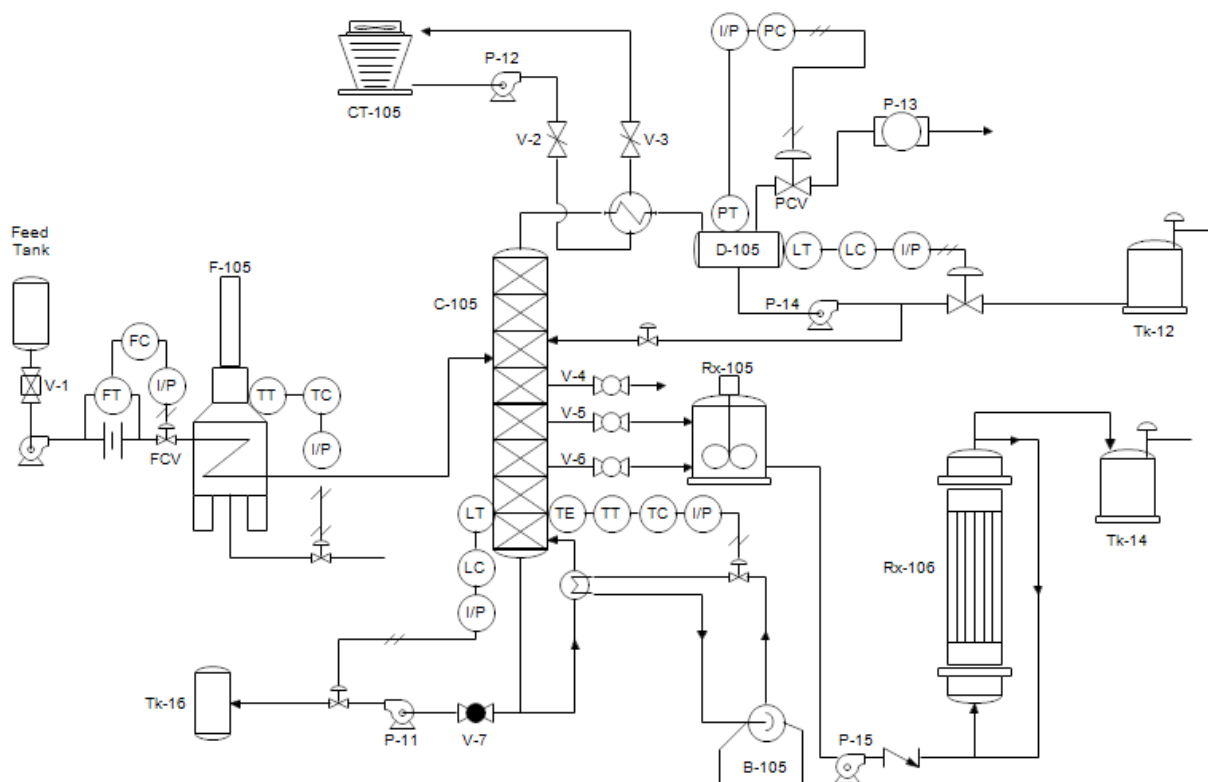


Figura 3 – P&ID Exemplo

O P&ID de um processo industrial é de extrema importância para termos a definição do sistema de controle a ser utilizado em cada caso. A partir do fluxograma, iniciamos os estudos e análises para o início do dimensionamento da automação.

Uma vez o P&ID definido com toda a instrumentação necessária para o funcionamento do processo, suas respectivas malhas de controle, segurança e intertravamento, damos início a próxima etapa; a de levantamento de pontos.

Um sistema de controle trabalha em um ciclo, que chamamos de varredura, este ciclo começa com a leitura dos sinais de entrada, processamento de um programa e acionamento de saídas.

3.1.1.1. Lista de I/O

A leitura dos sinais de entrada é realizada através de cartões específicos, assim também como os sinais de saída, os principais sinais são o 4 a 20 mA, 1 a 5 vcc, 0 ou 24vcc, RTD (PT-100), mV (Termopar), etc. Estes sinais são trocados (enviados e recebidos), entre os instrumentos e o sistema de controle, portanto para que possamos

definir tais cartões, é necessário elaborar uma lista de entradas e saídas do projeto, esta lista chamamos de Lista de I/O's de um projeto de automação.

Com essa lista, estabelecemos a quantidade de pontos de entrada e saída que serão necessários no CLP, esses sinais são divididos e identificados como:

- Entrada Discreta – DI (Discrete Input);
- Saída Discreta – DO (Discrete Output);
- Entrada Analógica – AI (Analog Input);
- Saída Analógica – AO (Analog Output).

De cada instrumento, identificamos os tipos e as quantidades de sinais que estes irão ocupar no CLP.

Item	Tag	P&ID	Serviço	DI	DO	AI	AO
1	FIT-1003402/1	DE-001-023	Vazão de alimentação do R-2371	1		1	
2	XV-1003401/1	DE-001-023	Saída de produtos do R-2373	2	1		
3	SX-1003503	DE-001-023	Controle de vazão	2	1	1	1

Figura 4 – Lista de I/O Exemplo

Com todas essas informações de todos os instrumentos do processo, pode-se definir o número de cartões e/ou remotas necessárias de acordo com o número de entradas e saídas utilizadas e também pode-se definir o porte do CLP mais apropriado para cada processo.

3.1.1.2. Descritivo Operacional

Outro importante documento usado neste processo da automação, é o descritivo operacional, que descreve o funcionamento do processo e ajuda no entendimento para a elaboração do programa e na concepção do sistema de controle, assim como as necessidades de lógicas de segurança e intertravamento.

O descritivo também possibilita entender o processo e como podemos contribuir operacionalmente usando as ferramentas da automação e controle.

3.1.1.3. Planta Baixa e localização dos Instrumentos e equipamentos

Identifica os instrumentos na planta geograficamente para permitir o encaminhamento de cabos até o sistema de controle. Dependendo da distribuição e da densidade de instrumento na área operacional, é possível decidir se o sistema de controle irá trabalhar com I/O local ou remoto (distribuído), neste caso, reduzindo consideravelmente o cabeamento de campo entre sensores e instrumentos e os painéis de automação. Este estudo deve ser feito baseado na viabilidade técnica e financeira do projeto, definido.

Enfim, define a arquitetura de rede de automação.

3.1.1.4. Arquitetura geral de Automação

Neste documento é representada a concepção geral da automação e sua estrutura principal, definido os principais componentes que irão direcionar o detalhamento e especificação final do sistema de controle.

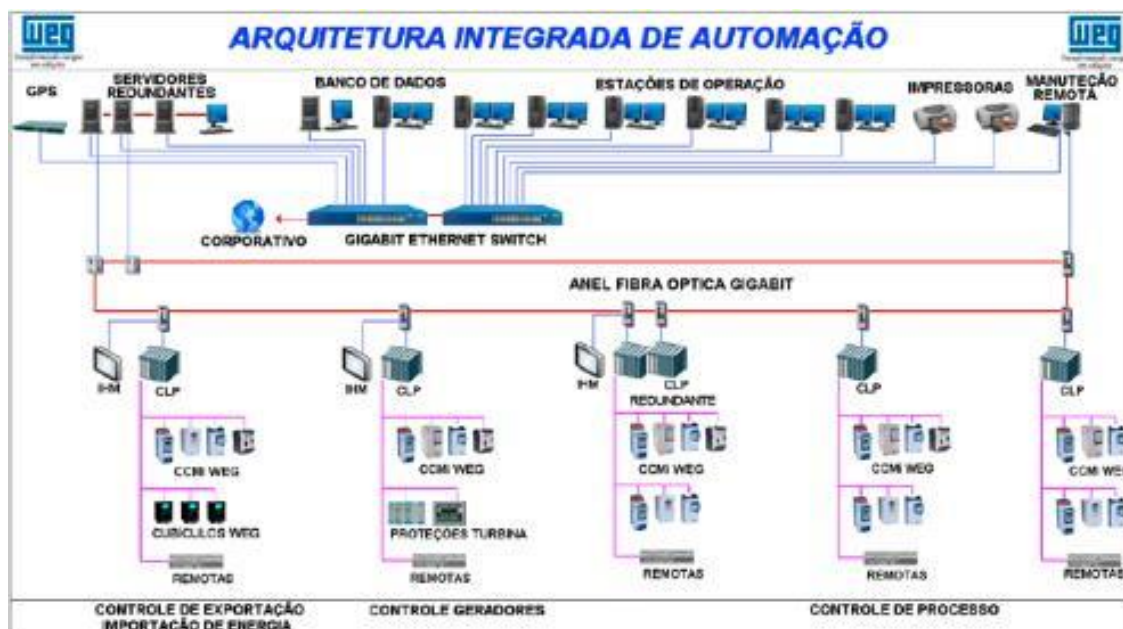


Figura 5 – Exemplo de Arquitetura de Automação (Fonte: Catálogo WEG)

4. ESTUDO E DEFINIÇÕES GERAIS SOBRE CLP

4.1. Definição

Segundo a **Nema (National Electrical Manufacturers Association)** é um “*aparelho eletrônico digital que utiliza uma memória programável para o armazenamento interno de instruções para implementações específicas, tais como lógica, sequenciamento, temporização, contagem e aritmética, para controlar, através de módulos de entradas e saídas, vários tipos de máquinas ou processos.*”.

4.2. Características Gerais

Basicamente, um controlador programável apresenta as seguintes características:

- Capacidade de operação em ambiente industrial;
- Sinalizadores de estado e módulos tipo plug-in de fácil manutenção e substituição;
- Possibilidade de monitoração do estado e operação do processo ou sistema, através da comunicação com computadores constituindo sistemas de supervisão, controle e aquisição de dados (SCADA);
- Compatibilidade com diferentes tipos de sinais de entrada e saída;



- Hardware de controle que permite a expansão dos diversos tipos de módulos, de acordo com a necessidade;
- Possibilidade de expansão da capacidade de memória através de cartões de memória externos;
- Conexão com outros CLPs através de rede de comunicação;
- Possibilidade de integração com redes de chão de fábrica.

4.3. Aplicações

Um controlador programável pode ser inserido para automatizar vários tipos de processos industriais, tais como: sequenciamento, intertravamento, controle de processos, batelada, etc. Pode ser usado tanto na área de automação de manufatura, como também de processos contínuos, elétrica, predial, entre outras.

Com a tendência dos CLPs terem baixo custo, muita inteligência, facilidade de uso e massificação das aplicações, a utilização deste equipamento não será apenas nos processos, mas também nos produtos, como, por exemplo, em eletrodomésticos, eletrônicos, residências e veículos.

4.4. Componentes e Estrutura

O controlador programável tem sua estrutura baseada em um sistema microprocessado semelhante à um hardware de computador, porém trata-se de um hardware relativamente mais simples. As principais diferenças em relação a um computador comum estão relacionadas à capacidade de operação em condições mais severas (ambiente industrial), para tal, é necessário que os componentes eletrônicos dos circuitos principais possuam um tratamento especial em seu projeto. Itens como fonte de alimentação e interfaces de I/Os devem possuir melhor desempenho em filtragem, estabilização, imunidade à ruídos e um invólucro específico para aplicações industriais.

Os componentes principais existentes na estrutura de um CLP são os seguintes:

- Unidade central de processamento (UCP);
- Interfaces de entrada e saída (I/O);
- Memórias.

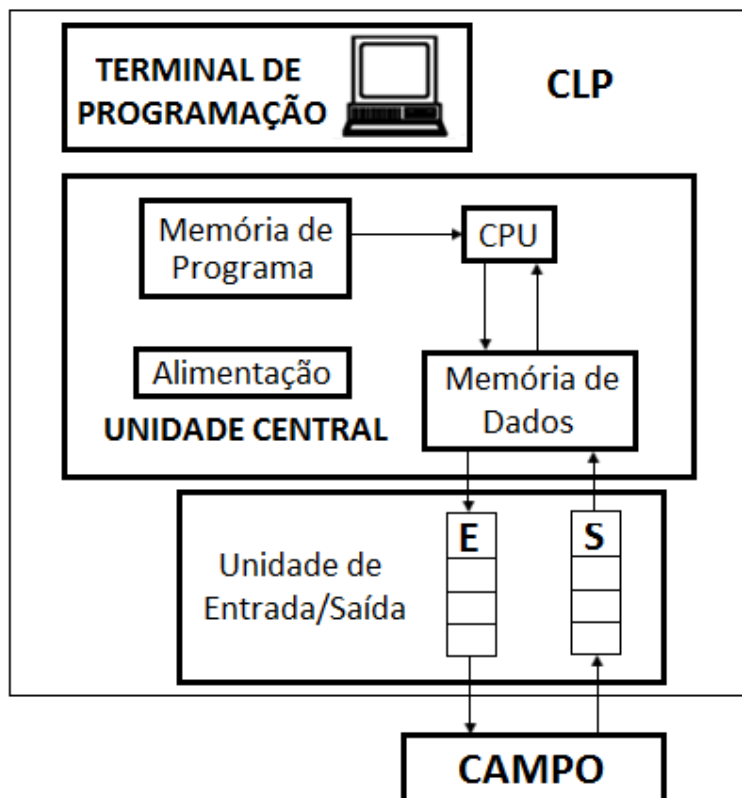


Figura 6 - Diagrama em Blocos com os componentes básicos de um CLP.

4.4.1. Microprocessador

O microprocessador possui a função de efetuar o processamento de acordo com o programa do usuário que está armazenado em memória, obtendo os dados dos cartões de entrada a partir dos sinais elétricos enviados pelos sensores/transdutores e, então, como resultado do processamento, escrevendo os dados para os cartões de saída de modo a permitir os comandos dos atuadores, ou seja, ele é considerado a “inteligência” do sistema de controle. Geralmente, cada CLP tem um microprocessador, o qual pode controlar vários pontos de entradas e saídas (E/S ou I/O). (VIANNA, 2008)

4.5. Memória

As memórias são dispositivos que utilizam mecanismos diversos para armazenar informações, tais como números, letras, caracteres, instruções, endereços, dados, etc.

Sua utilização ocorre principalmente em computadores em geral e subsistemas computacionais. Não utilizam apenas ou especificamente sistemas digitais, embora aquelas utilizadas em sistemas computacionais sejam, na prática, dispositivos digitais pois as informações armazenadas nesses casos são em forma binária.

Em função dos diferentes processos relacionados ao seu funcionamento e uso, as memórias podem apresentar a seguinte classificação:



1. Acesso: A informação é acessada através de endereços, que são um conjunto de bits associados às posições de memória. O tempo de acesso de uma memória é o tempo necessário desde a entrada de um endereço até o momento em que a informação apareça na saída. No caso de memórias de leitura/escrita, também é o tempo para a informação ser gravada. É possível ter acesso a uma dada posição de memória de duas maneiras diferentes:

- Sequencial: chegam à posição endereçada, passando por todas as intermediárias.

Exemplo: fita magnética.

- Aleatório ou direto: chegam diretamente à posição endereçada.

Exemplo: memórias semicondutoras, RAM.

2. Volatilidade: As memórias voláteis são aquelas que perdem as informações armazenadas com corte de alimentação. Ex.: memórias semicondutoras baseadas em Flip-Flops., RAM. As memórias não voláteis são aquelas que mesmo sem alimentação, continuam com as informações armazenadas. Exemplos são as memórias magnéticas, óticas e as memórias ROM, PROM e EPROM.

3. Escrita/Leitura: Exemplos de memória de escrita/leitura são as RAMs e as de apenas de leitura, as ROMs.

4. Armazenamento:

- Estáticas: são aquelas em que o dado inserido permanece armazenado indefinidamente. Como exemplo, armazenamento em flip-flop.
- Dinâmicas: é necessária a reinserção do dado periodicamente para que ele não se perca. Ex.: armazenamento em capacitores.

As memórias armazenam dados, na forma de bits, formando palavras (WORDS), essas palavras armazenadas representam alguma informação em uma determinada área da memória para ser lida posteriormente, essa informação será interpretada basicamente em função do tipo de dado ali armazenado, esses tipos de dados variam em função de diversas características, mas sempre no formato binário, embora tenha esse formato, podem produzir diferentes tipos de informação, conforme estudo a seguir:



4.5.1. Tipos de Dados

Os tipos dados servem para especificar e atribuir valores nas instruções durante a execução do programa desenvolvido pelo usuário, além de armazenar e organizar as informações na memória. Dentre os tipos de dados especificados pela norma IEC, temos:

Tipo de Dado	Descrição	Tamanho (em bits)	Range
BOOL	Booleano Falso ou Verdadeiro, 0 ou 1.	1	0 ou 1
SINT	Short Integer <i>Números inteiros curtos.</i>	8	$-(2^7)$ a $(2^7 - 1) =$ -128 a 127
INT	Integer <i>Números inteiros.</i>	16	$-(2^{15})$ a $(2^{15} - 1) =$ -32,768 a 32,767
DINT	Double Integer <i>Números inteiros duplos.</i>	32	$-(2^{31})$ a $(2^{31} - 1) =$ -2.147.483.648 a 2.147.483.648
USINT BYTE	Unsigned Short Integer <i>Números inteiros curtos, sem sinal.</i>	8	0 a $(2^8 - 1) =$ 0 a 255
UINT WORD	Unsigned Integer <i>Números inteiros, sem sinal.</i>	16	0 a $(2^{16} - 1) =$ 0 a 65,535
UDINT DWORD	Unsigned Double integer <i>Números inteiros duplos, sem sinal.</i>	32	0 a $(2^{31} - 1) =$ 0 a 4,294,967,295
REAL FLOAT	Números reais de precisão dupla.	32	Ver Norma IEEE-754
LREAL LFLOAT	Long Real ou Float Números reais de precisão dupla.	64	Ver Norma IEEE-754
TIME	Duração (em ms)	32	0 a $(2^{32} - 1) =$ 0 a 4,294,967,295 ms

4.5.1.1. Representação numérica e Números Negativos

Na tabela anterior, podemos identificar os tipos de dados que são interpretados pelos processadores, o computador lida com grandezas que são representadas em sequências de bits (na base 2), o que faz necessário a manipulação e conversão desses valores binários para bases que já são conhecidas para nosso entendimento. Como exemplo, para representar números inteiros, temos:

3 bits $2^3 \rightarrow 8$ números	Só valores positivos (Naturais)	Sinal e Magnitude
0 0 0	0	0
0 0 1	1	1
0 1 0	2	2
0 1 1	3	3
1 0 0	4	0
1 0 1	5	-1
1 1 0	6	-2
1 1 1	7	-3

onde o bit mais significativo representa 0 se positivo e 1 se negativo.

Essa representação possui dupla representação de valor 0 e dois circuitos, um para positivo e um para negativo. Assim, utilizamos a representação por complemento, em que a subtração entre dois números equivale à soma do primeiro com o complemento do segundo. Nesse caso, usa-se o complemento de 1: todos os bits de número são invertidos para representar seu complementar, convertendo, assim, um número positivo em negativo e vice-versa. Pela regra, o complemento de x (n bits, positivo) é dado por $x_c = (2^n - 1) - x$.

0 0 0	0
0 0 1	1
0 1 0	2
0 1 1	3
1 0 0	-3
1 0 1	-2
1 1 0	-1
1 1 1	0

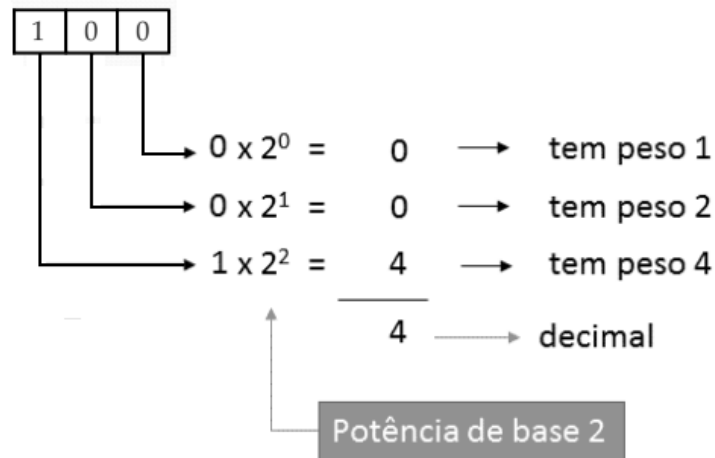
Por exemplo: utilizando 3 bits, o número 3 é representado por 011 em binário. Assim, seu complementar -3 é representado por 100 em binário. A partir da fórmula temos:

$$x_3 = (2^3 - 1) - 3$$

$$x_3 = (8-1) - 3$$

$$x_3 = 4$$

Conhecendo a transformação de número decimal para número binário, temos a representação do número -3 em binário (equivalente a 4 através da fórmula) como 100, da seguinte forma:



O complemento de 1 soluciona o problema do uso de dois circuitos, porém não elimina a redundância de 0. Desta forma, aplica-se o conceito do complemento de 2: todos os bits são invertidos e soma-se uma unidade para determinar o negativo de um número. Pela regra, o complemento de x (n bits, positivo) é dado por $x_c = 2^n - x$.

0	0	0	0
0	0	1	1
0	1	0	2
0	1	1	3
1	0	0	-4
1	0	1	-3
1	1	0	-2
1	1	1	-1

Positivos representáveis com n bits: 0 a $2^{n-1} - 1$

Negativos representáveis com n bits: -1 a -2^{n-1}

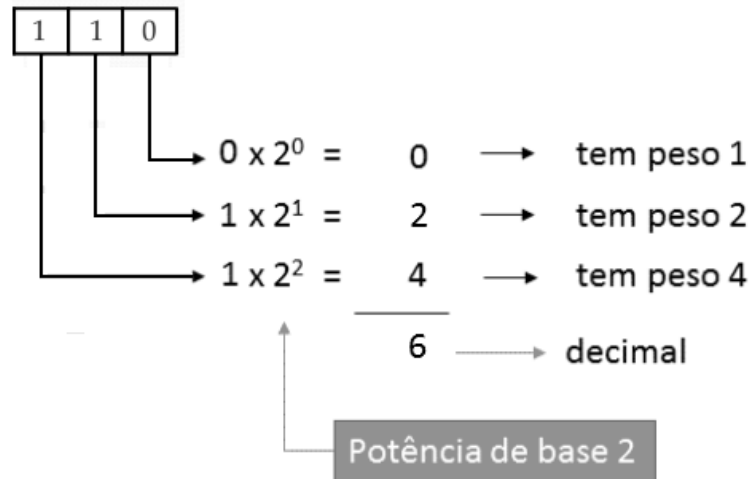
Por exemplo: utilizando 3 bits, o número 2 é representado por 010 em binário. Assim, invertendo todos os bits e somando 1, seu complementar -2 é representado por 110 em binário. A partir da fórmula, temos:

$$X_2 = 2^3 - 2$$

$$x_3 = 8 - 2$$

$$x_3 = 6$$

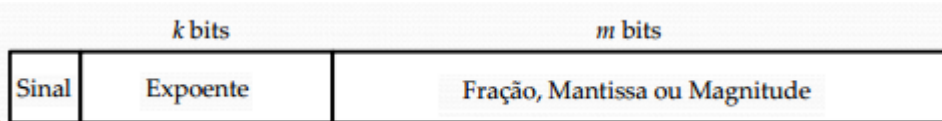
Conhecendo a transformação de número decimal para número binário, temos a representação do número -2 em binário (equivalente a 6 através da fórmula) como 110, da seguinte forma:



4.5.1.2. Norma IEEE-754

A Norma IEEE-754 foi publicada em 1985 com o intuito de unificar a representação de números em ponto flutuante, que é a forma padrão de expressar números reais em sistemas computacionais, assim como a maneira de operá-los. (Chouliaras & Nunez-Yanez, 2007)

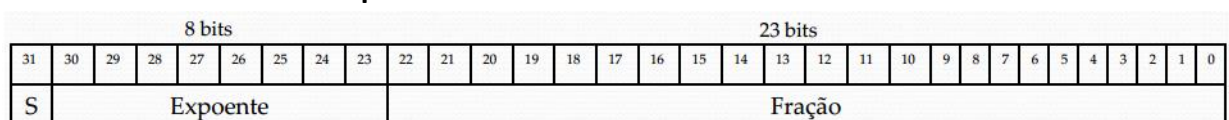
Neste tipo de representação, temos:



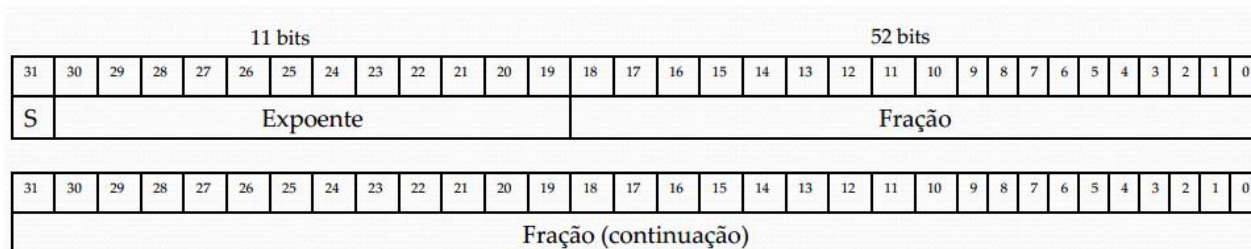
- o bit inicial indicando o sinal do número;
- o conjunto de bits a seguir é referente ao expoente, sendo positivo ou negativo, onde seu tamanho afeta a faixa de número que é possível representar;
- o último conjunto de bits é chamado de *fração*, *mantissa* ou *magnitude*, e determina a precisão da representação.

Assim, temos dois formatos: o simples e o duplo, com 32 e 64 bits, respectivamente:

➤ Formato simples – 32 bits:



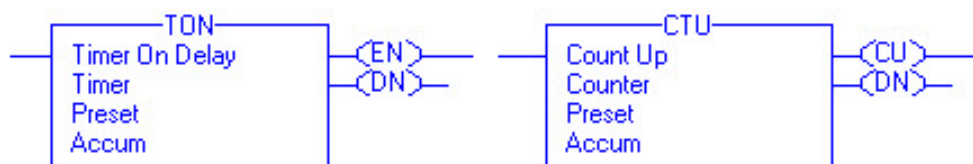
Formato duplo – 64 bits:



Desta forma, a representação em ponto flutuante está sujeita à ocorrência de dois eventos: o *overflow* e o *underflow*. **Overflow** é quando o expoente é grande de forma que não seja possível representa-lo em seu campo; **Underflow** é o oposto: quando o valor é tão pequeno que não é possível representá-lo, ou quando temos um expoente negativo tão grande que não cabe em seu campo.

4.5.1.3. Dados Estruturados

Além dos tipos de dados citados anteriormente, há também os **dados estruturados, customizados, strings e arrays**. Nos dados estruturados podemos ter dados mais complexos com informações em vários campos diferentes, ou seja, são várias variáveis agrupadas, e essas variáveis podem ser cada uma de um tipo de dado diferente, de acordo com a necessidade. Alguns exemplos práticos são os blocos de temporização e contagem, Timer e Counter, contido nos CLPs, onde são inseridas diversas informações necessárias para realização do controle no processo.



No TON por exemplo, essas variáveis internamente são representadas da seguinte forma:

Timer: *Timer_01* - representando o nome da variável, ou seja, seu endereço.

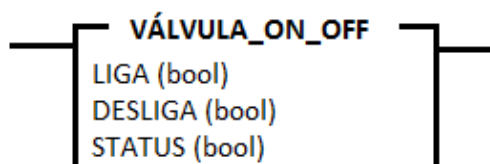
Preset: *Timer_01.PRE* – representando a quantidade de tempo, sendo do tipo Dint

Accum: *Timer_01.ACC* – representando quanto tempo já foi acumulado, sendo do tipo Dint

Timer_01.EN – representando a condição de funcionamento do Timer, sendo do tipo Bool.

Timer_01.DN – representando a condição de finalização da contagem de tempo, sendo do tipo Bool.

No tipo de dado customizado há a possibilidade de criação de blocos de instrução proprietários desenvolvidos pelo usuário, podendo ser inseridos espaços para alocação em memória de informações (variáveis) com os tipos de dados específicos para aquela realizar aquela instrução. Por exemplo: podemos criar um bloco de instrução chamado VÁLVULA_ON_OFF possuindo 3 variáveis como parâmetros (LIGA, DESLIGA E STATUS) do tipo que definirmos, nesse caso, booleano.



Assim, podemos criar variáveis do tipo VÁLVULA_ON_OFF a qualquer momento no programa, que possuirá LIGA, DESLIGA e STATUS como parâmetros para determinar sua saída.

O tipo de dado String é uma sequência de caracteres que pode representar palavras ou frases em um programa. Seu formato em memória depende do CLP, porém possui um tamanho de 85 bytes, geralmente tendo a seguinte estrutura:

Byte 0-1: offset e comprimento máximo (0 a 80 bytes);

Byte 2-3: comprimento atual;

Byte 4-83: caracteres que formam a(s) palavra(s);

Byte 84: terminador.

Arrays são estruturas que armazenam variáveis e dados em forma de arranjo, como em um vetor ou uma matriz, e geralmente possuem mesmo tamanho e tipo de dados. Por exemplo, pode-se criar um Array de dados do tipo inteiro (integer), com tamanho de 100 posições: **ARR_INT [0...99] of INT**, onde **ARR_INT** é o nome da variável, **[0...99]** indica seu tamanho e **INT** indica o tipo do dado. Desta forma, podemos a qualquer momento verificar ou inserir o valor em determinada posição do Array: **ARR_INT[0]** indica a primeira posição do array, **ARR_INT[1]** indica a segunda posição, e assim por diante.

4.5.2. Estrutura Interna e Mapa de Memória

Como citado anteriormente, uma memória armazena ou acessa informações, por meio de endereços, em lugares denominados localidades de memórias. Para o acesso a estas localidades, o bloco possui uma série de terminais de entrada de endereços que são ligados a um conjunto de fios denominado barra de endereços (address bus). Para

a entrada e saída dos dados, o bloco possui uma série de terminais ligados à barra de dados (data bus).

Além disto, o bloco possui terminais de controle ligados à barra de controle (control bus).

A figura 7 esquematiza uma memória eletrônica típica com os barramentos indicados.

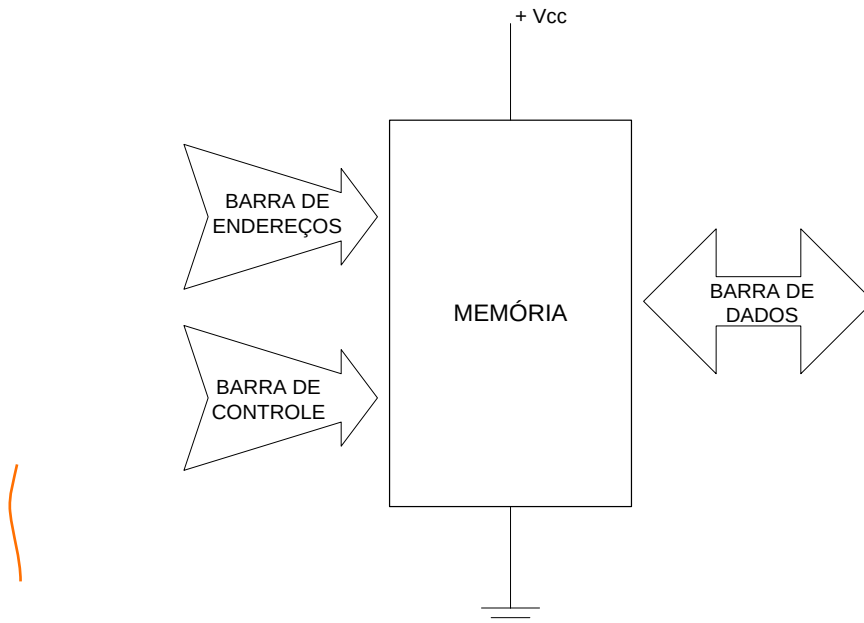


Figura 7 - Memória eletrônica típica com os barramentos indicados

A simbologia acima mostra que a barra de dados é bidirecional, isto indica que a memória em questão é do tipo leitura/escrita.

De forma geral, as memórias em termos da quantidade de dados armazenados, são especificadas pela notação $N \times m$, onde a primeira letra indica o número de localidades de memória e a segunda indica o número de bits de informação por localidade.

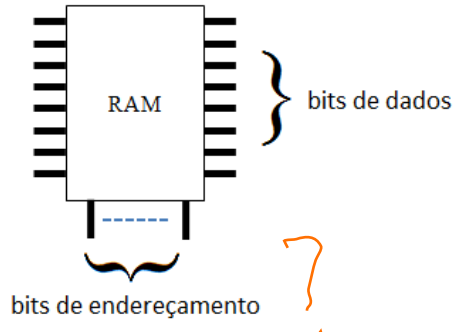
Por exemplo: 256x8; 1Kx16; 128Mx32, ...

A designação K (kilo) significa um fator $2^{10} = 1.024$, e a M (mega) significa um fator de $2^{20} = 1.048.576$.

Como exemplo, a memória 128Mx32 possui $128 \times 1.048.576 = 134.217.728$ localidades com 32 bits em cada uma, necessitando de 27 terminais para endereçamento.

4.5.3. Cálculo de Memória

A capacidade de armazenamento da memória, como o número de bits de endereçamento, de dados, ou número total de bits, da seguinte forma:



$$\text{TOTAL} = n^{\circ} \text{ bits de dados} * 2^{n^{\circ} \text{ bits de endereçamento}}$$

Exemplo: Sendo um total de 393.216 bits, e possuindo 12 bits de dados, temos:

$$393.216 = 12 * 2^x$$

$$\log_2 32.768 = X$$

$$2^x = \frac{393.216}{12}$$

$$X = \frac{\log 32.768}{\log 2} = 15$$

bits de endereçamento

$$2^x = 32.768$$

Exemplo2: Sendo um total de 128M x 32 bits, temos:

Sabendo que temos 32 bits de dados e 128M é a representação para $128 * 2^{20}$, temos:

$$128 * 2^{20} * 32 = 4.294.967.296$$

$$2^x = 134.217.728$$

$$4.294.967.296 = 32 * 2^x$$

$$\log_2 134.217.728 = X$$

$$2^x = \frac{4.294.967.296}{32}$$

$$X = \frac{\log 134.217.728}{\log 2} = 27$$

bits de endereçamento



4.5.4. Arquitetura de Memória

A arquitetura de memória de um controlador programável pode ser constituída por diferentes tipos de memória. A memória é onde se armazenam os dados que devem ser manipulados pelo computador, chamada **memória de dados** e também onde está armazenado o programa do computador, **memória de programa**.

Aparentemente não existe uma diferença física entre as memórias de programa, apenas utilizam-se memórias fixas para armazenar dados fixos ou programas e memórias que podem ser alteradas pelo sistema para armazenar dados que podem variar de acordo com o programa.

Existem diversos tipos de memórias que podem ser utilizadas pelo computador, dentre eles:

- **Memória ROM (*read only memory*):** memória apenas de leitura. São designadas como memória de programa por serem memórias que não podem ser alteradas em estado normal de funcionamento, ou seja, é escrita de forma definitiva pelo operador, podendo ser lida apenas pelo microprocessador. Porém, têm a vantagem de não perderem as suas informações mesmo na ausência de alimentação, o que a caracteriza como não-volátil.
- **Memória RAM (*random access memory*):** memória de acesso aleatório/livre. Pode ser tanto escrita como lida em qualquer momento no microprocessador. Se for desconectada da alimentação ocorre a perda das informações nela contidas, o que a caracteriza como memória volátil.
- **MEMÓRIA EAROM (*electrically alterable read only memory*):** memória não-volátil reprogramável. Pode ser alterada eletricamente, modificando alocações em memória, embora seja lenta.
- **MEMÓRIA EPROM (*erasable programmable read only memory*):** memória somente de leitura apagável e reprogramável. Possuindo característica não-volátil, pode ser lida apenas pelo microprocessador e, além disso, pode ser apagada expondo-a momentaneamente a raios ultravioleta, e reescrita pelo usuário com o uso de um aparelho específico, o programador de EPROM, elevando seu custo.
- **MEMÓRIA EEPROM (*electrically erasable programmable read only memory*):** memória somente de leitura apagável e reprogramável eletricamente. Possuem as mesmas características da EPROM, porém é apagada eletricamente. Embora seja parecida com a memória RAM, a escrita e leitura são bem mais lentos.



- **MEMÓRIA NOVRAM (non volatile RAM):** RAM não-volátil. A memória RAM pode ser transformada em não-volátil a partir do uso de acumuladores de forma a sustentar sua alimentação quando a energia é desligada, podendo manter suas informações por meses. Assim, associando-se as características da memória EEPROM, além de não ocorrer a perda de dados quando desenergizada, possui alta velocidade ao acessar dados, embora tenha o custo elevado.
- **MEMÓRIA FLASH:** substituta das memórias ROM e EPROM, possui alta confiabilidade, menor consumo, além da possibilidade de leitura e escrita através de sinais elétricos.

4.5.5. Estrutura do Mapa de Memória do CLP

Independente dos tipos de memórias utilizadas, o mapa de memória de um controlador programável pode ser dividido em seis áreas principais:

- **Memória Executiva**

É formada por memórias não-voláteis do tipo ROM ou EPROM, e em seu conteúdo está armazenado o sistema operacional responsável por todas as operações que são realizadas no CLP, além de gerenciar o funcionamento do hardware. O usuário não tem acesso a esta área de memória, ou seja, é somente de leitura.

- **Memória do Sistema**

Esta área é formada por memórias do tipo **RAM**, pois terá seu conteúdo constantemente alterado pelo sistema operacional. Armazena resultados e/ou operações intermediárias geradas pelo sistema quando necessário. Pode ser considerada como um tipo de rascunho e não pode ser acessada nem alterada pelo usuário.

- **Memória Imagem**

A memória imagem dos módulos de E/S são do tipo **RAM**. A UCP, após ter efetuado a leitura dos estados de todas as entradas, armazena essas informações na área denominada **status das entradas** ou **imagem das entradas**. Após o processamento dessas informações, os resultados serão armazenados na área denominada **status das saídas** ou **imagem das saídas**.

- **Memória de Dados**

As memórias de dados são do tipo **RAM**, e armazenam valores do processamento das instruções utilizadas pelo programa do usuário, ou seja, os dados de entrada e saída. Funções de temporização, contagem, aritméticas e especiais necessitam de uma área de memória para armazenamento de dados.



➤ Memória do Usuário

Esta área de memória é reservada para receber o programa desenvolvido pelo usuário. As instruções são registradas uma após a outra, e codificadas em uma sequência de bits. A UCP efetuará a leitura das instruções contidas nesta área a fim de executar o programa, de acordo com os procedimentos pré-determinados pelo sistema operacional. Essa memória pode ser do tipo RAM ou EEPROM, ou seja, são voláteis.

➤ Memória Retentiva

O conceito de memória retentiva se caracteriza pelo armazenamento de dados nessas determinadas áreas e, portanto, pela preservação dos dados mesmo após o desligamento da fonte de alimentação. Alguns CLPs possuem uma área de memória exclusiva para este fim, neste caso ao declarar uma variável esta deverá ser marcada como retentiva ou não para que o CLP defina a área onde esta variável será alocada.

Contudo, há fabricantes que não dividem estas áreas de memória, neste caso, todos os dados ficam sempre mantidos por baterias na memória RAM do CLP.

4.6. Cartões de Entrada e Saída

Os cartões de entrada e saída são circuitos destinados a tratar eletricamente os sinais para que possam ser processados pela CPU (ou microprocessador) do CLP, os sinais podem ser de entrada ou de saída, os de entrada recebem um sinal elétrico dos sensores ou chaves e converte para binário, os de saída convertem binário para sinal elétrico vindos do CLP para atuar nos elementos finais de controle.

Esses módulos funcionam a partir da leitura e acionamento de sinais lidos dos instrumentos e enviados aos instrumentos, de forma a monitorar e comandar ações ou eventos, como por exemplo, um interruptor, um pressostato, motores, válvulas, etc.

As entradas e saídas digitais são aquelas que possuem apenas dois estados possíveis, 0 ou 1, sendo $0 = 0V$ e $1 = V_{cc}$, sendo esse valor de tensão de acordo com a especificação de cada cartão do CLP (normalmente 24V), ou seja, estão ligados ou desligados, não há meio termo.

As entradas e saídas analógicas podem ter seu valor se alterando dentro de sua faixa de variação, ou seja, podem assumir quaisquer valores intermediários dentro um range como, por exemplo, o nível de um tanque, ou a abertura de uma válvula de controle.

Os cartões de uma forma geral, possuem memórias internas, que chamamos mapas de memória, as informações contidas nessas memórias são acessadas pela CPU, realizando a leitura, no caso dos cartões de entrada e a escrita, no caso dos cartões de saída.

DIAGRAMA
Ch → DADO
BIT
 $24V_{cc} / 0V_{cc} \rightarrow 1 / 0$

$4-20mA \Rightarrow$

Como a CPU só pode interpretar sinais binários, os sinais elétricos dos cartões passam por etapas de conversão desses para níveis lógicos (binários) para serem corretamente interpretados pela CPU.

4.6.1. Cartões de Entrada Digital

Cartões de entrada digital são interfaces para leitura de sinais discretos, logo, podem assumir apenas dois estados ou níveis: 0 ou 1, ligado ou desligado, aberto ou fechado, ou seja, enviam um sinal para a CPU como um dado *Booleano*, enviando nível 0 quando a entrada está desligada e nível 1 quando a entrada está energizada. Nos cartões de entrada discreta, normalmente são conectados dispositivos como botões, sensores, chaves, etc.

Os cartões discretos, de maneira geral, podem trabalhar em corrente contínua (24V) ou em corrente alternada (110 ou 220 Vca), como veremos nas figuras 8 e 9 a seguir, respectivamente:

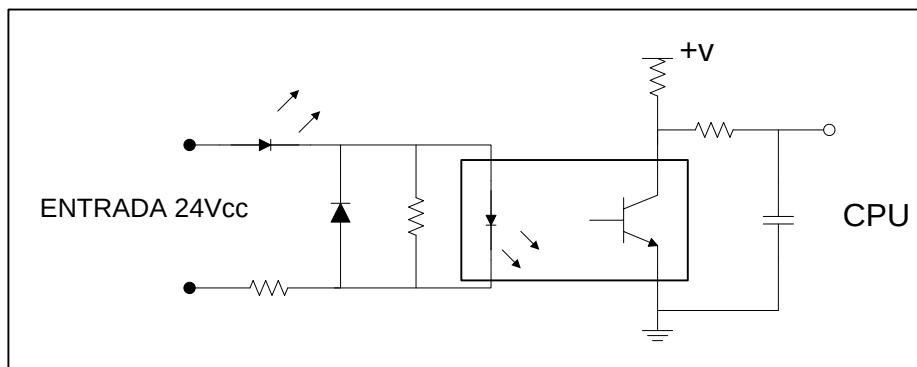


Figura 8 - Cartão de Entrada Digital com Alimentação em Corrente Contínua

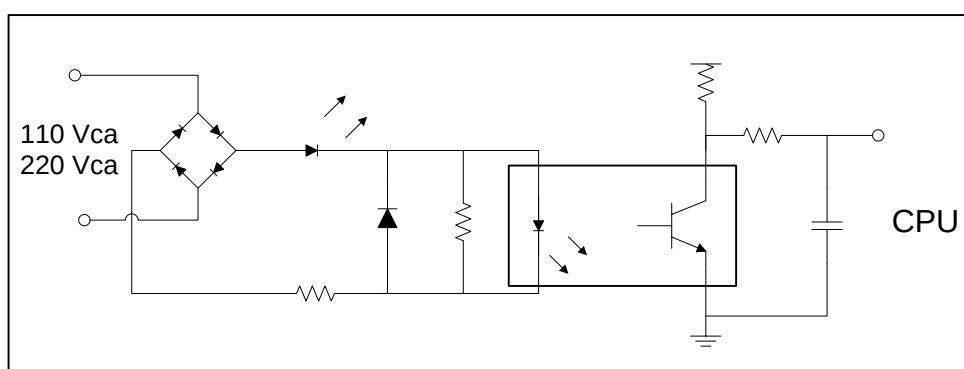


Figura 9 - Cartão de Entrada Digital com Alimentação em Corrente Alternada

Os cartões que trabalham com corrente contínua na sua entrada, podem ser do tipo N (PNP) ou P (NPN).

Os cartões do tipo N (ou PNP) são também chamados **Comum Negativo ou Sourcing**: a entrada do CLP reconhece o potencial negativo da fonte de alimentação

(terra ou neutro), ou seja, é necessário fornecer um potencial negativo proveniente da fonte para que as entradas sejam ativadas, conforme figura 10.

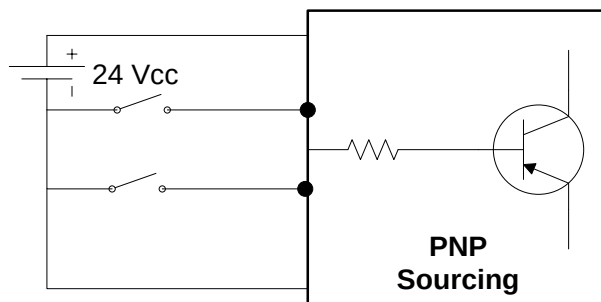


Figura 10 - Cartão de Entrada Digital do tipo PNP ou Sourcing

Os cartões do tipo P (ou NPN) são também chamados **Comum Positivo ou Sinkink**: a entrada do CLP reconhece o potencial positivo da fonte de alimentação, ou seja, é necessário fornecer um potencial positivo proveniente da fonte para que as entradas sejam ativadas, conforme figura 10:

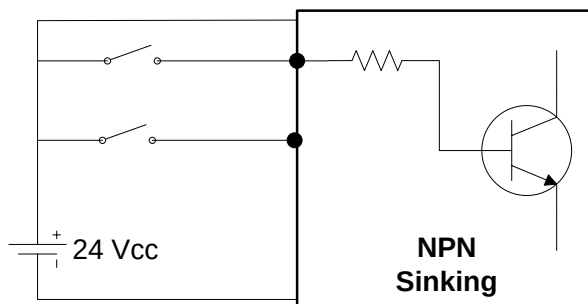


Figura 11 - Cartão de Entrada Digital do tipo NPN ou Sinking

Em geral, os cartões possuem um **Circuito Isolador (Isolação Galvânica)** entre o sinal elétrico e a CPU, utilizado para evitar que tensões ou ruídos indevidos cheguem a parte digital do cartão e eventualmente comprometa os componentes comuns do barramento e da CPU.

Os cartões de entrada digital também podem possuir internamente um circuito que funciona como **Diagnóstico de Fio Partido**, desta forma é possível diferenciar quando uma entrada está realmente desativada, devido ao contato aberto ou quando a entrada está desativada por causa de uma interrupção na ligação ou um fio partido, assim, não havendo esse recurso, quando o cartão indica nível lógico 0 (aberto) não há como ter certeza se está realmente em nível lógico 0 ou se a conexão foi interrompida.

Os cartões com esse tipo de recurso, possuem basicamente um resistor em paralelo com o contato de entrada, de modo que quando o contato está aberto, a corrente obrigatoriamente passa através desse resistor, ocorrendo uma queda no valor da

corrente, mas ainda assim há um valor de corrente. Quando temos o evento de fio partido, nenhuma corrente circula através do circuito, sendo assim seu valor igual a 0, indicando que o circuito está em estado de falha devido ao fio partido e não ao contato estar aberto.

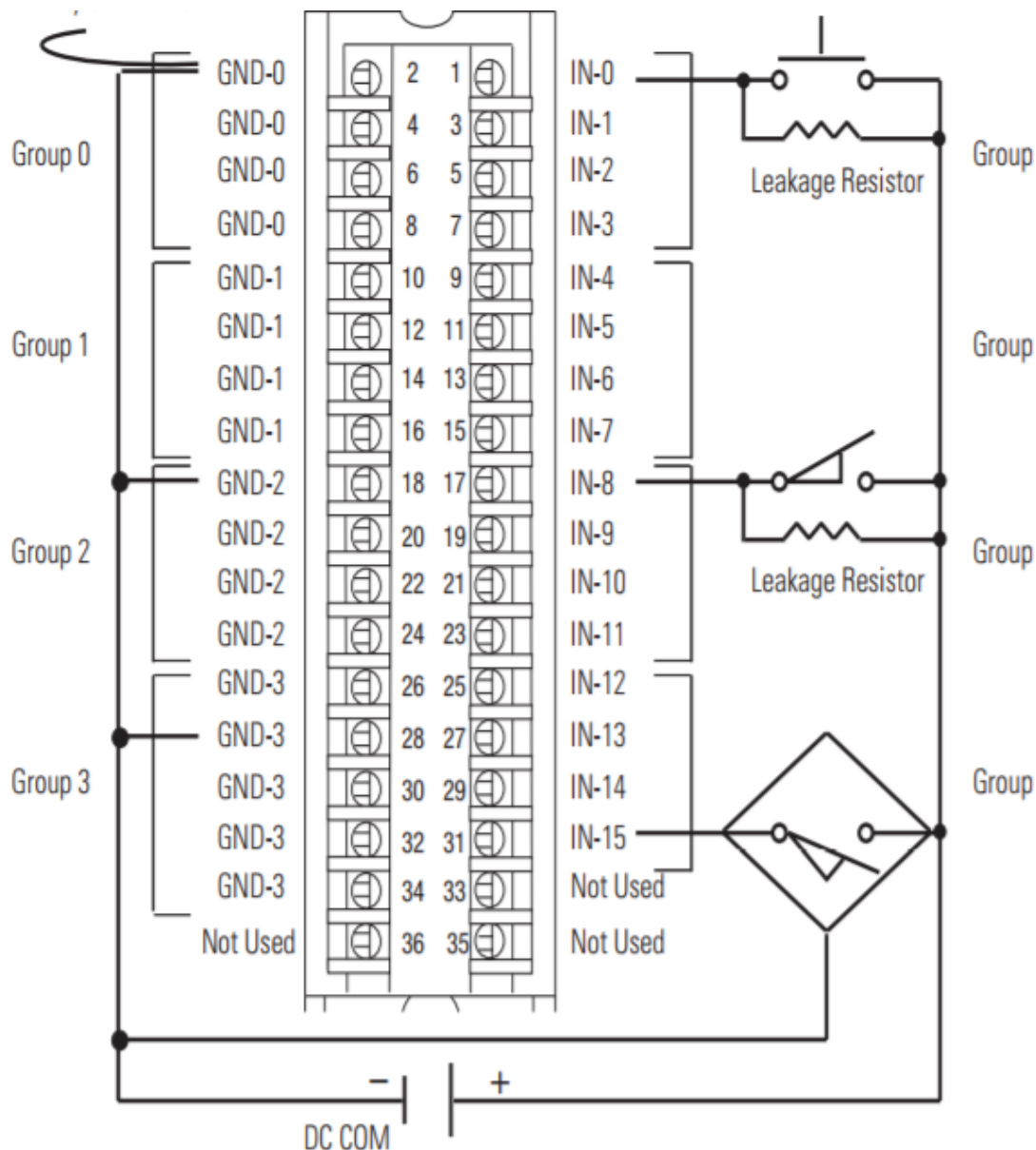
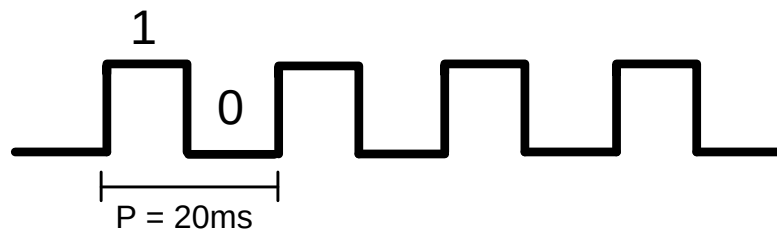


Figura 12 – Cartão de Entrada Digital com Diagnóstico de Fio Partido

Dentre os cuidados que devemos ter em relação aos cartões de entrada digital, temos:

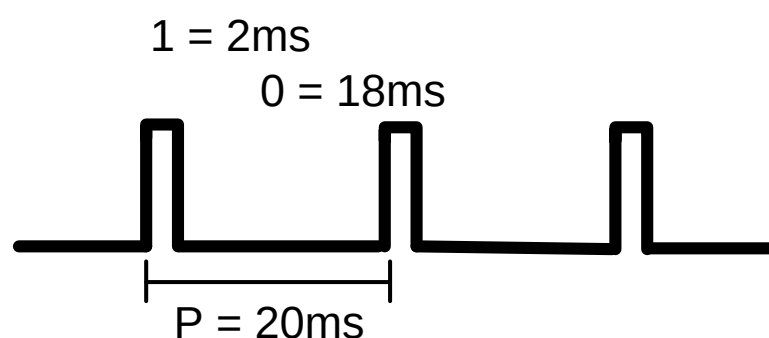
- **Nível de Tensão:** caso seja utilizada uma tensão diferente do nível que tensão que o cartão trabalha podem ser causados danos nesse cartão.

- **Frequência do Sinal de Entrada:** Sinais cuja origem é do tipo pulso ou trem de pulsos, devem estar dentro dos limites operacionais do cartão de entrada discreta, pois, esses possuem um limite de velocidade para leitura destes sinais. Os cartões convencionais podem ler sinais com frequência até 50Hz, porém, há cartões específicos para leitura de altas frequências, com valores de até faixas de 1, 10 ou 100KHz, esses são chamados cartões de entrada rápida, logo deve-se tomar cuidado ao usar sinais deste tipo na entrada, pois caso estejam fora da faixa operacional, pode-se haver perda de informação. Por exemplo:



Forma de onda quadrada com frequência de 50Hz, 20ms de Período, portanto tempo de permanência em 1 e 0 de 10ms..

- **Tempo de Permanência no Estado 0 ou 1:** além do cuidado com a frequência do sinal de entrada, deve-se atentar-se no tempo de permanência nos estados 0 ou 1, ou seja, por quanto tempo está em nível lógico 0 e em nível lógico 1, pois da mesma forma que pode haver perda de dados no caso de frequência alta, se o tempo de permanência no estado 0 ou 1, por exemplo, for muito pequeno, pode-se perder essa informação e o CLP entenderá como se estivesse continuamente em nível lógico 0 ou 1.



Forma de onda retangular com frequência de 50Hz, 20ms de Período, tempo de permanência em 1 de 2ms e 0 de 18ms.

4.6.2. Cartões de Saída Digital

Cartões de saída digital são interfaces para escrita de sinais discretos, logo, podem assumir apenas dois estados ou níveis: 0 ou 1, ligado ou desligado, ou seja, recebem um sinal da CPU como um dado Booleano, enviando nível 0 quando se deseja desligar a saída e nível 1 quando se deseja energizar a saída. Nos cartões de saída discreta, normalmente são conectados dispositivos como válvulas solenoide, alarmes, lâmpadas, contadoras, relés, etc.

Podem trabalhar com corrente contínua – 24V (transistor), com corrente alternada – 110Vca ou 220Vca (Triac) ou contato seco (relé).

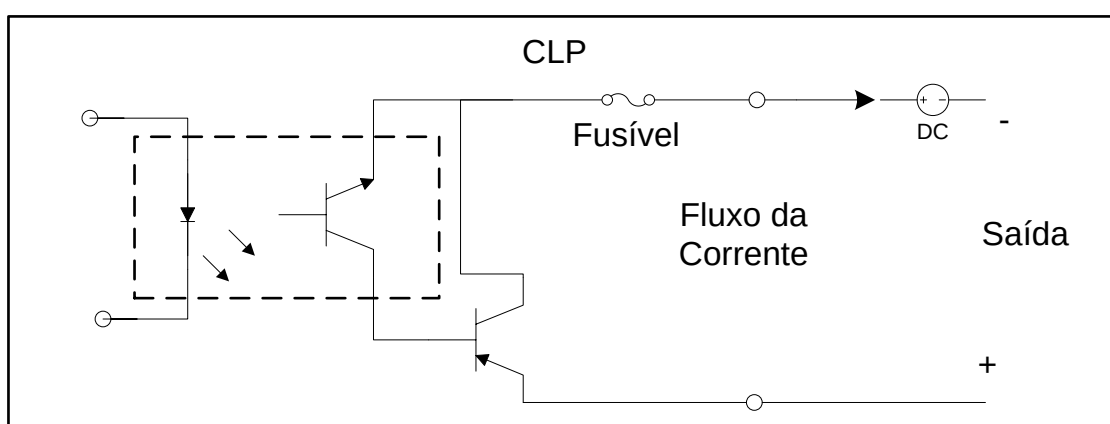


Figura 13 - Cartão de Saída Digital com Alimentação em Corrente Contínua - 24Vcc

Os cartões de saída digital em corrente contínua do tipo Transistor também podem ser do tipo PNP ou NPN, no qual o cartão entrega o potencial positivo ou negativo à carga, dependendo de seu tipo.

No tipo PNP, nesse caso também denominado **Sinking**, a carga recebe o potencial positivo proveniente do cartão e, portanto, deve estar ligada ao comum negativo da fonte, como mostra a figura 13.

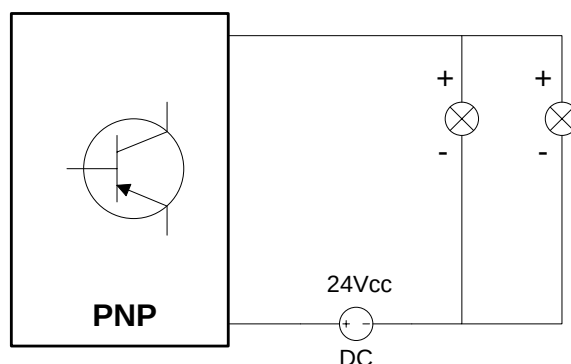


Figura 14 - Cartão de Saída Digital em Corrente Contínua do tipo Transistor PNP

No tipo NPN, que também é chamado de **Sourcing**, a carga recebe o potencial negativo proveniente do cartão e, portanto, deve estar ligada ao comum positivo da fonte, como mostra a figura 14.

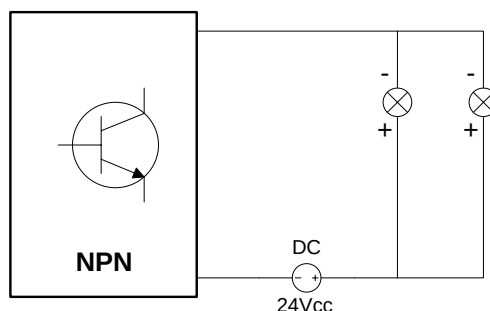


Figura 15 - Cartão de Saída Digital em Corrente Contínua do tipo Transistor NPN

Os cartões de saída digital do tipo alternado utilizam como elemento de chaveamento um Triac, esses trabalham a partir do acionamento de um terminal (Gate) e, a partir desse acionamento esse dispositivo conduz a corrente enquanto o seu Gate permanecer acionado.

O Triac (Triode For Alternating Current) possui dois retificadores controlados de silício, mais conhecidos como SCR ou tiristores, que são ligados em lados opostos e em paralelo com o gate, resultando em uma chave eletrônica bidirecional que é capaz de conduzir corrente nos dois sentidos quando o gatilho é acionado.

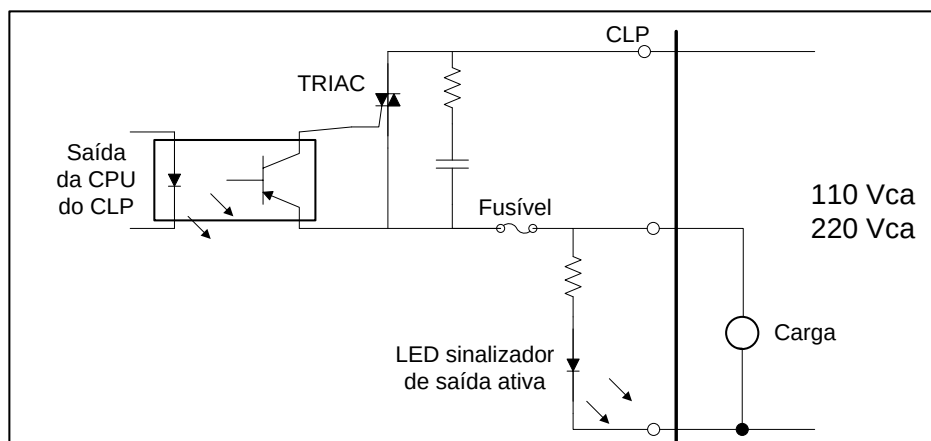


Figura 16 - Circuito Interno de Cartão de Saída Digital do Tipo TRIAC

Enquanto não houver corrente no gate, o Triac permanece em estado de bloqueio. Quando a corrente ultrapassa a corrente de disparo, o Triac inicia condução. Como trabalha em corrente alternada, o TRIAC pode ser disparado através de pulsos tanto positivos como negativos em seu gate, ou seja, em diversos pontos do sinal senoidal da corrente de alimentação, podendo aplicar potências diferentes em uma mesma carga a partir de um mesmo valor de corrente.

Se o disparo no gate ocorrer no início do semiciclo de corrente, a carga receberá uma maior potência, da mesma forma que se ocorrer no final do semiciclo, a carga receberá uma potência menor. Quando a corrente diminui e atinge um valor menor que o valor de corte, a condução é interrompida quando o sinal senoidal passa pelo 0.

Portanto, observando os gráficos a seguir, observamos que esse componente possui um comportamento especial, pois, basta que o Gate receba um pulso para que se inicie a condução de corrente, entretanto, essa interrompe quando cruza o potencial zero de corrente.

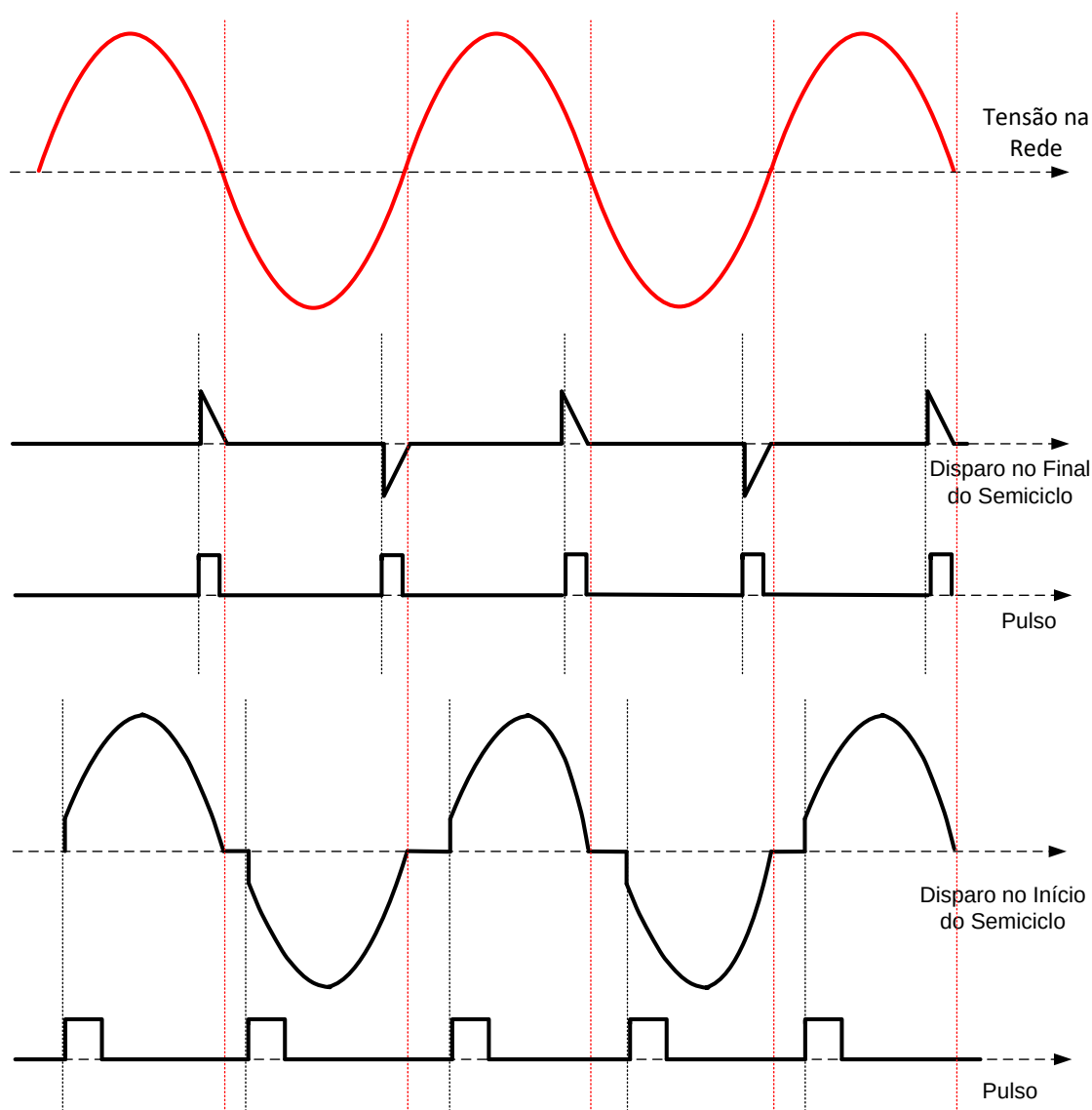


Figura 17 - Demonstração de disparo de pulso no Gate em momentos diferentes do semiciclo

Por tal característica, ao se usar cartões cujo elemento de chaveamento é um TRIAC, devemos observar, quando a saída da CPU interrompe o sinal lógico de 1 para 0, a saída elétrica pode não se desligar imediatamente, conforme figura 17, esse fenômeno ocorre devido ao fato do TRIAC só interromper a condução de corrente quando cruza o potencial 0 de tensão.

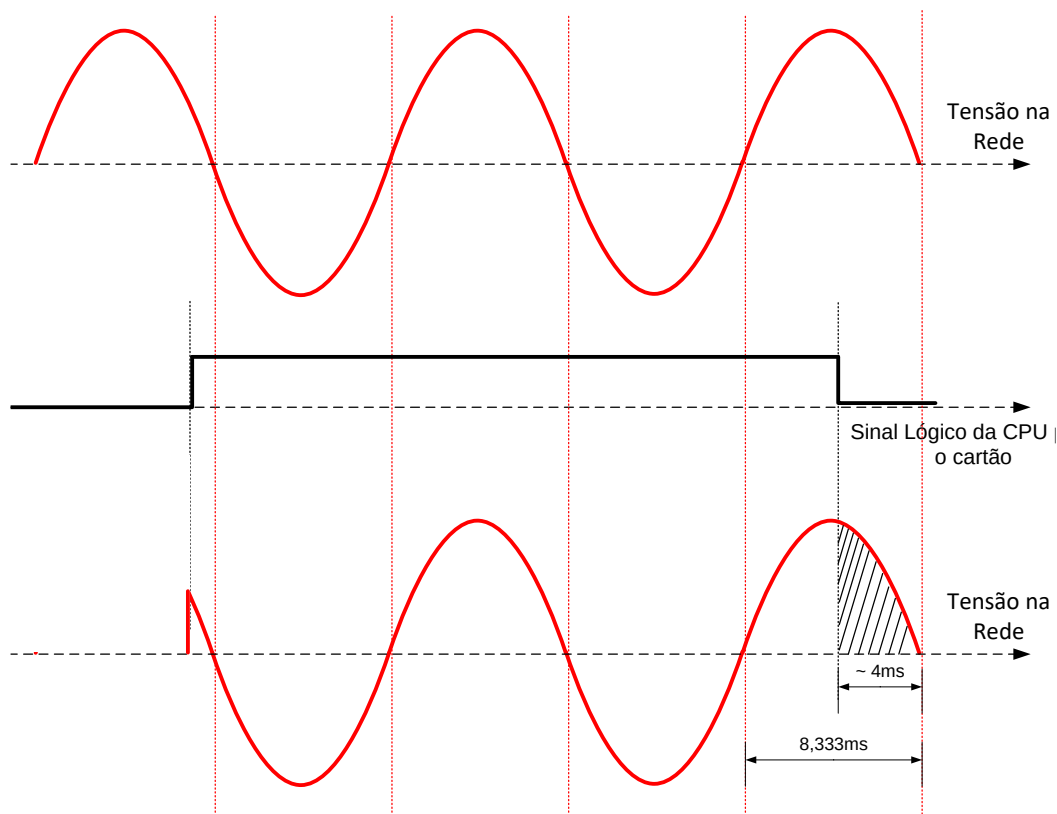


Figura 18 - Demonstração de atraso no desligamento da saída vs sinal lógico

Os cartões de saída com contato seco, possuem um Relé eletromecânico, podem atuar chaveando correntes contínuas ou alternadas. Seu funcionamento ocorre por meio do acionamento de um relé, que fica dentro do próprio cartão, que faz a comutação do(s) seu(s) contato(s), figura 19.

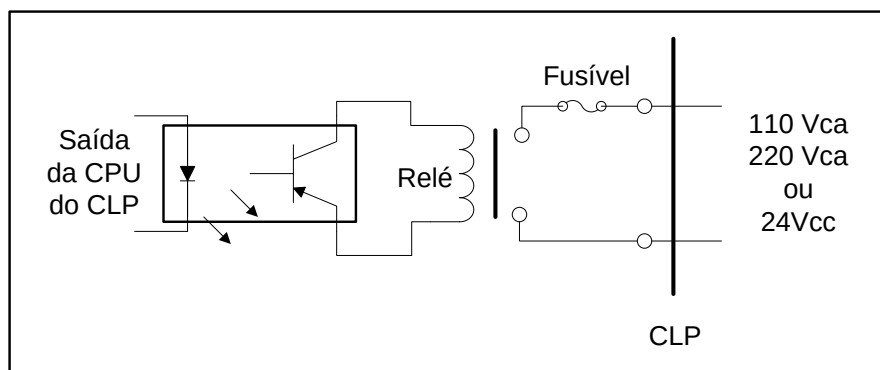


Figura 19 - Circuito Interno de Cartão de Saída do Tipo Relé

Utilizar relés ao invés de Triacs possui a vantagem de que o circuito de carga se encontra isolado do circuito de controle, o que possibilita o uso de diferentes níveis de tensão entre o controle e a carga. Porém, o desgaste mecânico pode ser uma desvantagem, pois, torna a vida útil do relé reduzida.

Assim como os cartões de entrada, os cartões de saída digital possuem **Circuito Isolador (Isolação Galvânica)** e podem possuir **Diagnóstico de Fio Partido**. No circuito do diagnóstico de fio partido há uma emissão de um pulso de corrente na carga, de modo que mesmo quando a saída estiver em nível lógico 0 gere uma corrente mínima para a carga para detectar que o loop está fechado, esta corrente não é suficiente para acionar a carga. Quando temos o fio partido, nenhuma corrente circula através da carga, sendo assim seu valor igual a 0, indicando que o circuito está em estado de falha devido ao fio partido.

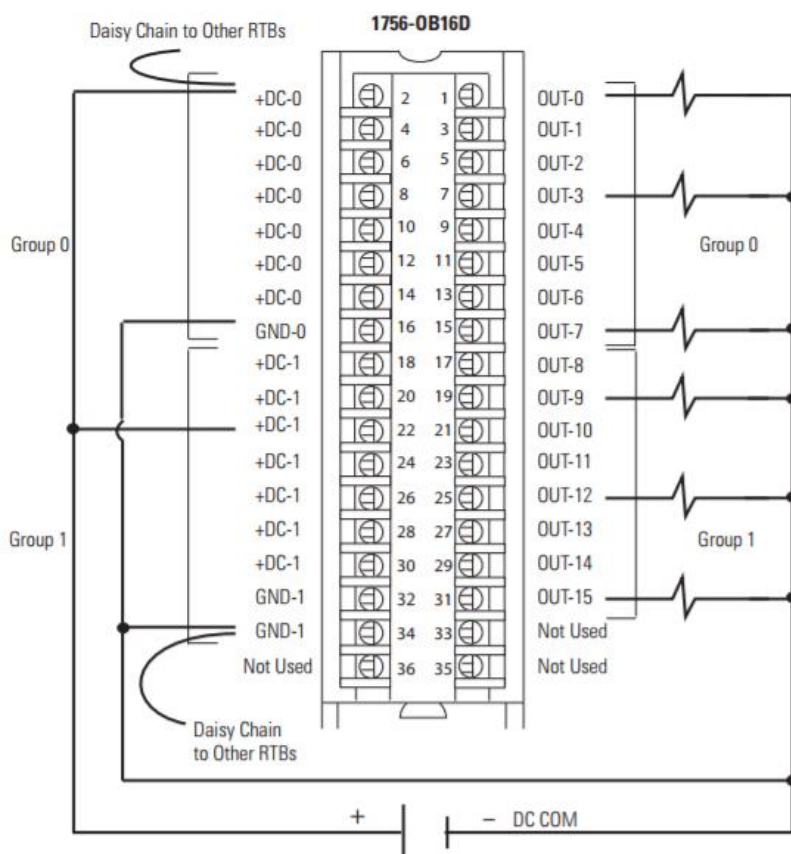


Figura 20 - Cartão de saída com Diagnóstico de Fio Partido

Os cuidados que devemos ter em relação aos cartões de saída digital são semelhantes aos cuidados para os cartões de entrada digital:

- **Corrente de Carga:** caso seja utilizada uma carga, cujo consumo de corrente ultrapasse os limites do cartão, essa pode causar danos na respectiva saída.

- **Frequência do Sinal de Saída:** os cartões possuem limites máximos para acionamento em pulsos, caso se deseje acionar uma saída em onda quadrada, por exemplo, deve ser observado os limites de frequência e tempo de estado 1 ou 0.
- **Acionamento de Cargas Indutivas (CC):** deve-se ter cuidado ao acionar cargas indutivas quando a saída for do tipo corrente contínua, pois estas podem causar a queima do componente interno do cartão, devido a um pico de tensão inversamente polarizado muito alto, gerado quando há o desligamento da carga, como mostra o gráfico abaixo:

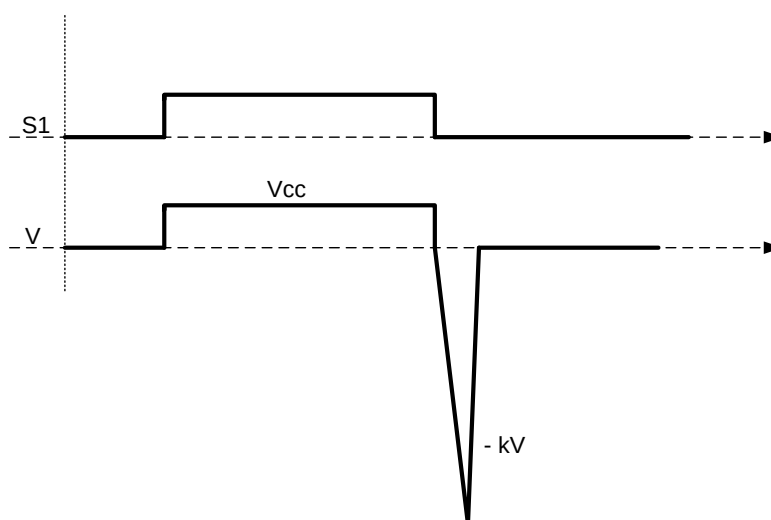


Figura 21 - Demonstração de acionamento de carga indutiva em saídas digitais sem proteção

Para solucionar este problema devemos inserir um diodo inversamente polarizado e em paralelo com a carga, de forma que esse diodo impeça o fluxo de corrente e evite danos no CLP, da seguinte forma:

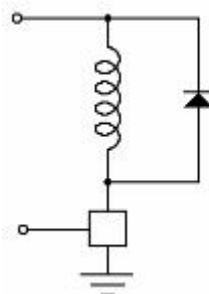


Figura 22 - Diodo inversamente polarizado em paralelo com carga indutiva

4.6.3. Cartões de Entrada e Saída Analógica

Sinais analógicos são sinais elétricos que representam grandezas físicas que variam dentro de um determinado range. Esses sinais trabalham em corrente contínua, possuindo valores padronizados de grandezas elétricas:

- Tensão: 0V a 10V, 0V a 5V, -10V a 10V.
- Corrente: 0mA a 20mA, 4mA a 20mA, -20mA a 20mA.

Como sabemos, processos industriais não são relacionados somente a variações binárias (on/off). Muitas vezes é preciso trabalhar também com essas grandezas físicas, como velocidade, temperatura, pressão, etc, que possuem um range pré-determinado pelo fabricante do CLP e varia de acordo com cada processo. Além disso, essas grandezas físicas podem ser transformadas em grandezas elétricas através de transdutores como, por exemplo, os termopares e termoresistências (RTD). Porém, os CLPs interpretam dados como grandezas digitais e, por isso, é necessário ocorrer a conversão desses valores através dos conversores A/D ou D/A. Essa conversão possui uma precisão de acordo com o número de bits do conversor, onde quanto maior o número de bits melhor a precisão e, portanto, a resolução do instrumento.

Nas entradas analógicas há a conversão dos sinais analógicos em sinais binários, para que ocorra a interpretação pelo CLP:

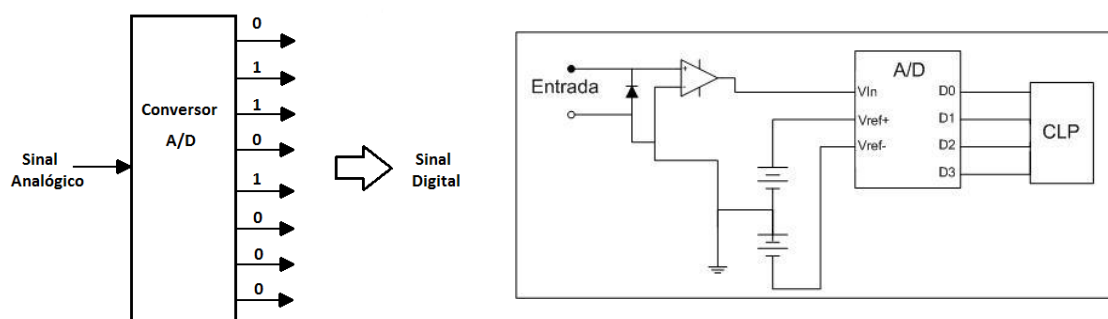


Figura 23 - Estrutura Básica e circuito de Conversão A/D

Já nas saídas analógicas ocorre o inverso: há a conversão de sinais binários provenientes do CLP em sinais analógicos, os quais serão enviados aos elementos finais de controle.

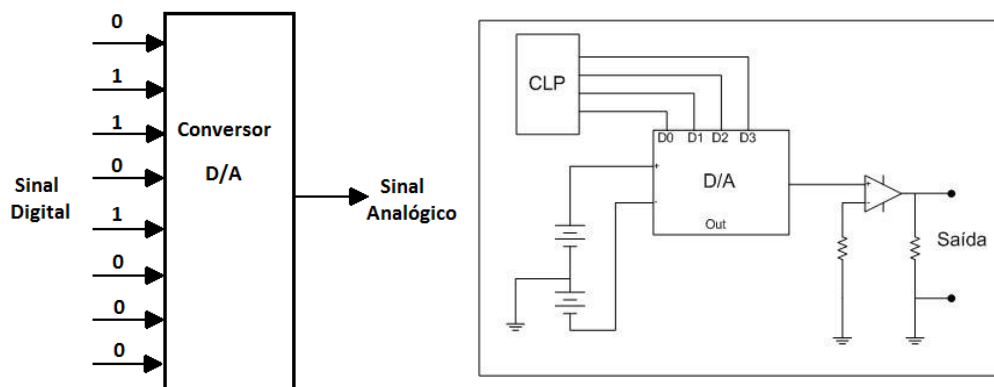


Figura 24 - Estrutura Básica e circuito de conversão D/A

4.7. Cálculo de Resolução

O cálculo da resolução ocorre utilizando a variação (range) do nível lógico, de acordo com o número de bits do cartão, e a variação de corrente (mA), unidade de engenharia (UE), ou porcentagem.

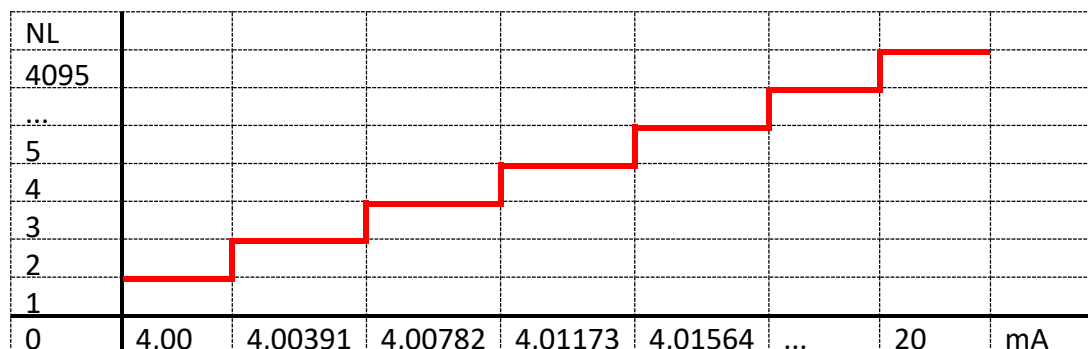
Como exemplo, em um cartão de 12 bits temos um range de 4096 possibilidades em nível lógico ($2^{12} - 1$, ou seja, de 0 a 4095). Assim, supondo que tenhamos um sinal de corrente de 4 a 20 mA, e variação de UE de 0 a 4000 PSI, podemos obter as seguintes resoluções:

$$\frac{\text{mA}}{\text{NL}} = \frac{20-4}{4095-0} = \frac{16}{4095} = 0,00391$$

$$\frac{\text{UE}}{\text{NL}} = \frac{4000-0}{4095-0} = 0,977$$

$$\frac{\%}{\text{NL}} = \frac{100-0}{4095-0} = 0,0244$$

Dessa forma, temos uma variação de 0,00391mA a cada variação de nível lógico, como mostra o seguinte gráfico:





Os cartões analógicos podem ser adquiridos com uma resolução específica, isso tem relação direta com o custo do cartão, portanto é importante definir a resolução em função da necessidade da aplicação.

Portanto, o número de bits de resolução e a faixa de leitura do cartão determinam a resolução final, sendo possível diversas combinações entre eles, conforme exemplo abaixo:

Bits de Resolução	Faixa de Entrada ou Saída	Instrumento	Cartão	Resolução mA/NL	Resolução %/NL
12 bits	4 a 20 mA	4 mA	0 NL	0,0039	0,024
		20 mA	4095 NL	mA/NL	%/NL
12 bits	0 a 20 mA	4 mA	819 NL	0,0049	0,031
		20 mA	4095 NL	mA/NL	%/NL

Logo, a resolução mínima deve ser definida para que o número de bits de resolução seja calculado antes de se definir a compra do cartão analógico, conforme exemplo a seguir:

Num processo, onde há uma medição de vazão, cuja faixa de calibração do instrumento é de 0 a 500 m³/h com saída 4 a 20 mA e cartão com leitura também em 4 a 20 mA, deseja-se obter leituras de vazão cujo valor seja maior que 0,02 m³/h, temos a seguinte solução:

Cálculo:

$$0,02 = \frac{500 - 0}{(2^n - 1)}$$

$$(2^n - 1) = \frac{500}{0,02}$$

$$(2^n - 1) = 25.000$$

$$2^n = 25.001$$

$$n = \frac{\log 25001}{\log 2}$$

$$n \cong 14,61$$

Portanto o cartão deve ter resolução de pelo menos **15 bits**, para que a leitura seja no máximo 0,02 m³/h.



4.7.1. Instalação e troca dos Cartões

Eventualmente, um cartão pode precisar de uma manutenção ou mesmo ser trocados, devido a queima ou mau funcionamento, esta troca pode ser feita de duas, sendo uma como “troca a frio” e a outra como “troca a quente”, no primeiro caso o barramento onde se encontra o cartão deve estar desligado, para evitar problemas na eletrônica do próprio cartão ou do barramento, no segundo caso isso pode ser feito com tudo ligado e isso não deve parar o funcionamento dos outros cartões adjacentes, esta última característica é chamada Hot Swap, e deve ser observada no momento da especificação dos componentes.

4.8. Classificação quanto ao porte

Os CLPs podem ser classificados de acordo com seu porte: micro porte, pequeno porte, médio porte e grande porte. Esse é definido de acordo com algumas características específicas e diretamente proporcionais umas às outras, conforme a seguir:

- Velocidade de processamento;
- Memória;
- Quantidade de I/Os;
- Capacidade de redundância.

Cada fabricante define seus respectivos modelos de CLP dentro de cada um dos portes mencionados anteriormente. O tipo de processo industrial e suas características definirão o porte a ser utilizado.

CLPs de micro pequeno porte são geralmente utilizados em processos simples, como em automação residencial e predial, pequenos processos industriais, etc, normalmente caracterizam-se por serem equipamentos de baixo custo e com menos recursos tecnológicos.

Os CLPs de médio e grande porte são desenvolvidos para atender processos industriais e sistemas de controle mais complexos, que necessitam de um maior número de entradas e saídas.

4.8.1. Velocidade de Processamento

Define a capacidade de processar determinada quantidade de instruções em um determinado tempo, como exemplo podemos citar 1K de instruções a cada 1ms.

4.8.2. Memória

A quantidade de memória está relacionada normalmente à tamanho de programa, conseqüentemente, quantidade de instruções, códigos de programas e quantidades de variáveis internas ou externas.

4.8.3. Quantidade de I/Os

Os I/Os de um CLP são as interfaces responsáveis pela troca de sinais dos sensores e atuadores com a CPU. Esses I/Os podem ser embutidos, modulares ou remotos:

CLP com I/O embutido: quando todos os I/Os são integrados à CPU do CLP em um único módulo.

CLP com I/O modular: constituído de um ou mais racks que permitem a expansão e configuração de I/Os de acordo com a necessidade do usuário.

CLP com I/O distribuído: constituído por módulos de redes com I/Os integrados ou modulares que se comunicam com a CPU através de um protocolo de programação específico (uma rede Fieldbus).

Embora os CLPs possam ser separados de acordo com as classificações citadas acima, atualmente, existem no mercado controladores programáveis que englobam todas essas características ao mesmo tempo. Como exemplo, existem CLPs que possuem I/Os embutidas, ao mesmo tempo possui capacidade de expansão através de cartões modulares de entradas e saídas, e também pode trabalhar com I/O distribuído através de uma rede Fieldbus.

4.8.4. Redundância por Software e Hardware

Em todo e qualquer processo de automação industrial é de suma importância que haja a máxima confiabilidade e disponibilidade nos sistemas de controle, uma vez que qualquer tipo de falha provocada pelo sistema pode vir a trazer danos ao processo. Tais falhas podem ser tanto provenientes de erros de software quanto de hardware, como por exemplo erros de programação, conexão de rede, fluxo de dados, cabeamento, degradação física, etc.

De forma a prevenir essas falhas e evitar os prejuízos impostos pela interrupção no controle, é aplicada a técnica de redundância dos sistemas, que é o meio mais simples e fácil de se alcançar uma alta disponibilidade. Redundância nada mais é que o uso de um segundo recurso físico ou lógico que está disponível para assumir o controle do processo imediatamente em caso de falha no sistema primário, de modo que não haja necessariamente a interrupção do controle. Com isso, temos dois tipos de redundância: a redundância por software e a redundância por hardware.

A redundância por hardware é baseada na duplicação de componentes físicos, e pode ser feita, por exemplo, através de um cartão específico, sem a necessidade de se criar códigos específicos para tal função.

Os programas ficam armazenados nas duas CPUs e um cartão específico para redundância monitora e gerencia os mapas de memória, garantindo a atualização permanente nos dois lados, assim, quando a CPU em standby assume, as informações já estão atualizadas e o processo não para.

A figura 25 abaixo representa esta arquitetura redundante.

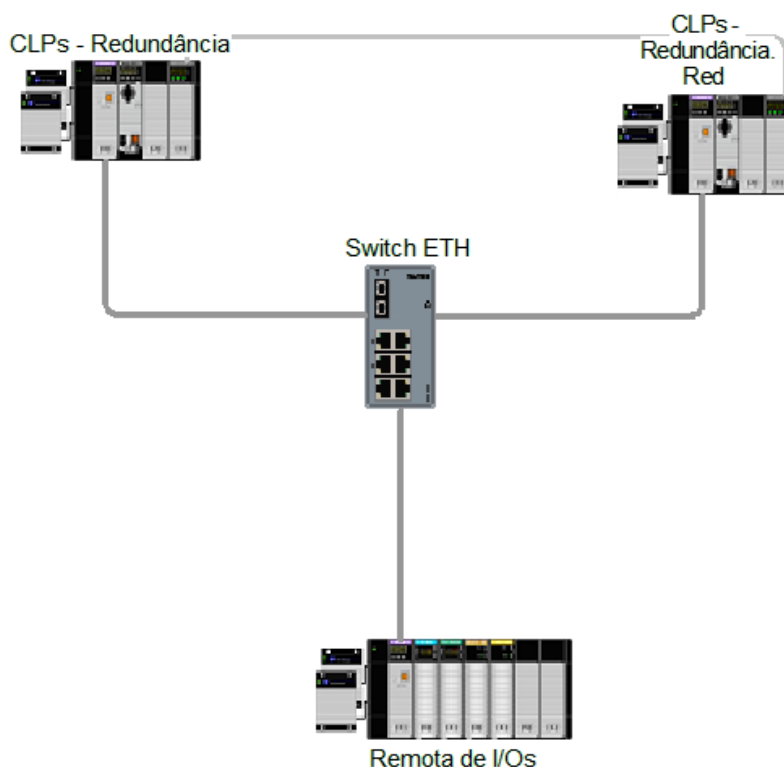


Figura 25 - Exemplo de Redundância de Hardware

A redundância por software ocorre através de dois CLPs distintos, normalmente idênticos, interligados através de uma rede de comunicação. O tratamento da redundância é feito dentro do programa de usuário, através de instruções específicas, que são configuradas e parametrizadas durante o desenvolvimento do programa do usuário, e por isso é menos robusta. O gráfico abaixo descreve o princípio da redundância por software:

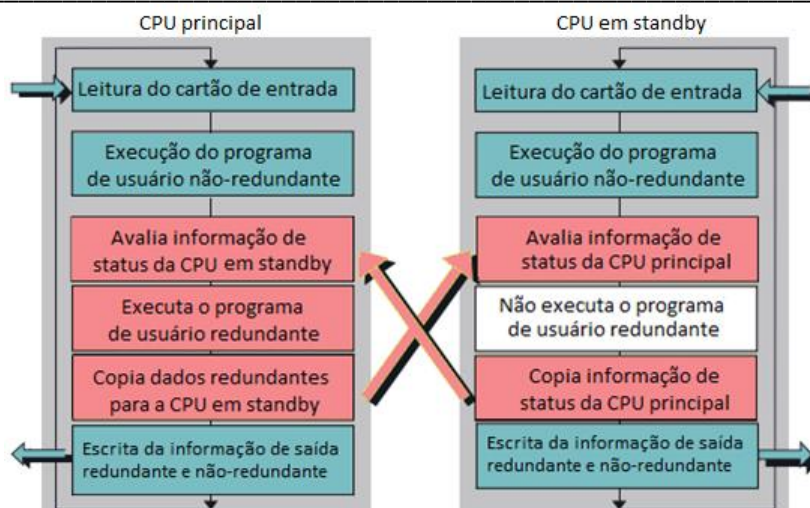


Figura 26 - Diagrama de fluxo que descreve o processo de redundância por Software

Em condições normais de processamento, a CPU principal faz a leitura dos cartões de entrada, executa o programa de usuário e escreve a informação de saída nos cartões de saída.

Em caso de falha na CPU principal, para evitar que o programa da CPU redundante comece do início, a CPU principal transfere continuamente as informações de processo atuais para a estação em standby. Para isso, o programa é carregado no programa em ambas estações: na CPU principal e na CPU em standby (redundante). Enquanto ocorre seu processamento na CPU principal, esta etapa é ignorada pela CPU em standby, de modo que a CPU em standby esteja sempre preparada para assumir o controle do processo em caso de falha na CPU principal.

4.9. Software de Programação

O software de programação é um componente nesse contexto que permite o desenvolvimento do programa do usuário e a configuração do sistema. Atualmente, é utilizado em PCs convencionais. Neste software são feitas as codificações das informações vindas do usuário numa linguagem que possa ser entendida pelo processador de um CLP. Algumas funções podem ser realizadas, tais como:

- Elaboração do programa do usuário;
- Análise do conteúdo dos endereços de memória;
- Introdução de novas instruções;
- Modificação de instruções já existentes;
- Monitoração do programa do usuário;
- Verificação do estado de funcionamento do hardware do CLP;
- Atuação de saídas independente da lógica (force);
- Cópia do programa do usuário em disco ou impressora.



5. PRINCÍPIO DE FUNCIONAMENTO DE UM CLP

Um controlador lógico programável tem seu funcionamento baseado num sistema de microcomputador, onde se tem uma estrutura de software que realiza continuamente ciclos de varredura.

5.1. Funcionamento

A estrutura de um CLP pode dividida em três etapas: entrada, processamento e saída.

Basicamente, a cada leitura das entradas os dados enviados pelos sinais dos sensores/transdutores são transferidos e armazenados na memória imagem de entrada do CLP. Estes sinais passam pelo processamento de acordo com o código de programação desenvolvido pelo usuário e, em seguida, os dados de saída são também transferidos e armazenados na memória imagem de saída do CLP, para então serem aplicados externamente no comando dos atuadores.

Todo esse processo de leitura e escrita de dados de entrada e saída constituem o ciclo de scan, também chamado de ciclo de varredura ou loop, ou seja, é quando ocorre um ciclo completo de operação. O tempo necessário para ocorrer esse loop é denominado tempo de scan ou tempo de varredura, e depende do tamanho do programa do usuário e da quantidade de pontos de entrada e saída.

Desta forma, se o tempo de varredura é muito grande, ou seja, se o processamento é muito longo, pode ocorrer a perda de dados, uma vez que as entradas podem ter seu estado variando de 0 pra 1 e novamente de 1 pra 0 enquanto o CLP ainda se encontra em processamento e não consegue capturar a informação desse novo estado de entrada. Assim, ao perder informações de entrada o CLP não gera a saída (instrução) necessária para essa determinada informação.

Por isso, é importante sempre verificar o tempo de scan na escolha dos CLPS, visando sempre escolher um com a velocidade e porte suficientes para determinados processos. Porém, para evitar a geração de loops repetitivos devido à perda de dados de entrada pelo alto tempo de processamento do programa, existe um timer chamado *Watch Dog Time* que supervisiona o tempo de processamento. Esse tempo geralmente é estipulado a um valor de em média 150 ms, e ocorrendo a ultrapassagem deste tempo máximo, o funcionamento da UCP é imediatamente interrompido, sendo assumido um estado de erro, podendo ser emitido um sinal de alarme visual e até sonoro para indicar falha.



Figura 27 – Ciclo de operação do CLP

Em uma explicação mais detalhada, vemos que, ao ser energizado (estando o CLP no estado de execução), o mesmo cumpre uma rotina de inicialização gravada em seu sistema operacional. Esta rotina realiza a limpeza da memória imagem, o teste de memória RAM e o teste de executabilidade do programa.

Após a execução desta rotina, a UCP passa a fazer uma varredura (ciclo) constante, isto é, uma leitura sequencial das instruções em loop (laço).

Entrando no loop, o primeiro passo a ser executado é a leitura dos pontos de entrada. Com a leitura do último ponto irá ocorrer a transferência de todos os valores para a memória ou tabela imagem das entradas. Após a gravação dos valores na tabela imagem, o processador inicia a execução do programa do usuário de acordo com as instruções armazenadas na memória.

Terminando o processamento do programa, os valores obtidos neste processamento serão transferidos para a memória ou tabela imagem das saídas, como também a transferência de valores de outros operandos, como resultados aritméticos, contagens, etc.

Ao término da atualização da tabela imagem, será feita a transferência dos valores da tabela imagem das saídas para os cartões de saída, fechando o loop. Neste momento é iniciado um novo loop.

Durante a execução do ciclo de varredura ocorre a leitura das entradas e atualização da memória imagem de E/S. A figura seguinte ilustra como esse processo funciona para entradas digitais. Estando o ponto entrada energizado (contato fechado) o bit correspondente da memória imagem ficará em nível lógico 1.

Caso o contato esteja aberto o bit correspondente na memória imagem ficará em nível lógico 0. Observe que esses estados independem se o contato de campo é normalmente aberto (NA) ou normalmente fechado (NF). Na sequência da execução do ciclo de varredura é executado o programa do usuário que, entre outros, utilizará os dados da memória imagem de E/S. Após o resultado será escrito na tabela da memória imagem de saída. Observando a lógica do programa do usuário apresentado na figura

28, pode ser observado que no estado atual das entradas (IN 00 acionada, nível lógico 1 e IN 02 desativada, nível lógico 0), o programa acionará o bit correspondente a saída digital 05.

Pode ser observado que o programa acionou a saída mesmo com uma das entradas físicas desativadas. Isso se deve ao fato que o contato NF no programa corresponde a lógica de negação, então considerando a lógica combinacional tem-se:

$$OUT05 = IN00 * \overline{IN02}$$

Portanto, a saída digital 05 só será acionada quando $IN00 = 1$ E $IN02 = 0$. (VIANNA, 2008)

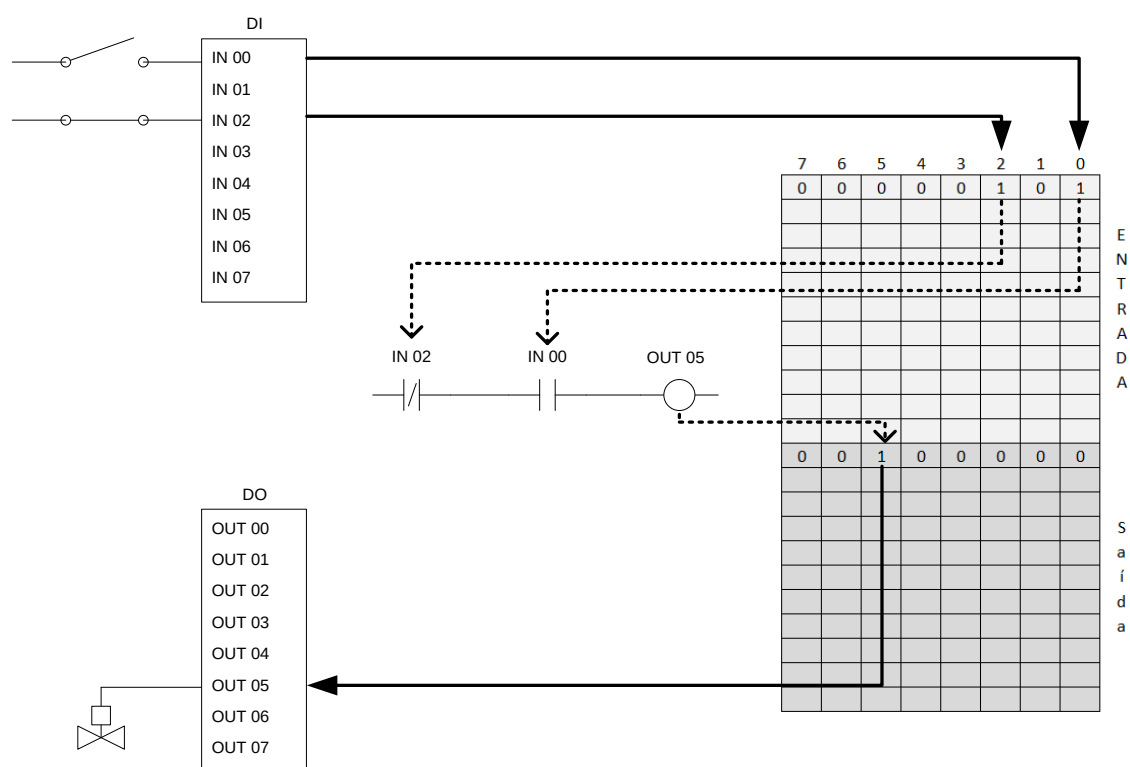


Figura 28 – Ciclo de operação do CLP

5.2. Estados de Operação

Basicamente, a UCP de um CLP pode assumir um estado de erro, que aponta falhas de operação e execução do programa, além de possui 2 estados de programação:

▪ Programação (STOP)

Neste estado o CP não executa o programa, ou seja, não assume nenhuma lógica de controle. Assim, ele fica preparado para ser configurado ou para receber novos programas ou modificações em programas já inseridos em sua memória. Esse tipo de programação é chamada de *off-line*.

▪ Execução (RUN)

Neste estado o CP assume a função de execução do programa desenvolvido pelo usuário. Porém, em alguns controladores, pode-se fazer modificações nos programas mesmo quando este está em execução, sendo este tipo de programação chamada *on-line*.

Assim, quando uma lógica é modificada e enviada para a memória do CLP, o novo programa entra em operação, contudo o seu estado não muda e o ciclo continua em operação.

5.2.1. Teste de Circuito Interno e Proteção de Dados

A cada inicialização de um novo loop, o CLP executa um auto teste para verificar o funcionamento de todo seu circuito interno e, caso haja alguma falha ou alguma anormalidade em seu sistema operacional, como por exemplo falha na memória ou no microprocessador, um sinal de alarme é enviado para advertir o operador sobre tais irregularidades. Além disso, em situações que ocorram a falta de energia, todas as memórias são alimentadas automaticamente pelo sistema operacional através das baterias de backup, de modo a evitar a perda de dados.

6. Linguagens de Programação para CLP

A linguagem de programação é uma ferramenta necessária para gerar o programa, que vai coordenar e sequenciar as operações que o microprocessador deve executar. (VIANNA, 2008)

6.1. Classificação

As linguagens de programação podem ser classificadas em dois grandes grupos:

- Linguagem de baixo nível
- Linguagem de alto nível

A programação do CLP, de acordo com a norma, é realizada através de linguagens de alta nível, utilizando até 6 linguagens específicas, são elas:

- Texto Estruturado (ST)
- Lista de Instruções (IL)
- Ladder Diagrama (LD)
- Diagramas de Blocos Funcionais (FBD)
- Funções Gráficas de Sequenciamento (SFC)
- Funções Gráficas Contínuas (CFC) – extensão da IEC 61131-3, que dá liberdade à posição dos elementos do gráfico.

As características de cada uma delas são apresentadas a seguir:

6.1.1. Texto Estruturado (ST)

É uma linguagem textual que possui base da informática, muito semelhante à utilizada na elaboração de programas para computadores. Utiliza estruturas do tipo IF-THEN, DO-ELSE, FOR, WHILE, entre outras, e por isso é altamente recomendada para desenvolvimento de lógicas mais complexas.

Exemplo:

```
(* ANIMAÇÃO DA PORTA DO IBC *)

TONR(TONR_1);
TONR_1.Reset := 0;

IF TONR_1.DN AND ZZ_2951 THEN
  IF PORTA_IBC_OPEN < 5 THEN
    PORTA_IBC_OPEN := PORTA_IBC_OPEN + 1;
    TONR_1.Reset := 1;
  END_IF;
END_IF;

IF TONR_1.DN AND (NOT ZZ_2951) AND (NOT ZS_2950_2) THEN
  IF PORTA_IBC_OPEN > 0 THEN
    PORTA_IBC_OPEN := PORTA_IBC_OPEN - 1;
    TONR_1.Reset := 1;
  END_IF;
END_IF;

(* ANIMAÇÃO DA ESTEIRA DO IBC *)

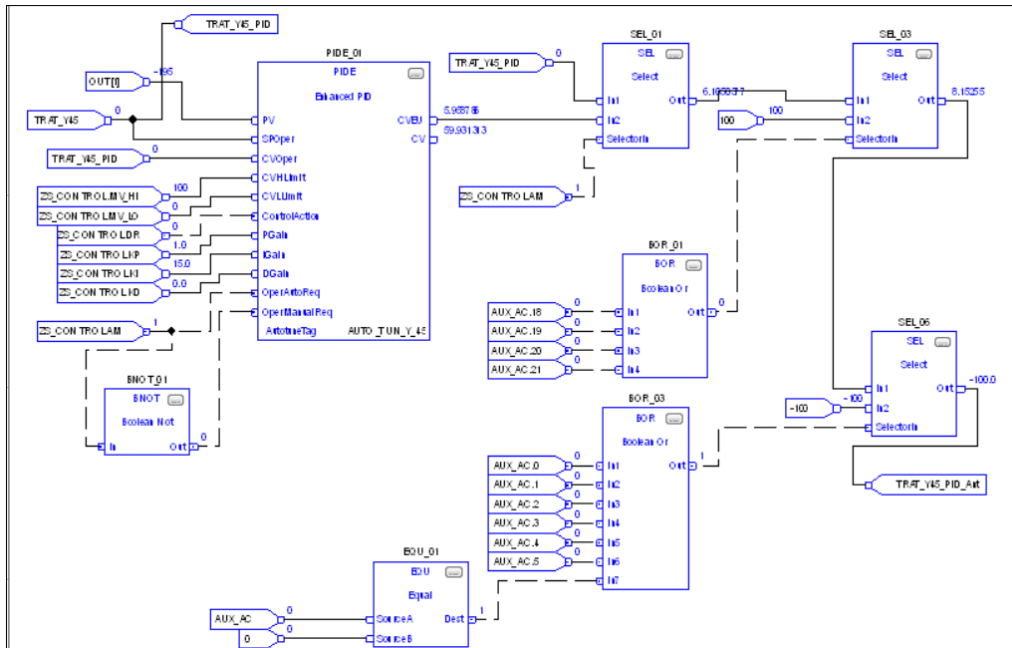
TONR(TONR_2);
TONR_2.Reset := 0;

IF M_2950_1_IN.S THEN
  ESTEIRA_IBC_IN_OUT := ESTEIRA_IBC_IN_OUT + 1;
  TONR_2.Reset := 1;
END_IF;

IF M_2950_1_OUT.S THEN
  ESTEIRA_IBC_IN_OUT := ESTEIRA_IBC_IN_OUT - 1;
  TONR_2.Reset := 1;
END_IF;
```

6.1.2. Diagrama de Blocos (FBD)

É a mesma linguagem utilizada em lógica digital, onde sua representação gráfica é feita através de símbolos gráficos provenientes da lógica combinatória, como as portas lógicas. Permite o desenvolvimento de blocos de funções complexas a partir de blocos mais simples, o que o torna versátil e ideal para processos distribuídos.



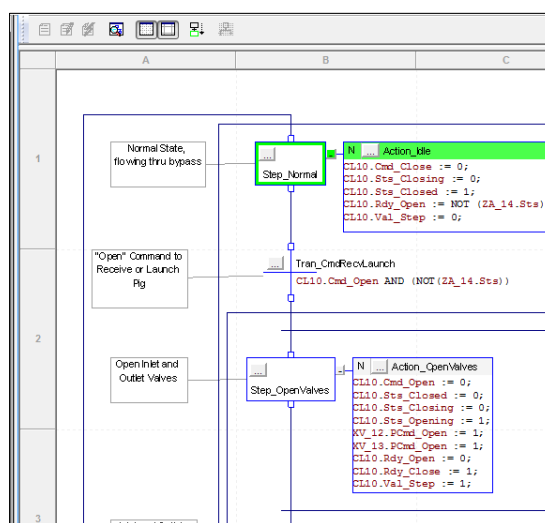
6.1.3. Lista de Instruções (IL)

É uma linguagem textual sequencial semelhante à utilizada na elaboração de programas para computadores. É indicada para processos pequenos e simples, pois não há um entendimento rápido e fácil do funcionamento do programa, como ocorre nas linguagens de bloco, porém é diretamente entendida pelo controlador, uma vez que já é a própria linguagem da máquina, sendo desnecessário ocorrer alguma espécie de conversão pelo software de programação.

read as O:001/00 = I:000/00 AND (I:000/01 OR (I:000/02 AND NOT I:000/03))			
Label	Opcode	Operand	Comment
START:	LD	%I:000/00	(* Load input bit 00 *)
	AND(	%I:000/01	(* Start a branch and load input bit 01 *)
	OR(	%I:000/02	(* Load input bit 02 *)
	ANDN	%I:000/03	(* Load input bit 03 and invert *)
	)		
	)		
	ST	%O:001/00	(* SET the output bit 00 *)

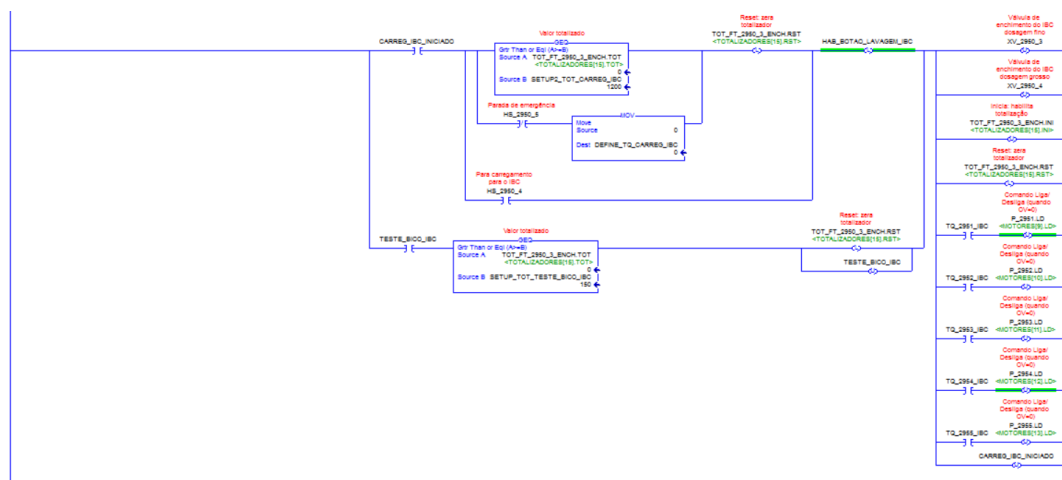
6.1.4. Passos ou Steps (SFC – Sequential Function Chart)

Antigamente chamada de GRAFCET, essa linguagem de programação executa rotinas baseadas em passos que são executados mediante a certas condições lógicas satisfeitas, ou seja, parte do pressuposto que o processo possui uma sequencialidade automática dos eventos ocorridos. O SFC estrutura a organização interna de um programa, ajudando a decompor um problema de controle em partes gerenciáveis, enquanto mantém uma visão global da solução do problema. Consiste em um conjunto de steps, ligados a blocos de ação e transições. Cada step representa um estado particular do sistema sob controle. A transição é associada com a condição, que, quando verdadeira, desativa o step anterior a ela e ativa o step seguinte.



6.1.5. Ladder (LD – Ladder Diagram)

É também conhecida como diagrama de “relés” e sua representação gráfica está muito próxima a normalmente usada em diagramas elétricos. É a mais utilizada para programação em CLP internacionalmente, sendo sua simbologia padronizada pela norma, a qual será tratada mais profundamente no Capítulo 8.





6.2. Processamento do Programa

O processamento dos programas inseridos no CLP, é feito conforme organização dos programas e rotinas distribuídas nas TASKS (tarefas), estas podem ser criadas conforme estratégia de programação e definições do usuário.

As TASKS são responsáveis pela forma e como as rotinas são executadas pela CPU do CLP, portanto, podem ser dos seguintes tipos:

1. Tarefa Contínua (Continuous Task):

A tarefa contínua executa em background (ou seja, com menor prioridade). Qualquer tempo da CPU não alocado para outras operações (p. ex. comunicação de dados ou tarefas periódicas) é usado para executar os programas dentro da tarefa contínua.

A tarefa contínua executa o tempo todo. Quando a tarefa contínua completa um ciclo de SCAN, ela recomeça imediatamente.

Um projeto não requer necessariamente uma tarefa contínua. Se usada, pode haver apenas uma tarefa contínua.

2. Tarefa Periódica (Periodic Task):

Uma tarefa periódica executa uma função em intervalos de tempo regulares. Sempre que o período de tempo para a tarefa periódica se renova, esta interrompe qualquer outra tarefa de prioridade mais baixa.

Após a execução, a tarefa periódica devolve o controle para o mesmo ponto onde a tarefa anterior foi interrompida.

3. Tarefa por Evento (Event Task):

Executa apenas quando um evento específico (disparo) ocorrer.

Sempre que ocorrer o disparo para a tarefa de evento, esta interrompe todas as tarefas com prioridades mais baixas, executa uma vez e retorna o controle para o ponto em que a tarefa anterior parou.

6.2.1. Características Gerais das Tasks

Um dado importante de uma UCP é o seu tempo de ciclo, ou seja, o tempo gasto para a execução de uma varredura. Este tempo está relacionado com o tamanho do programa do usuário e a velocidade de processamento da CPU (um CPU de pequeno porte executa em média 1.000 instruções a cada 1ms).



Cada Task possui um controle de tempo de execução, configurado pelo usuário, chamado WDT (*Watch Dog Time*), que normalmente ocorre como procedimento ao se detectar uma condição de estouro de tempo de ciclo da UCP, parando o processamento numa condição de falha e indicando ao operador através de sinal visual e, às vezes, sonoro.

6.3. Alocação Dinâmica de Dados em Memória

Os controladores lógicos programáveis possuem, atualmente, a alocação dinâmica das variáveis em sua memória, ou seja, as variáveis são criadas pelo usuário com o tipo de dado definido na sua criação, não precisam ter um tamanho fixo e nem precisam ser alocados (organizados) na memória de forma sequencial, podendo estar distribuídos internamente.

6.4. Tipagem de Dados

Além dos tipos básicos de dados que foram estudados anteriormente, pode-se haver a criação de novos dados. Um exemplo é a criação de dados estruturados, que pode englobar diversas variáveis de diferentes tipos de dados em uma mesma estrutura, ou criar uma variável de um tipo já existente, como uma string, porém com tamanho menor.

6.5. Estruturação, modularização, reutilização e modelo de software

Modelo de software é uma combinação de conceitos e ideias que simplificam e promovem uma estrutura para os projetos de solução de problemas da automação. As Unidades de Organização de Programas (POU) que incorporam o modelo de software são as funções, os blocos funcionais e os programas, que podem ser implementados a partir das linguagens de programação previamente citadas.

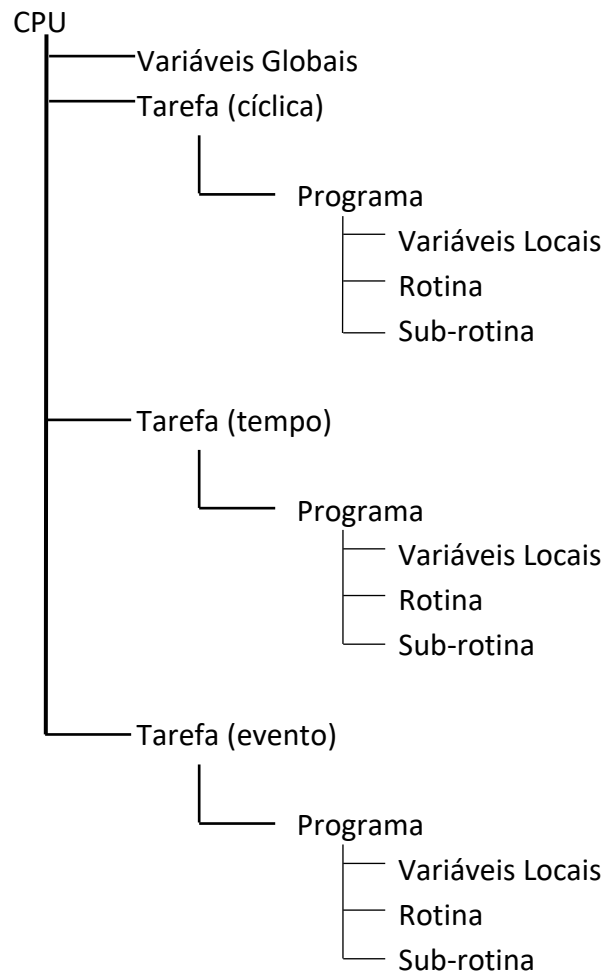
Os blocos de função podem gerar um ou mais valores a partir de sua execução, e as funções geram apenas um único valor de saída, tendo esses um uso mais geral.

Todas as funções, blocos de função e programas, que podem sempre ser reutilizados em outros programas, interagem continuamente dentro da estrutura de processamento dos controladores na CPU, onde são separadas as variáveis globais das locais, internas das externas e também os diferentes tipos de tarefas (cíclicas, de tempo e de evento).

As variáveis internas não são diretamente associadas ao hardware de entrada e saída, ou seja, não possuem conexão direta com as variáveis físicas do CLP, e podem ser utilizadas para armazenar resultados intermediários de cálculos, comparações, ou serem apenas variáveis auxiliares para o programa.

As variáveis externas possuem vínculo direto com o hardware de controle, isto é, são variáveis de pontos de entrada ou saída físicos, que recebem ou enviam valores para um dispositivo externo.

Variáveis globais são declaradas acima de todos os programas e podem ser utilizadas no programa principal e por todos os outros programas, rotinas e sub-rotinas, enquanto as variáveis locais são declaradas no início de uma rotina/subrotina e podem ser utilizadas somente pela rotina/subrotina onde foram declaradas.



7. NORMA IEC 61131

Apesar da criação de algumas normas de programação nos anos 70, como por exemplo a norma alemã DIN, para atender às demandas da comunidade industrial, um grupo de trabalho foi formado dentro da International Electrotechnical Commission (IEC) em 1979, para avaliar o projeto completo de controladores lógicos programáveis, incluindo o hardware, instalação, teste, documentação, programação e comunicação e iniciar o processo de definição de uma norma. A IEC é uma organização normativa internacional formada por representantes de todo o mundo. Foram formados diversos grupos de trabalho para elaboração das diversas partes da norma IEC 61131. A tabela seguinte apresenta o estado da norma.

PARTE	TÍTULO	CONTEÚDO
Part 1	General Information	Definição da terminologia e conceitos.
Part 2	Equipment requirements and tests	Teste de verificação e fabricação eletrônica e mecânica.
Part 3	Programmable Languages	Estrutura do software do CLP, linguagens e execução de programas.
Part 4	User guidelines	Orientações para seleção, instalação e manutenção de CLPs.
Part 5	Messaging service specification	Funcionalidades para comunicação com outros dispositivos.
Part 6	Reservada	Reservada
Part 7	Fuzzy Control Programming	Funcionalidades de software, incluindo blocos funcionais padrões para tratamento de lógica nebulosa dentro de CLPs.
Part 8	Guidelines for the implementation of languages for programmable controllers.	Orientações para implementação das linguagens IEC 1131-3.

Figura 29 - Partes da Norma IEC 61131

7.1. IEC 61131-1 – Informações Gerais

A primeira parte da norma IEC 61131 define as informações gerais dos controladores programáveis (CP), delimitando e identificando as principais características relevantes para a seleção e aplicação de CP's, e também para qualquer equipamento ou acessório ligado à CPU.

Algumas características funcionais associadas são:

Interface Homem-Máquina: o operário interpreta as funções da máquina, sendo através de monitores, botoeiras, touchscreen, etc;

Estrutura do hardware do CP;

Processamento dos sinais;



Fontes de alimentação.

A programação de um CLP pode ser realizada, bem como sobre uma plataforma de hardware e software específico de um computador de uso geral ou de um computador pessoal com funções de ambiente industrial. Esta norma aplica-se a todos os produtos que executam a função de CLP's e/ou seus periféricos associados.

Ela também foi constituída para que os CLP's de diferentes fabricantes possam ser utilizados por programas distintos, e não apenas pelos dos próprios fabricantes. Apesar de definir informações gerais dos CLP's, esta norma não lida com a segurança funcional ou outros aspectos do sistema automatizado, como choques elétricos, riscos de incêndio e erros de operação.

7.2. IEC 61131-2 - Requisitos de Equipamentos e Testes

A parte 2 da norma estabelece requisitos funcionais de Segurança de Manuseio, proteções e recomendações contra Interferências eletromagnéticas e requisitos construtivos elétricos, mecânicos e ambientais.

Outro ponto definido por esta parte da norma refere-se a memórias de backup: tipos, especificação, capacidades. Por consequência das definições de hardware, a parte 2 define também todos os testes necessários à certificação de um determinado CP conforme definido por esta norma.

7.3. IEC 61131-3 – Linguagens de Programação

A norma IEC em sua parte 3 tem por objetivo, entre outros:

- Fornecer metodologias de construção de lógicas de programação de forma estruturada e modular, permitindo a quebra dos programas em partes gerenciáveis;
- Definir 6 linguagens de programação, cada uma com suas características, de forma a cobrir a maioria das necessidades de controle atuais;
 - Texto Estruturado (ST)
 - Lista de Instruções (IL)
 - Ladder Diagrama (LD)
 - Diagramas de Blocos Funcionais (FBD)
 - Funções Gráficas de Sequenciamento (SFC)
 - Funções Gráficas Contínuas (CFC) – extensão da IEC 61131-3, que dá liberdade à posição dos elementos do gráfico.
 - Permite o uso de outras linguagens de programação, desde que obedecidas as mesmas formas de chamadas e trocas de dados (Visual Basic, Flow Chart, C++, etc);
 - Abordagem e estruturação top-down e botton-up, fundamentada em 3 princípios:
 - Modularização;



- Estruturação;
- Reutilização;

Principais Conceitos:

Configurações (Configurations): corresponde ao software necessário a um CLP ou conjunto de CLP's para que este(s) cumpra(m) suas funções de controle. A configuração define todos os elementos interagentes com suas configurações individuais e uma configuração total dada pelo resultado das diversas configurações.

Recursos (Resources): qualquer elemento com capacidade de processamento dentro de uma configuração, capaz de executar programas. O recurso pode existir fisicamente (CPU do processador, interfaces de operação IHM, gateways de comunicação) ou virtualmente (uso compartilhado de memórias de processamento por softwares distintos).

Tarefas (tasks): controla a execução de programas ou blocos funcionais de forma periódica ou por disparo por eventos ("triggers")

Unidade de Organização de Programas (POU): é a forma definida pela norma para se implementar o software do CLP através da associação de variáveis e instruções, utilizando as linguagens da norma ou linguagens adicionais. Portanto, o software aplicativo do CP consiste na criação e associação de POUs entre si.

POUs podem ser formadas a partir de associação de outras POUs, de forma a estruturar a programação e reutilizar o código.

Estruturam-se em Programas, Blocos Funcionais e Funções, reutilizáveis mediante réplicas.

Podem ser de fornecimento do fabricante do CLP ou desenvolvidas pelo usuário.

Programas (programs): construído a partir de Blocos Funcionais e Funções em qualquer das linguagens da norma. Pode acessar diretamente as Entradas e Saídas e comunicar com outros programas. Diferentes partes de um programa podem ser controladas por tarefas.

Blocos Funcionais: Partes de programas hierarquizados e estruturados de forma a serem parametrizáveis e reutilizáveis. Os dados nos Blocos possuem persistência, mantendo-se inalterados entre cada execução do bloco.

Exemplos de blocos funcionais são PID, temporizadores, contadores e blocos criados com funções específicas como controle de motores. Podem ser usados como partes integrantes de Blocos Funcionais mais complexos.

Funções (Functions): funções ou procedimentos são elementos de programação que, diferentemente de blocos funcionais, não possuem persistência, gerando resultados a cada execução.

Exemplos de funções são blocos aritméticos, comparadores e lógicos.

Variáveis Globais e Locais:

Variáveis são declaradas de duas formas: localmente e globalmente. Uma variável é dita Global quando é declarada na Configuração ou no Recurso, passando a ser aceita por todas as partes dentro do nível declarado.

Variáveis globais podem ser aceitas também por outras Configurações ou mesmo diferentes CLP's, desde que devidamente declaradas em cada um deles.

Variáveis

Bit Strings	LINT - long integer (8 byte)
BIT - 1 bit	U – Unsigned
BYTE - 8 bit (1 byte)	REAL - floating point
WORD - 16 bit (2 byte)	REAL - (4 byte)
DWORD - 32 bit (4 byte)	LREAL - (8 byte)
LWORD - 64 bit (8 byte)	TIME – duração do tempo.
INTEIROS	DATE – data do lendário
SINT - signed short (1 byte)	TIME_OF_DAY – tempo do relógio
INT - signed integer (2 byte)	DATE_AND_TIME: tempo e data
DINT - double integer (4 byte)	

7.4. IEC 61131-4 – Orientações para o usuário

Aqui são apresentadas as orientações para que aqueles que querem utilizar os Controladores Lógicos Programáveis possam os comprar e instalar, assim como a seus módulos. Ou seja, indica a forma como seus usuários/fabricantes devem especificar hardware e software necessários ao projeto, assim como instalar, comissionar e certificar o sistema de automação instalado e como formalizar a comunicação entre fornecedores e usuários finais. Essa parte também recomenda quanto a condições ambientais, assim como o uso de fiações corretas e aterramento e supressões de ruídos e transientes. Ela ainda interage com as normas IEC61508 e IEC 61511 que tratam de aplicação de Controladores Lógico Programáveis em sistemas de segurança.

7.5. IEC 61131-5 - Comunicações

A parte 5 está relacionada com os aspectos de comunicação de um CP, como a definição do módulo de comunicação, seus blocos funcionais e mecanismos para conexão entre CP's e outros dispositivos de automação. Ele especifica como um CP pode se comunicar com um servidor ou qualquer outro tipo de dispositivo.

A norma não pretende especificar como um dispositivo qualquer pode se comunicar com outro através de um CP como servidor. O comportamento do CP como um cliente e servidor de comunicação é especificado independentemente do subsistema de



comunicações em particular, mas a funcionalidade de comunicação pode ser dependente das capacidades do subsistema de comunicações utilizado.

Além disso, os meios definidos nesta parte da IEC 61131 podem ser utilizados para comunicações dentro de um programa ou entre programas.

7.6. IEC 61131-7 – Programação de controle FUZZY

A norma define e estrutura o uso de linguagem utilizada em programação difusa – Fuzzy Control Language (FCL).

Composta de seis tópicos, essa parte define:

- O escopo e objetivo da programação Fuzzy;
- Referências normativas;
- Termos técnicos;
- Integração da lógica Fuzzy com os CLPs;
- Semânticas e sintaxes da FCL;

A lógica fuzzy é uma extensão da lógica booleana, ela admite valores intermediários entre 0 e 1, como se existisse entre o verdadeiro e o falso um talvez, ou um 0,5 entre o 0 e 1.

7.7. IEC 61131-8 – Implementação das Linguagens

A oitava parte da norma complementa a terceira de forma a orientar usuários envolvidos com programação, configuração, instalação e manutenção de Controladores Lógicos Programáveis, de forma que a implementação de elementos comuns e linguagens de programação se dê de forma sistematizada, melhorando as práticas e garantindo a qualidade. Essa parte também orienta como devem ser utilizadas linguagens adicionais, assim como variáveis locais e globais.



8. PRÁTICA – PROGRAMAÇÃO

8.1. Introdução

O diagrama Ladder utiliza a lógica de relé, com contatos (ou chaves) e bobinas, e por isso é a linguagem de programação de CLP mais simples de ser assimilada por quem já tenha conhecimento de circuitos de comando elétrico.

Compõe-se de vários circuitos dispostos horizontalmente, com a bobina na extremidade direita, alimentados por duas barras verticais laterais. Por esse formato é que recebe o nome de Ladder, que significa escada em inglês. Cada uma das linhas horizontais é uma sentença lógica onde os contatos são as entradas das sentenças, as bobinas são as saídas e a associação dos contatos é a lógica.

No Ladder, cada operando (contatos/bobinas/instruções) é identificado com um endereço da memória à qual se associa no CLP.

As bobinas acionam o seu endereço: enquanto uma bobina com endereço de saída estiver acionada, um par de terminais no módulo de saída será mantido em condição de condução elétrica.

Os contatos se acionam pelo endereço que os identifica: os contatos endereçados como entrada se acionam enquanto seu respectivo par de terminais no módulo de entrada é acionado: fecham-se se forem NA e abrem-se se forem NF. Assim, os contatos que tiverem por finalidade acionar ou energizar uma bobina deverão ser do mesmo tipo do contato externo que aciona seu respectivo ponto no módulo de entrada; já os que forem usados para desacionar ou desenergizar uma bobina devem ser de tipo contrário do contato externo que os aciona.

	Se a chave externa for	o contato no ladder deve ser
Para ligar	NA	NA
	NF	NF
Para desligar	NA	NF
	NF	NA

8.2. Instruções e Blocos de Função

As instruções básicas são representadas por blocos funcionais introduzidos na linha de programação em lógica Ladder. Estes blocos funcionais podem se apresentar de formas diferentes de um CLP para outro, mas a filosofia de funcionamento é invariável. Estes blocos auxiliam ou complementam o controle do equipamento, introduzindo na lógica Ladder instruções como de temporização, contagem, soma, divisão, subtração, multiplicação, PID, etc.

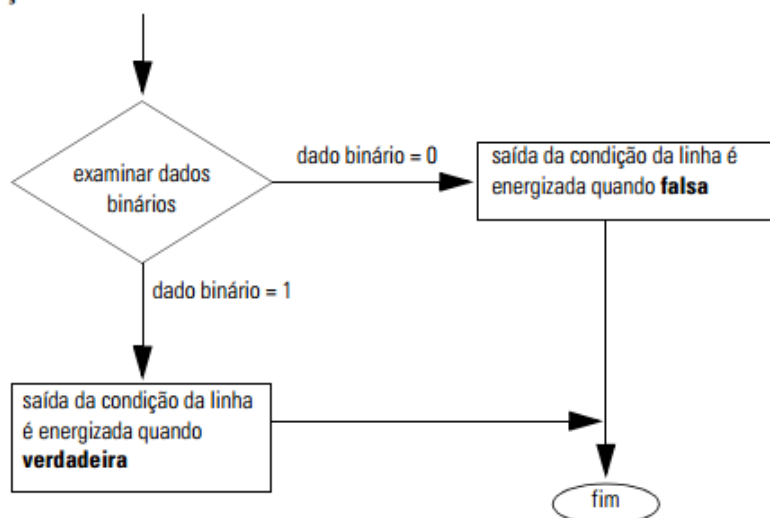
8.2.1. Booleanas

8.2.1.1. XIC (Examine If Closed)

Diagrama Ladder:

A instrução XIC examina os dados binários para ver se estão energizados, funcionando como uma chave normalmente aberta, ou seja, examina o bit de informação que os identifica para checar se foi acionada ou não. Se sim, a próxima operação da linha de instrução é habilitada.

Condição:	Ação:
pré-varredura	A saída da condição da linha é definida como falsa.
entrada da condição da linha for falsa	A saída da condição da linha é definida como falsa.
entrada da condição da linha for verdadeira	



No Texto Estruturado essa instrução é equivalente ao uso do “IF...THEN”.

Diagrama Ladder



Texto Estruturado

```

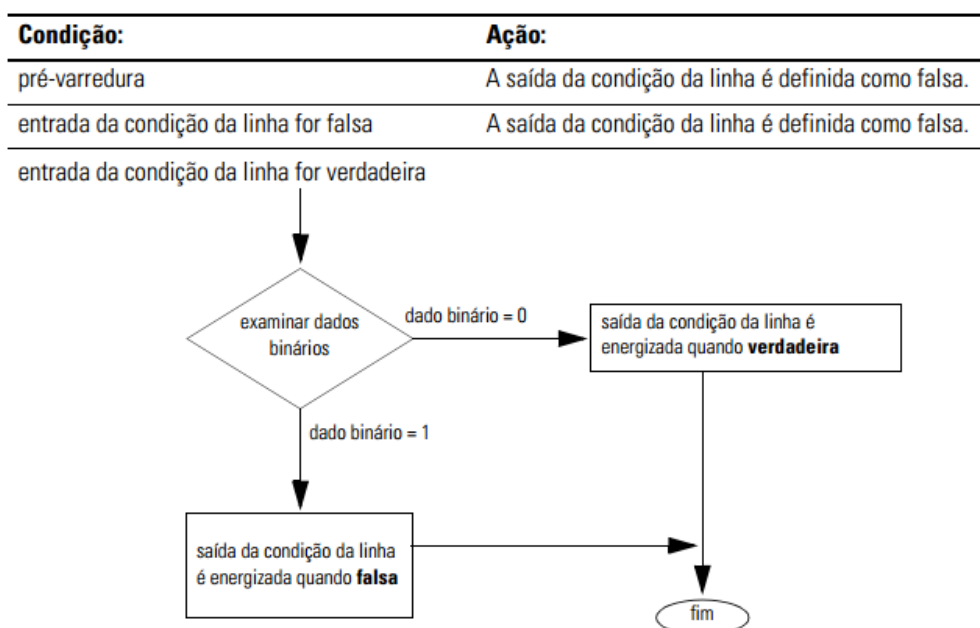
IF chave1 THEN
    <operação>;
END_IF;
  
```

Se *chave1* for fechada e consequentemente estiver energizada, a próxima instrução é habilitada (a saída da condição da linha é verdadeira).

8.2.1.2. XIO (Examine If Opened)

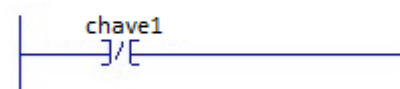
Diagrama Ladder:

A instrução XIO examina os dados binários para ver se foram desenergizados, funcionando como uma chave normalmente fechada, ou seja, examina o bit de informação que o identifica para checar se foi aberta ou não. Se sim, a próxima operação da linha de instrução é habilitada.



No Texto Estruturado essa instrução também é equivalente ao uso do “IF...THEN”.

Diagrama Ladder



Texto Estruturado

```

IF NOT chave1 THEN
    <operação>;
END_IF;
  
```

Se *chave1* estiver aberta a próxima instrução é habilitada.

8.2.1.3. OTE (Output Energize - Bobina Simples)

Diagrama Ladder:

A instrução OTE energiza ou desenergiza o dado binário. Quando a instrução OTE estávhabilitada, o controlador energiza o dado binário. Quando a instrução OTE está desabilitada, o controlador desenergiza o dado binário.



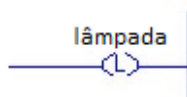
Quando o contato *chave* se fecha e a instrução é habilitada, a instrução OTE energiza *lâmpada1*, mas somente enquanto esse contato permanecer fechado. Se *chave* for desabilitada, a bobina *lâmpada1* também é desenergizado.

8.2.1.4. OTL (Output Latch)

Diagrama Ladder:

A instrução OTL energiza (com retenção) o dado binário, ou seja, ocorre o travamento da saída em nível lógico 1. Quando é habilitado, a instrução OTL energiza o dado binário, que permanece em 1 (energizado) até que seja desenergizado por outra instrução (OTU). Quando desabilitada, a instrução OTL não muda o status do dado binário.

Diagrama Ladder



Texto Estruturado

```
IF BOOL_expression THEN
    lâmpada := 1;
END IF;
```

Nessa operação, quando habilitada, a instrução OTL energiza *lâmpada*, que permanece energizado até ser desenergizado por uma instrução OTU.

8.2.1.5. OTU (Output Unlatch)

Diagrama Ladder: 

A instrução OTU desenergiza (com retenção) o dado binário, ou seja, ocorre o destravamento da saída. Quando habilitada, a instrução OTU desenergiza o dado binário, retornando-o para nível lógico 0. Quando desabilitada, a instrução OTU não muda o status do dado binário.

Diagrama Ladder



Texto Estruturado

```
IF BOOL expression THEN
    lâmpada2 := 0;
END_IF;
```

Quando habilitada, a instrução OTU desenergiza *lâmpada2*.

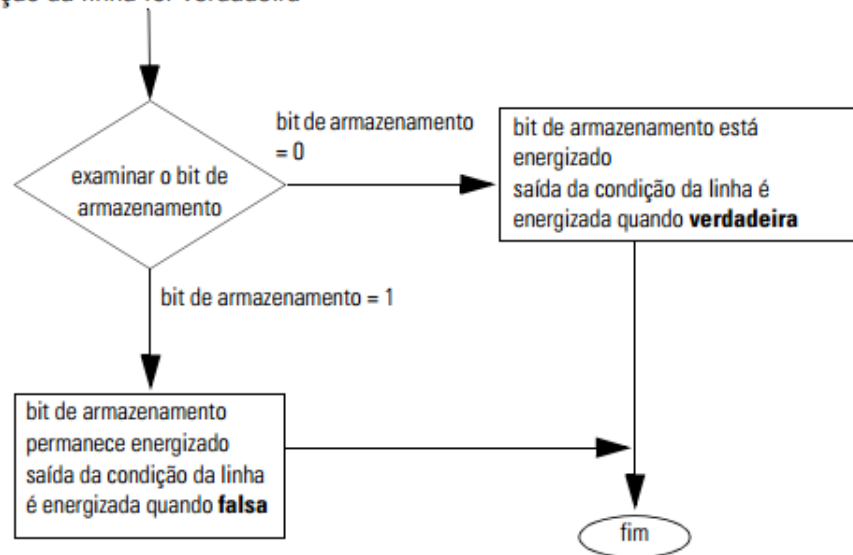
8.2.1.6. ONS (One Shot)

Diagrama Ladder: 

Essa instrução habilita ou desabilita o restante da linha, dependendo do status do bit de armazenamento (storage bit). Quando está habilitado e o bit de armazenamento está desenergizado, o restante da linha de instrução também é habilitado; e quando está habilitado ou o bit de armazenamento está energizado, o ONS desabilita o restante da linha de instrução. Geralmente a instrução ONS é precedida por uma instrução de entrada, como uma chave.

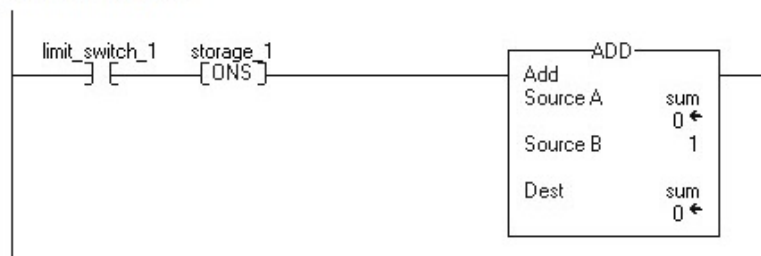
Condição:	Ação:
pré-varredura	O bit de armazenamento está energizado para prevenir um disparo inválido durante a primeira varredura. A saída da condição da linha é definida como falsa.
entrada da condição da linha for falsa	O bit de armazenamento é desenergizado. A saída da condição da linha é definida como falsa.

entrada da condição da linha for verdadeira



Geralmente, você antecede a instrução ONS com uma instrução de entrada porque realiza a varredura da instrução ONS quando a mesma está habilitada e quando está desabilitada para que a mesma opere corretamente. Uma vez que a instrução ONS está habilitada, a condição de entrada de linha deve ser desenergizada ou o bit de armazenamento deve ser desenergizado para a instrução ONS ser habilitada novamente.

Diagrama Ladder



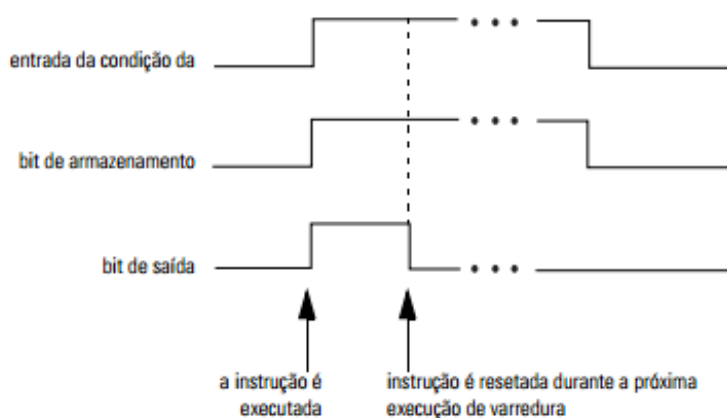
Quando em uma varredura o *limit_switch_1* está desenergizado e o *storage_1* está energizado, a linha não possui nenhum efeito. Porém, quando o *limit_switch_1* está energizado e o *storage_1* está desenergizado, a instrução ONS energiza o *storage_1* e então a instrução ADD realiza o incremento em 1. Enquanto o *limit_switch_1* estiver energizado, a soma permanece com o mesmo valor. O *limit_switch_1* deve ir de desenergizado para energizado novamente para que a soma seja incrementada novamente.

8.2.1.7. OSR (One Shot Rising)

Diagrama Ladder:



Essa instrução energiza ou desenergiza o bit de saída (output bit), dependendo do status do bit de armazenamento (storage bit). Quando habilitado e o bit de armazenamento está desenergizado, o OSR energiza a o bit de saída. Quando habilitado e o bit de armazenamento está energizado, ou quando desabilitado, o OSR desenergiza o bit de saída.



Condição:	Ação:
pré-varredura	O bit de armazenamento está energizado para prevenir um disparo inválido durante a primeira varredura. O bit de saída é desenergizado. A saída da condição da linha é definida como falsa.
entrada da condição da linha for falsa	O bit de armazenamento é desenergizado. O bit de saída não é modificado. A saída da condição da linha é definida como falsa.

entrada da condição da linha for verdadeira

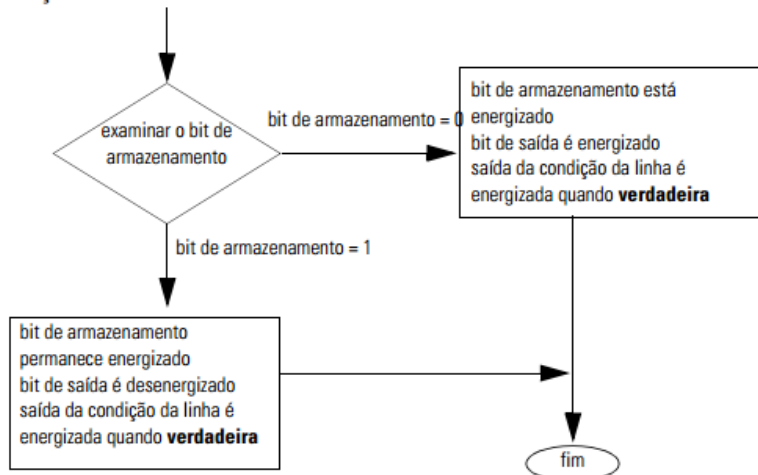
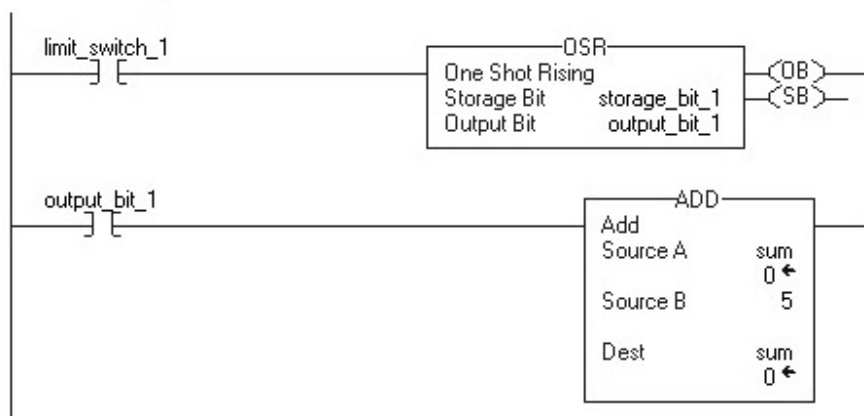


Diagrama Ladder



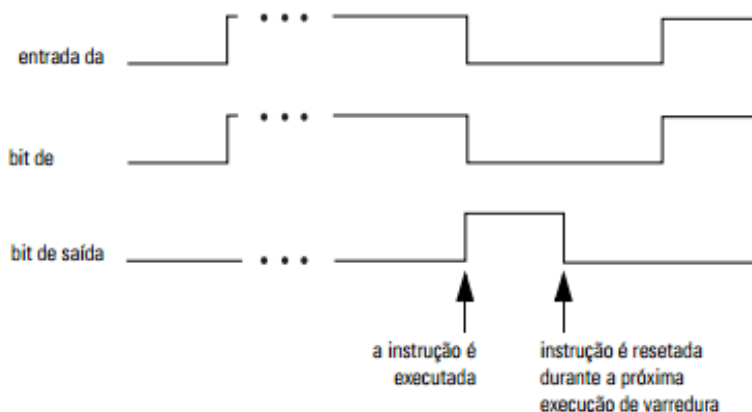
Cada vez que o `limit_switch_1` vai de desenergizado para energizado, o OSR energiza o `output_bit_1` e ocorre o incremento de 5 através da instrução ADD. Enquanto o `limit_switch_1` permanecer energizado, a soma permanece com o mesmo valor. O `limit_switch_1` deve ir de desenergizado para energizado novamente para que a soma ocorra mais uma vez. O `output_bit_1` pode ser usado em várias linhas de instruções de modo a realizar outras operações.

8.2.1.8. OSF (One Shot Falling)

Diagrama Ladder:



A instrução OSF energiza ou remove o bit de saída, dependendo do status do bit de armazenamento. Quando desabilitado e o bit de armazenamento está energizado, o OSF energiza o bit de saída. Quando desabilitado e o bit de armazenamento está desenergizado, o OSF remove o bit de saída.



Condição:	Ação:
pré-varredura	O bit de armazenamento é desenergizado para prevenir um disparo inválido durante a primeira varredura. O bit de saída é desenergizado. A saída da condição da linha é definida como falsa.
entrada da condição da linha for verdadeira	O bit de armazenamento é energizado. O bit de saída é desenergizado. A saída da condição da linha está definida como verdadeira.

entrada da condição da linha for falsa

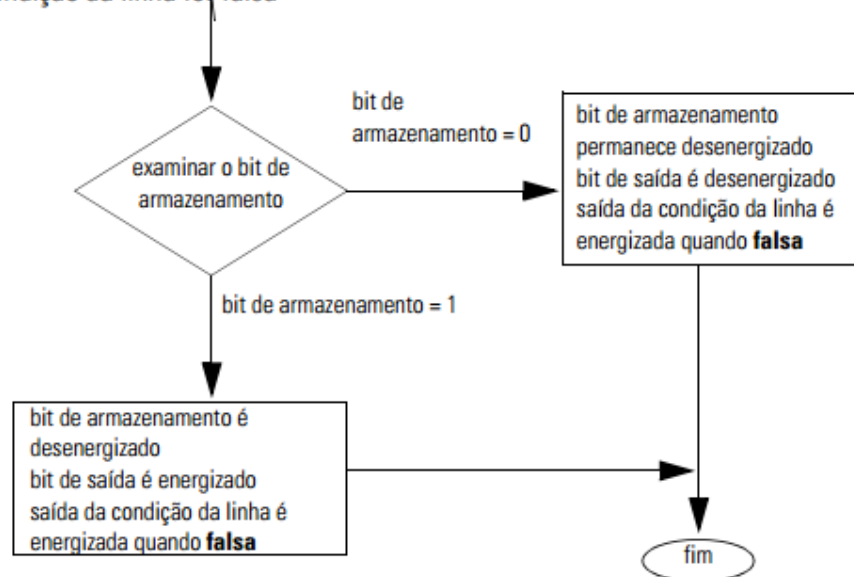
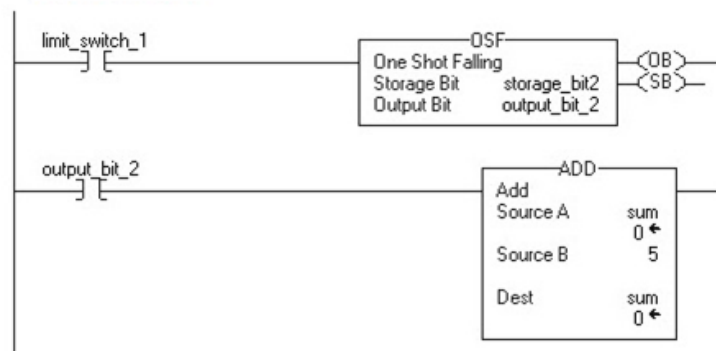


Diagrama Ladder



A cada vez que o *limit_switch_1* vai de energizado para desenergizado, o OSF energiza o *output_bit_2* e ocorre o incremento de 5 através da instrução ADD. Enquanto o *limit_switch_1* permanece desenergizado, o valor da soma se mantém o mesmo. Para realizar um novo incremento o *limit_switch_1* deve ir de energizado para desenergizado. O *output_bit_2* pode ser usado em várias linhas de instruções de modo a realizar outras operações.

8.2.2. Timer/Counter

8.2.2.1. TON (Timer On Delay)

Diagrama Ladder:



Essa instrução é um temporizador não-retentivo que acumula tempo quando a instrução é energizada e a linha de condição for verdadeira. Basicamente, o temporizador conta o intervalo de tempo transcorrido a partir da sua habilitação até este se igualar ao tempo preestabelecido. O Preset é o tempo desejado para o temporizador acumular; o Accum é o total em milissegundos que o temporizador já acumulou, possuindo valor inicial geralmente igual a 0. Enquanto o tempo está sendo acumulado, a saída EN permanece ativada; e quando o Accum atinge o valor especificado pelo Preset o temporizador é desabilitado, desativando a saída EN e ativando a saída DN. A base de tempo é sempre dada em milissegundos. Por exemplo, para 2 segundos, deve-se colocar 2000 como valor de Preset. O valor acumulado no Accum é resetado quando o temporizador é desenergizado, ou seja, sua linha é falsa.

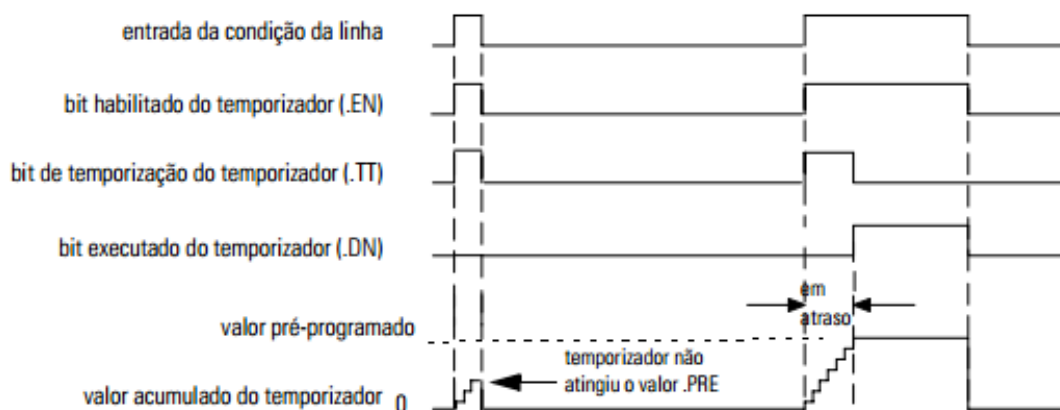
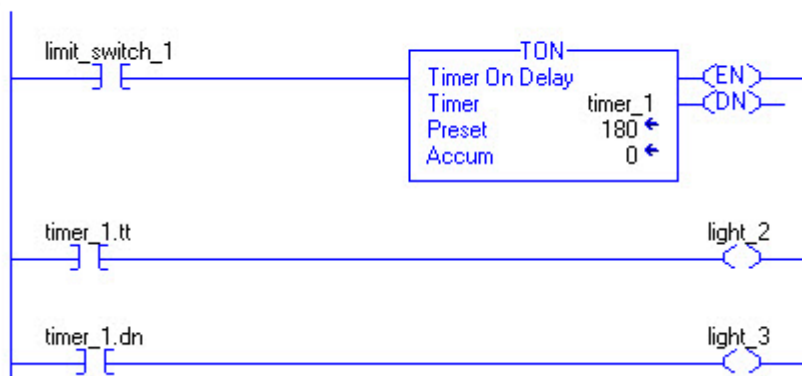


Diagrama Ladder



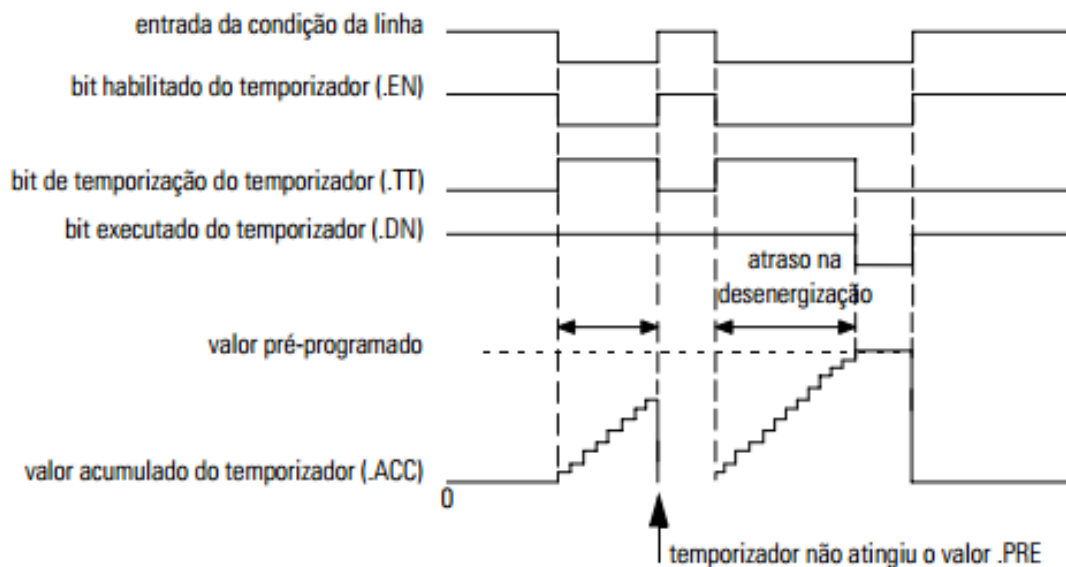
Quando o *limit_switch_1* é energizado, o TON é habilitado, iniciando a contagem de 180ms. Enquanto está ocorrendo essa contagem, o *timer_1.tt* permanece fechado pois o temporizador está em funcionamento, energizando *light_2*. Ao final da contagem, quando *timer_1* atinge 180ms, como o temporizador não está mais em operação, *timer_1.tt* é aberto, desenergizando *light_2* e a saída DN é energizada, portanto *timer_1.dn* se fecha, energizando *light_3*. Se o *limit_switch_1* for desenergizado enquanto o *timer_1* está cronometrando, o *light_2* desenergiza.

8.2.2.2. TOF (Timer Off Delay)

Diagrama Ladder:

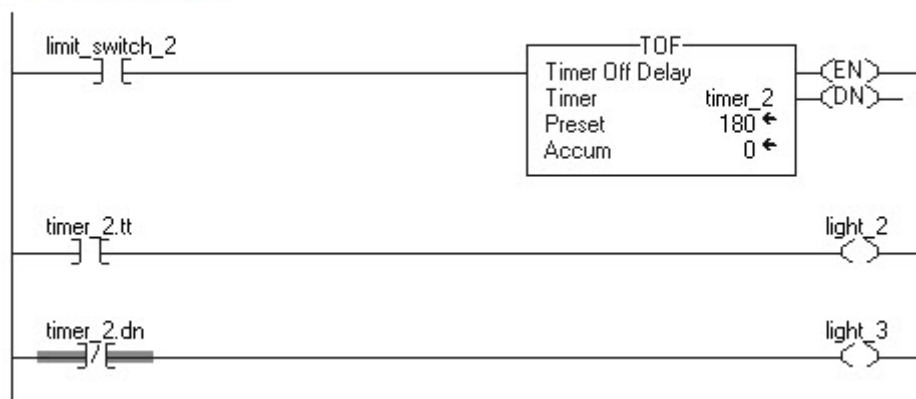


Essa instrução é um temporizador não-retentivo semelhante ao TON, mas que acumula tempo quando a instrução é habilitada e a linha de condição for falsa.



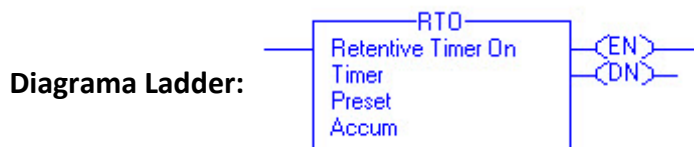
A diferença entre o TON e o TOF é que o TON inicia a contagem do tempo e só quando atinge o valor de Preset tem a saída DN acionada. O TOF aciona a sua saída DN quando é energizado e a mantém até que o acumulado atinja o valor de Preset, quando então o DN é desligado. O tempo só inicia quando a entrada é desligada.

Diagrama Ladder

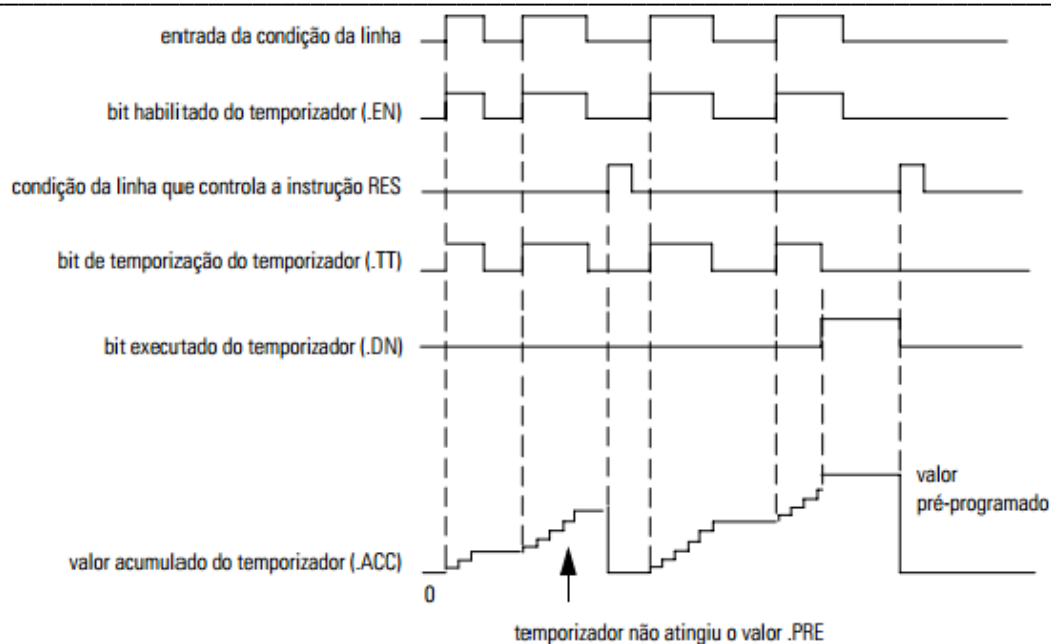


Quando *limit_switch_2* é desenergizado, *light_2* é energizado por 180ms (enquanto *timer_2* está cronometrando) e *timer_2.dn* se abre, pois DN está energizado. Quando o *Accum* atinge 180, *light_2* é desenergizada e *light_3* é energizada, pois a saída *EN* é energizada, permanecendo assim até que o TOF seja habilitada novamente. Se o *limit_switch_2* for energizado enquanto o *timer_2* está cronometrando, o *light_2* desenergiza.

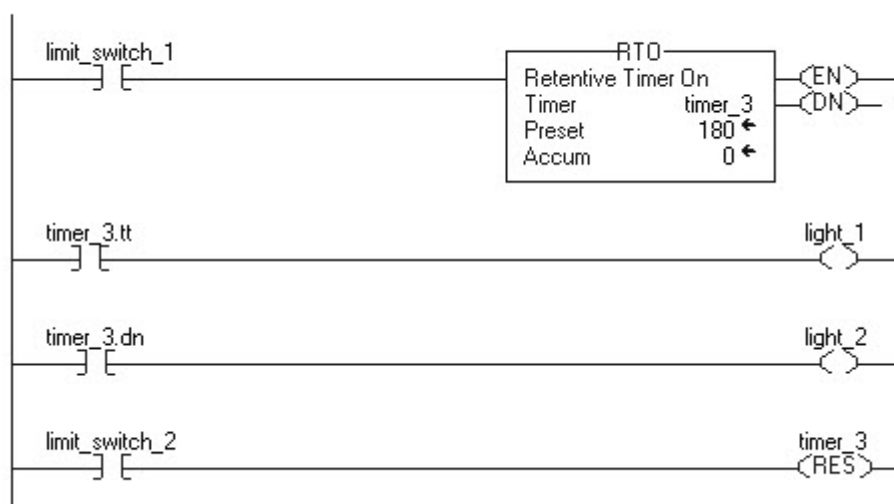
8.2.2.3. RTO (Retentive Timer On)



Essa instrução é um temporizador retentivo que acumula tempo quando a instrução é habilitada. O Preset é o tempo desejado para o temporizador acumular; o Accum é o total em milissegundos que o temporizador já acumulou, possuindo valor inicial geralmente igual a 0. O RTO acumula tempo até que a instrução seja desabilitada e, quando isso acontece, ele retém o valor contido no .ACC, sendo necessário resetar esse valor, geralmente utilizando a instrução RES (reset). A base de tempo é sempre dada em milissegundos.



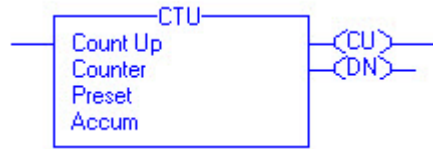
Ladder Diagram



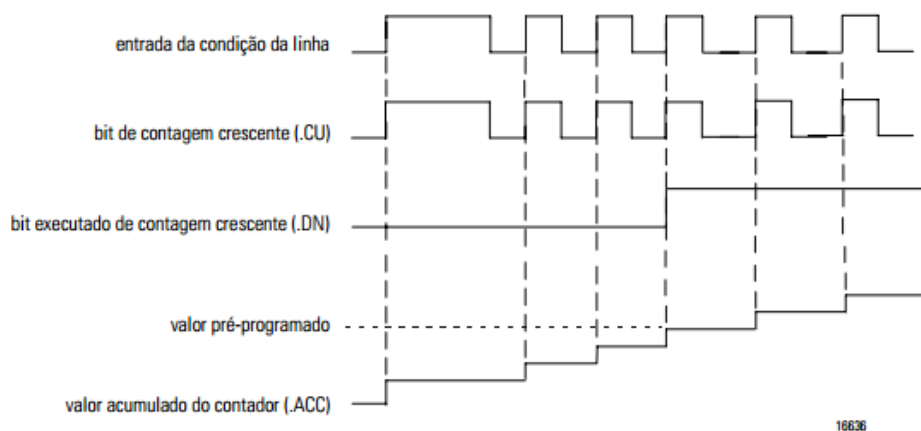
Quando o *limit_switch_1* se fecha, *light_1* é energizado por 180ms (quando o *timer_3* está cronometrando). Quando *timer_3.acc* atinge 180, *light_1* é desenergizado e *light_2* é energizado, permanecendo energizado até que *timer_3* seja resetado, ou seja, tenha seu valor acumulado zerado, através do *limit_switch_2* que energiza a instrução RES para o *timer_3*.

8.2.2.4. CTU (Count Up)

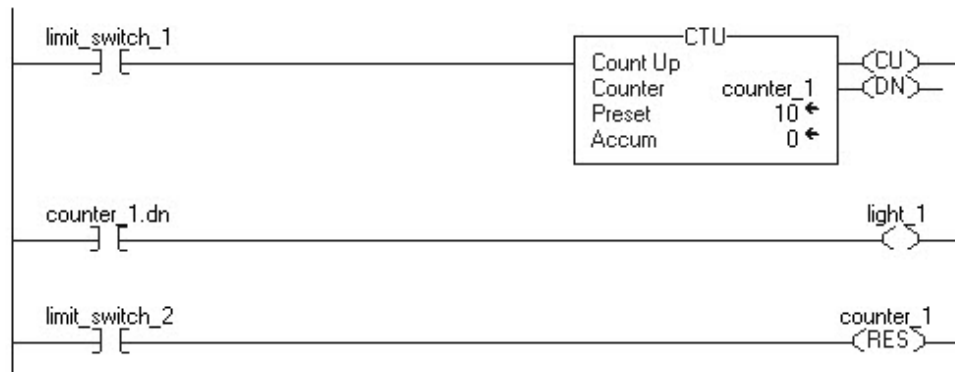
Diagrama Ladder:



Essa instrução é um contador crescente. O Preset é o valor desejado para o contador acumular; o Accum é o total que o contador já acumulou, possuindo valor inicial geralmente igual a 0. Quando energizado, o CTU incrementa o valor acumulado em 1; e quando desenergizado o valor acumulado é mantido, sendo necessário o uso da instrução RES. O valor acumulado continua sendo incrementado, mesmo após o .DN ser ativado, até que o contador seja resetado.



Ladder Diagram

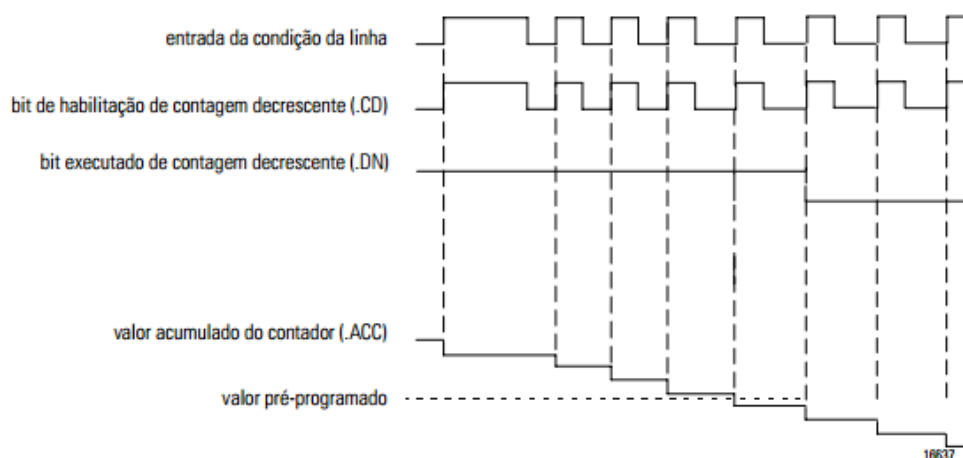


Depois que o *limit_switch_1* muda de aberto para fechado 10 vezes, o .DN do contador é energizado e *light_1* é energizado. Se *limit_switch_1* continuar sendo energizado e desenergizado, o contador continua a incrementar seu valor e o .DN permanece energizado. Quando *limit_switch_2* é fechado, a instrução RES é energizada de forma a resetar o *counter_1*, resetando o valor acumulado do contador. Assim, *light_1* é desenergizada.

8.2.2.5. CTD (Counter Down)



Essa instrução é um contador decrescente. O Preset é o valor desejado para o contador acumular; o Accum é o total que o contador já acumulou. A estrutura do CTD é semelhante à do CTU, porém quando energizado, o CTD decrementa o seu valor em 1. Da mesma forma que ocorre no CTU, deve-se resetar o contador para zerar seu valor acumulado e recomeçar a contagem.

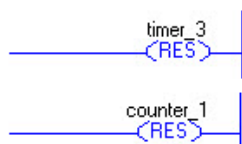


8.2.2.6. RES



Essa instrução tem a função de resetar timers, contadores, entre outros.

Exemplo:



Quando energizado, o *timer_3* e o *conter_1* são resetados.

8.2.3. Comparadores

8.2.3.1. Equal (EQU)

Diagrama Ladder:

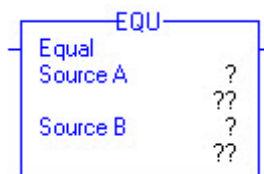
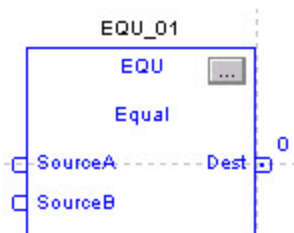


Diagrama de Blocos:

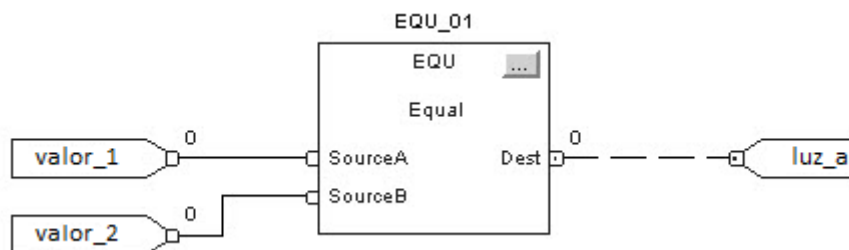


Essa instrução testa se o valor escrito em *Source A* é igual ao valor escrito em *Source B*. Pode-se usar essa instrução para comparar dois números e também é possível comparar duas strings, ou seja, duas palavras, e serão iguais se seus caracteres se correspondem.

Exemplos:

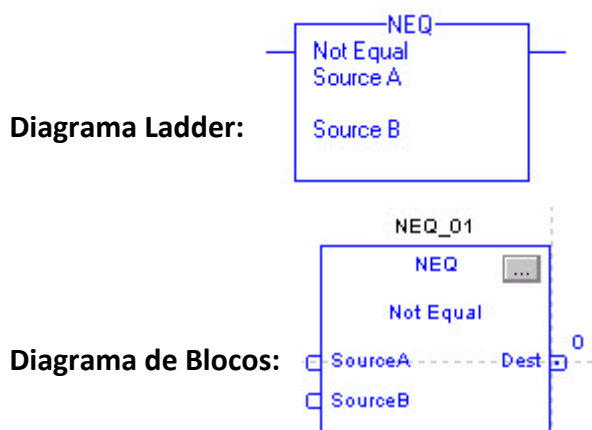


Neste exemplo, se *valor_1* for igual a *valor_2*, então a saída da condição da linha é definida como verdadeira e, portanto, *luz_a* é energizado.



Este exemplo é o mesmo anterior, porém em Diagrama de Blocos.

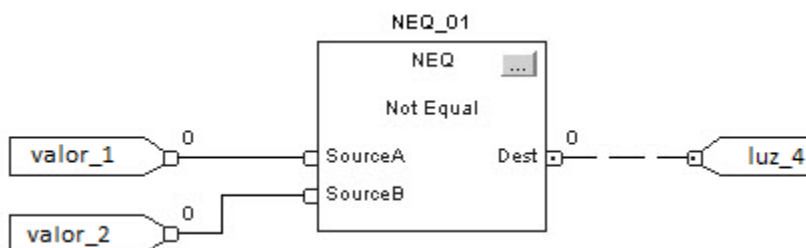
8.2.3.2. Not Equal To (NEQ)



Essa instrução testa se *Source A* não é igual a *Source B*.



Nesse exemplo em diagrama Ladder, se *valor_1* for diferente de *valor_2*, *luz_4* é habilitada e pode ser energizado. Caso contrário, não é habilitada.



Este exemplo é o mesmo anterior, porém em Diagrama de Blocos.

8.2.3.3. Less Than (LES)

Diagrama Ladder:

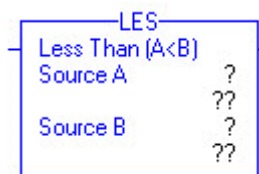
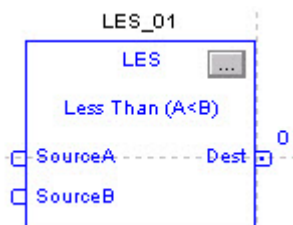
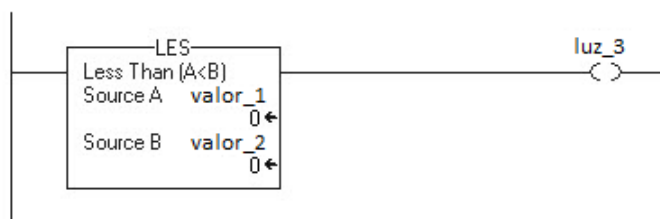


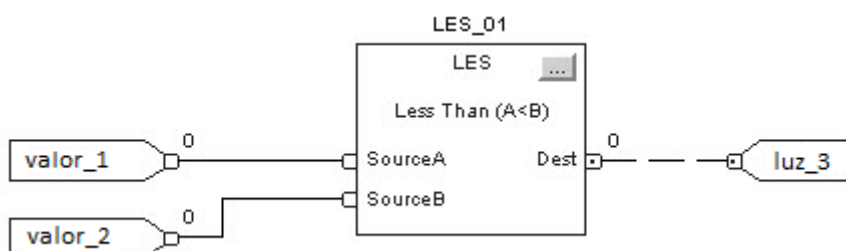
Diagrama de Blocos:



Essa instrução testa se *Source A* é menor que *Source B*. Se sim, a linha de condição é habilitada para realizar a próxima operação.



Nesse exemplo em diagrama Ladder, se o valor contido em *valor_1* for menor que o valor contido em *valor_2*, então *luz_3* é energizada. Caso contrário, *luz_3* não é habilitada e, logo, não pode ser energizada.



O mesmo ocorre nesse exemplo em diagrama de Blocos.

8.2.3.4. Greater Than (GRT)

Diagrama Ladder:

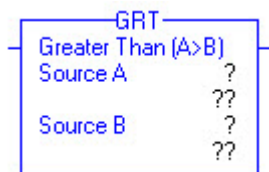
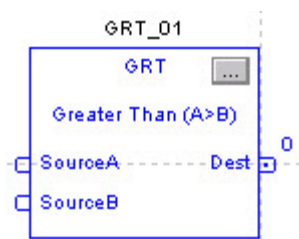
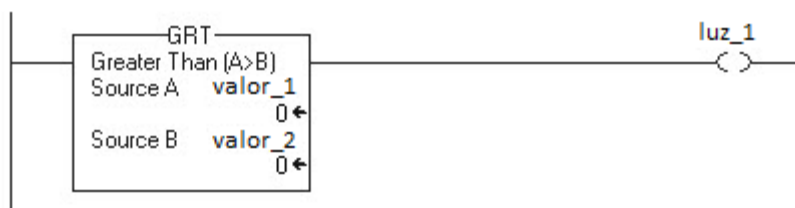


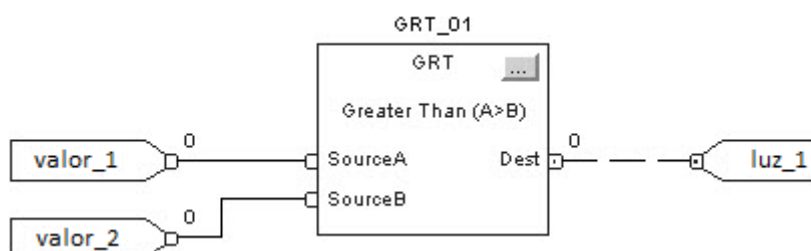
Diagrama de Blocos:



Essa instrução testa se o valor contido em *Source A* é maior que o valor contido em *Source B*. Se sim, a saída da condição da linha é definida como verdadeira e é habilitada a realizar a próxima operação.



Nesse exemplo em diagrama Ladder, se o valor contido em *valor_1* for maior que o valor contido em *valor_2*, então *luz_1* é energizada. Caso contrário, a condição da linha não é habilitada e, logo, não pode energizar *luz_1*.



O mesmo ocorre nesse exemplo em diagrama de Blocos.

8.2.3.5. Less Than or Equal To (LEQ)

Diagrama Ladder:

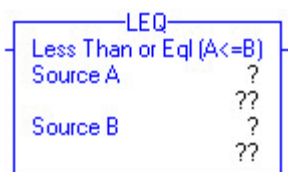
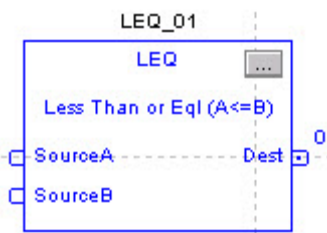
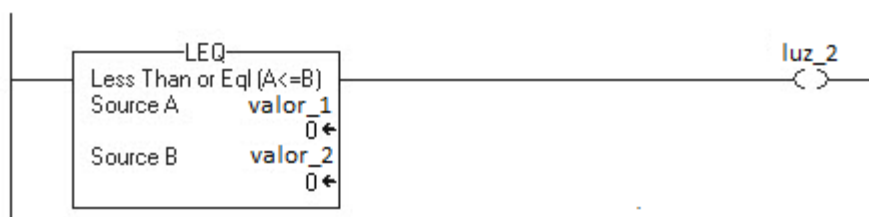


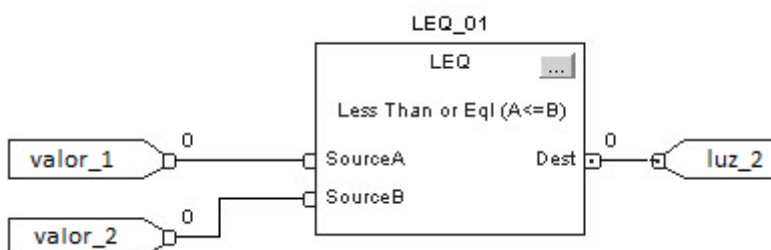
Diagrama de Blocos:



Essa instrução verifica se *Source A* é menor ou igual a *Source B*. Se sim, a linha de condição é habilitada para realizar a próxima operação.

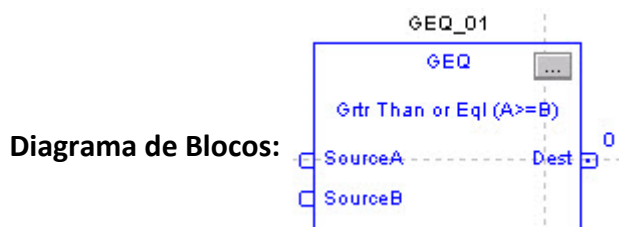


Nesse exemplo em diagrama Ladder, se o valor contido em *valor_1* for menor ou igual ao valor contido em *valor_2*, então a linha de condição é verdadeira e *luz_2* é energizada. Caso contrário, a linha de condição não é habilitada e, logo, *luz_2* não pode ser energizada.



O mesmo ocorre nesse exemplo em diagrama de Blocos.

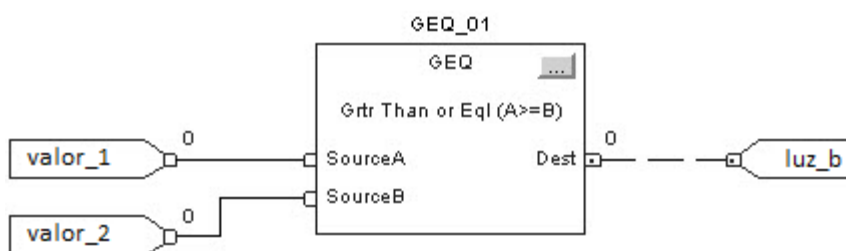
8.2.3.6. Greater Than or Equal To (GEQ)



Essa instrução verifica se o valor contido em *Source A* é maior ou igual ao valor contido em *Source B*. Se sim, a linha de condição é habilitada para realizar a próxima operação.



Nesse exemplo em diagrama Ladder, se o valor contido em *valor_1* for maior ou igual ao valor contido em *valor_2*, então a linha de condição é verdadeira e *luz_b* é energizada. Caso contrário, a linha não é habilitada e, logo, *luz_2* não pode ser ativada (energizada).



O mesmo ocorre nesse exemplo em diagrama de Blocos.

8.2.3.7. Compare (CMP)



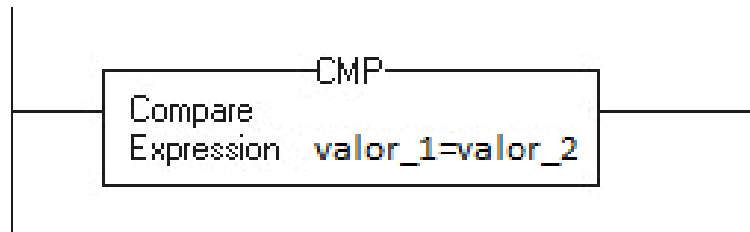
A instrução CMP tem a função de comparar operações aritméticas que são especificadas pelo programador em '*Expression*'. A expressão CMP é definida usando operadores, tags ou valores imediatos. Deve-se usar parênteses para expressões mais complexas. Essa instrução possui uma execução mais lenta e requer mais memória do que a execução de outras instruções de comparação, porém possui a vantagem de ser possível resolver expressões complexas em apenas uma instrução.

As operações não são realizadas necessariamente na ordem em que são escritas, mas o operador pode forçar a ordem que necessitar usar, agrupando os termos através do uso dos parênteses. Operações de ordem iguais são realizadas da esquerda para a direita.

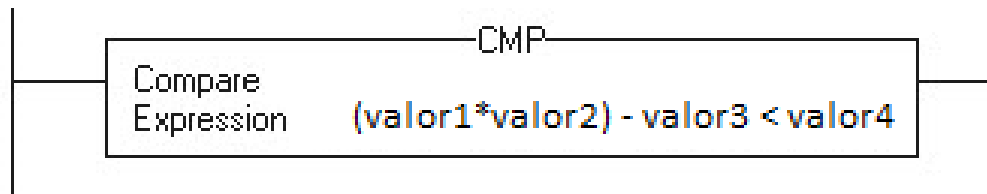
O CMP pode realizar diversas funções, entre elas:

Operador:	Descrição:	Ótimo:
+	adição	DINT, REAL
-	subtração/negativo	DINT, REAL
*	multiplicação	DINT, REAL
/	divisão	DINT, REAL
=	igual	DINT, REAL
<	menor do que	DINT, REAL
<=	menor ou igual a	DINT, REAL
>	maior do que	DINT, REAL
>=	maior ou igual a	DINT, REAL
<>	diferente de	DINT, REAL
**	expoente (x elevado a y)	DINT, REAL
ABS	valor absoluto	DINT, REAL
ACS	arco co-seno	REAL
AND	bitwise AND	DINT
ASN	arco seno	REAL
ATN	arco tangente	REAL
COS	co-seno	REAL

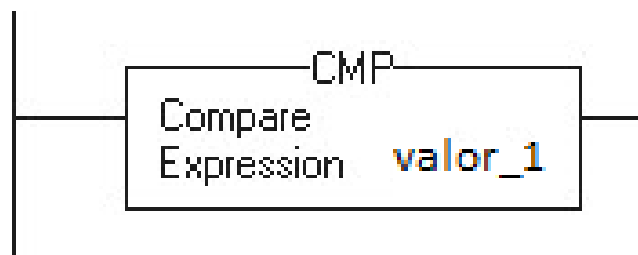
Operador:	Descrição:	Ótimo:
DEG	radianos para graus	DINT, REAL
FRD	BCD para inteiro	DINT
LN	natural log	REAL
LOG	logaritmo na base de 10	REAL
MOD	módulo-divisão	DINT, REAL
NOT	bitwise complemento	DINT
OR	bitwise OR	DINT
RAD	graus para radianos	DINT, REAL
SIN	seno	REAL
SQR	raiz quadrada	DINT, REAL
TAN	tangente	REAL
TOD	inteiro para BCD	DINT
TRN	truncado	DINT, REAL
XOR	bitwise exclusivo OR	DINT



Nessa operação, se a expressão de comparação for verdadeira, ou seja, se *valor_1* for igual a *valor_2*, a linha de condição é verdadeira e, então, é habilitada (energizada).



Como no exemplo anterior, se o resultado da expressão for verdadeiro, ou seja, se $(valor1 * valor2) - valor3$ for menor que *valor4*, a linha de operação é habilitada.



Nesse caso, quando não usamos um operador de comparação, a expressão é verdadeira se o valor de *valor_1* for diferente de zero, e falsa se for igual à zero.

8.2.4. Matemáticos

8.2.4.1. ADD

Diagrama Ladder:

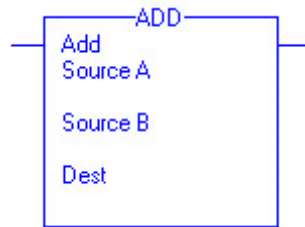
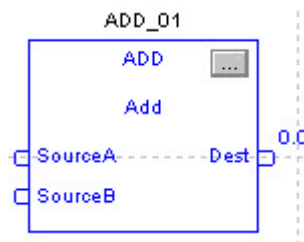
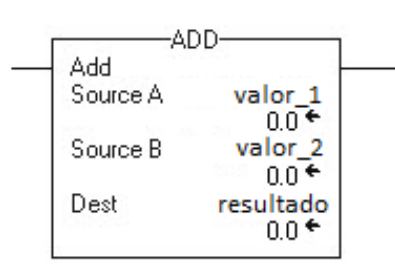


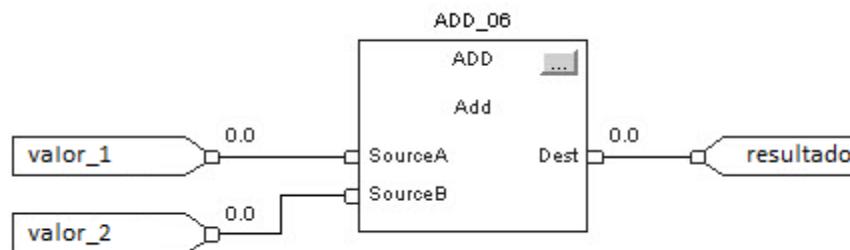
Diagrama de Blocos:



Essa instrução realiza a soma do valor contido em *Source A* com o valor contido em *Source B*, e o resultado obtido é armazenado em '*Dest*' (destination).



Nesse exemplo em Diagrama Ladder, o valor obtido com a soma de *valor_1* e *valor_2* é armazenado em *resultado*.



O mesmo ocorre nesse exemplo em diagrama de Blocos.

8.2.4.2. SUB

Diagrama Ladder:

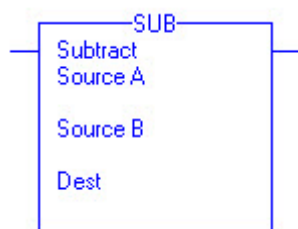
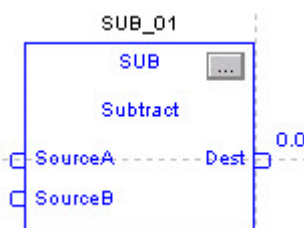
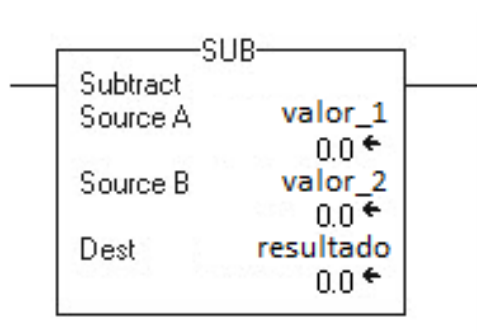


Diagrama de Blocos:

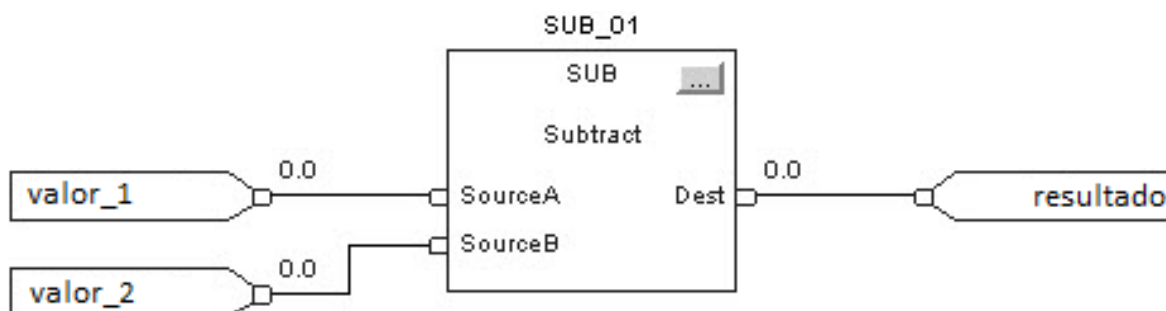


Essa instrução realiza a subtração do valor contido em *Source A* com o valor contido em *Source B*, e o resultado obtido é armazenado em '*Dest*' (destination).

Exemplo:



Nesse exemplo em Diagrama Ladder, ocorre a subtração de *valor_1* com *valor_2*, e o valor obtido é armazenado em *resultado*.



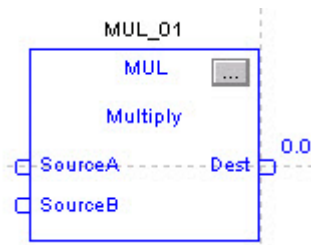
O mesmo ocorre nesse exemplo em diagrama de Blocos.

8.2.4.3. MUL

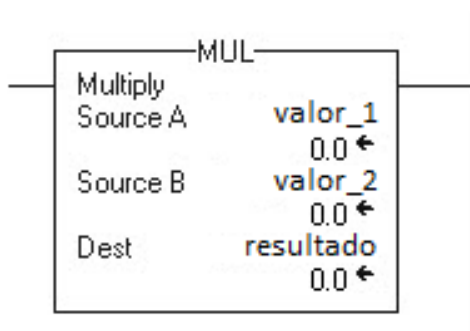
Diagrama Ladder:



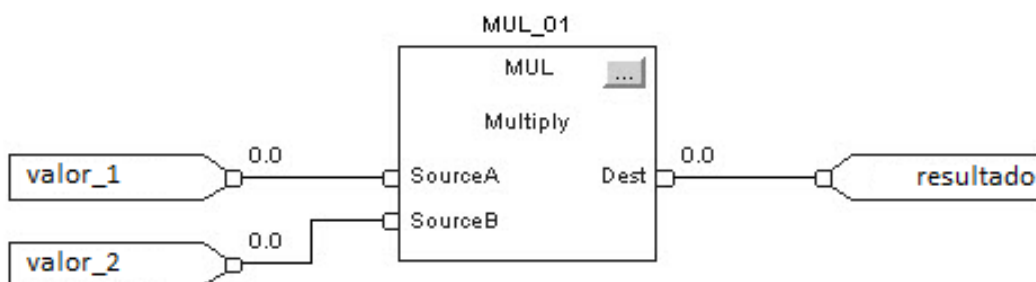
Diagrama de Blocos:



Essa instrução realiza a multiplicação do valor contido em *Source A* (*multiplicando*) com o valor contido em *Source B* (*multiplicador*), e o resultado obtido é armazenado em '*Dest*' (destination).



Nesse exemplo em Diagrama Ladder, ocorre a multiplicação de *valor_1* com *valor_2*, e o valor obtido é armazenado em *resultado*.



O mesmo ocorre nesse exemplo em diagrama de Blocos.

8.2.4.4. DIV

Diagrama Ladder:

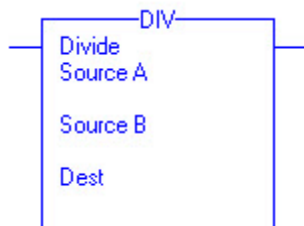
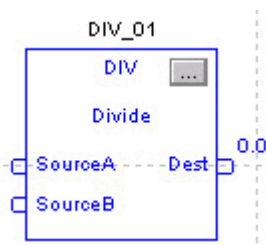
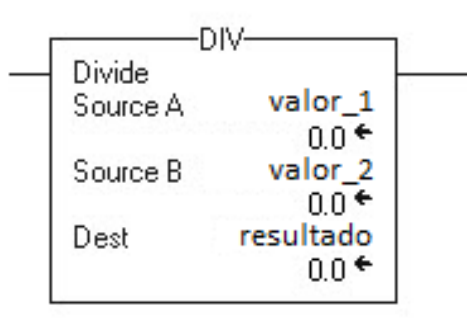


Diagrama de Blocos:

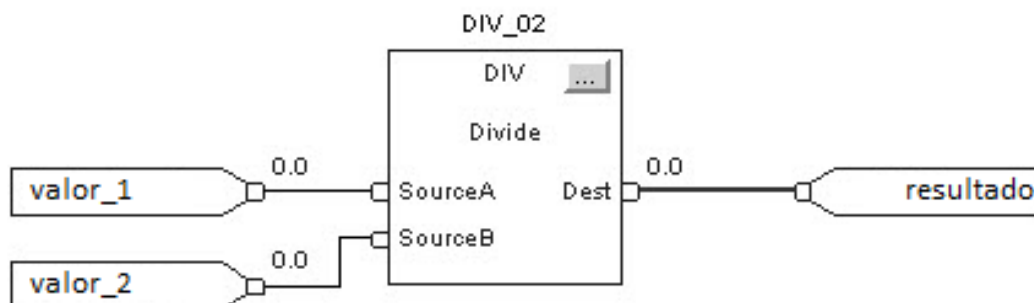


Essa instrução realiza a divisão do valor contido em *Source A* (dividendo) com o valor contido em *Source B* (divisor), e o resultado obtido é armazenado em '*Dest*' (destination).

Exemplo:



Nesse exemplo em Diagrama Ladder, ocorre a divisão de *valor_1* com *valor_2*, e o valor obtido é armazenado em *resultado*.



O mesmo ocorre nesse exemplo em diagrama de Blocos.

8.2.4.5. MOD

Diagrama Ladder:

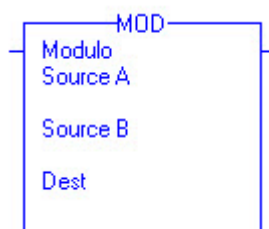
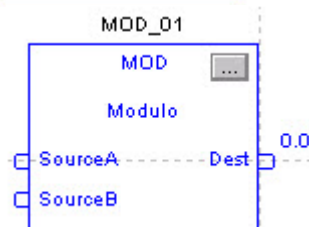
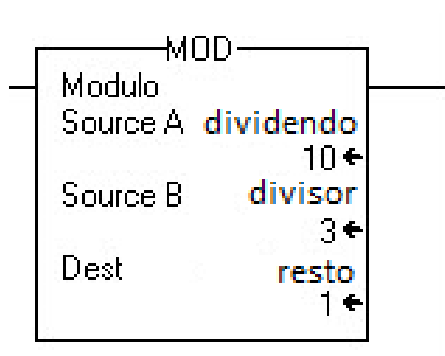


Diagrama de Blocos:

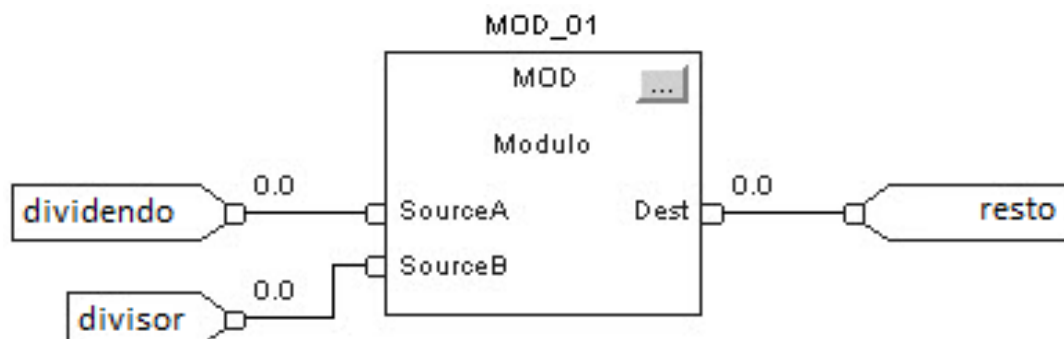


Essa instrução realiza a divisão do valor contido em *Source A* (dividendo) com o valor contido em *Source B* (*divisor*), e o **resto** obtido é armazenado em '*Dest*' (destino).

Exemplo:



Nesse exemplo em Diagrama Ladder, ocorre a divisão do *dividendo* (10) pelo *divisor* (3), e o resto (1) é armazenado em *resto*.



O mesmo ocorre nesse exemplo em diagrama de Blocos.

8.2.4.6. SQR

Diagrama Ladder:

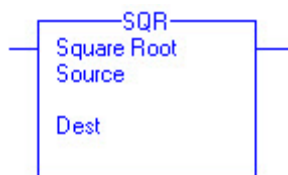
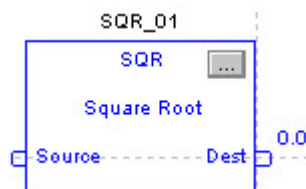
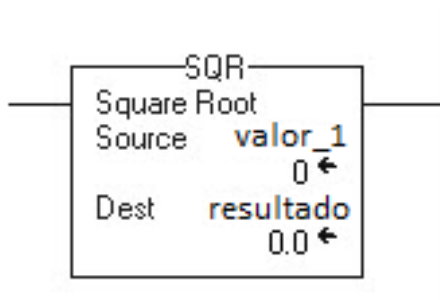


Diagrama de Blocos:

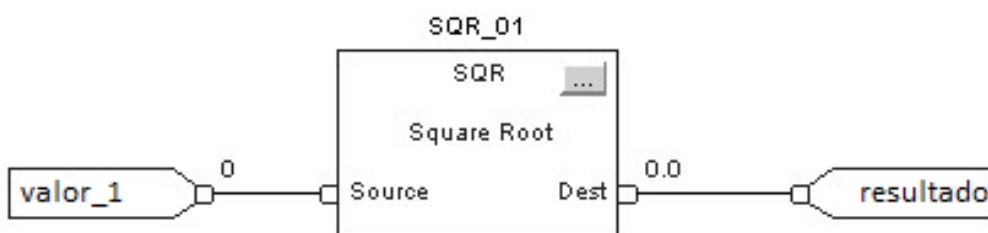


Essa instrução calcula a raiz quadrada do valor armazenado em *Source* e armazena o resultado obtido em *Dest* (destino). Se o valor armazenado em *Source* for negativo, a instrução usa o valor absoluto para calcular a raiz quadrada.

Exemplo:



Nesse exemplo em Diagrama Ladder, ocorre o cálculo da raiz quadrada de *valor_1*, e o resultado é armazenado em *resultado*.



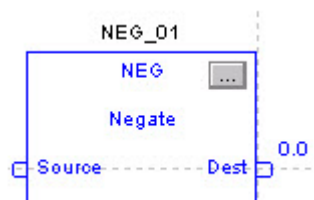
O mesmo ocorre nesse exemplo em diagrama de Blocos.

8.2.4.7. Negate (NEG)

Diagrama Ladder:

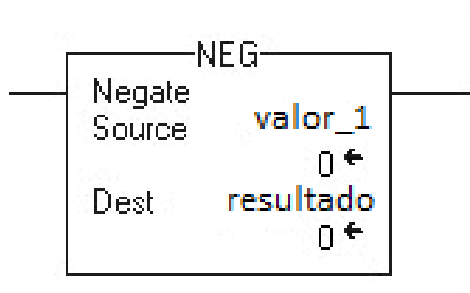


Diagrama de Blocos:

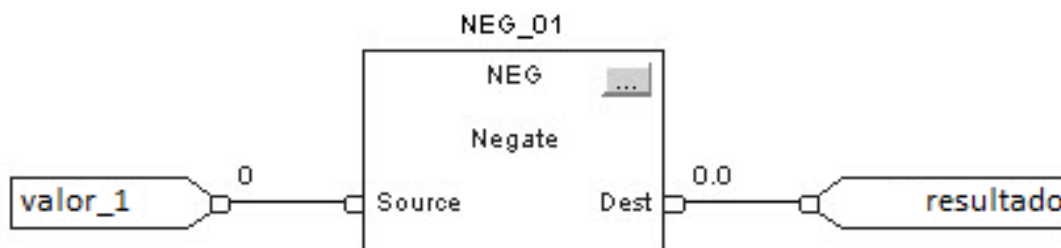


Essa instrução realiza a negação do valor contido em *Source*, ou seja, ocorre a inversão do sinal. O valor obtido é armazenado em *Dest* (destination). Se fizermos a negação de um valor negativo, o resultado é positivo; e se fizermos a negação de um valor positivo, o resultado é negativo.

Exemplo:



Nesse exemplo em Diagrama Ladder, ocorre a negação do valor armazenado em *valor_1*, e o resultado é armazenado em *resultado*.



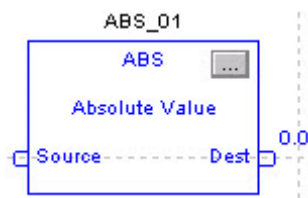
O mesmo ocorre nesse exemplo em diagrama de Blocos.

8.2.4.8. ABS

Diagrama Ladder:

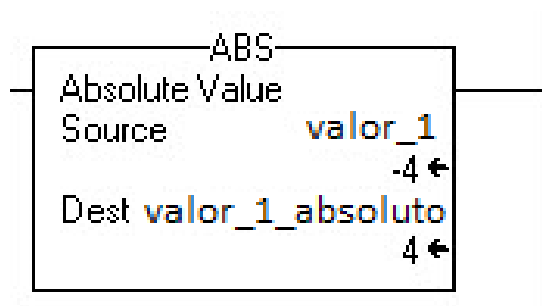


Diagrama de Blocos:

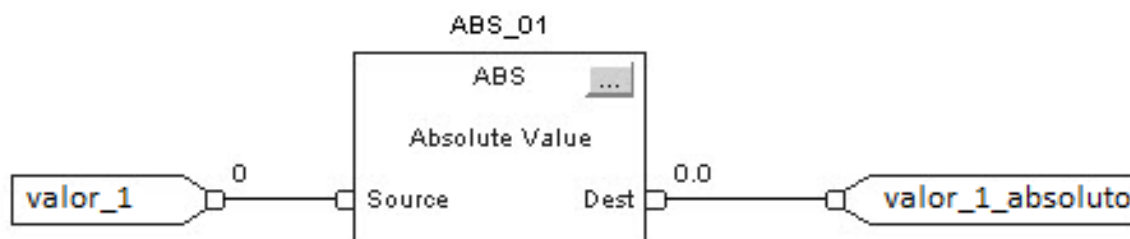


Essa instrução obtém o módulo do valor que está contido em *Source* e armazena-o em *Dest* (destino).

Exemplo:



Nesse exemplo em Diagrama Ladder, obtemos o módulo do valor armazenado em *valor_1*, e o resultado é armazenado em *valor_1_absoluto*.



O mesmo ocorre nesse exemplo em diagrama de Blocos.

8.2.4.9. Compute (CPT)



Essa instrução realiza a operação aritmética definida pelo operador em 'Expression'. Quando habilitada, o CPT avalia a expressão e o resultado dessa operação é armazenado no bit de destino 'Dest'.

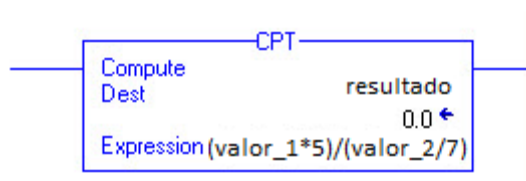
Assim como no bloco de instrução Compare, a instrução possui a vantagem de ser possível resolver expressões complexas em apenas uma instrução.

Entre as operações válidas, temos:

Operador:	Descrição:	Ótimo:
+	adição	DINT, REAL
-	subtração/negativo	DINT, REAL
*	multiplicação	DINT, REAL
/	divisão	DINT, REAL
**	expoente (x elevado a y)	DINT, REAL
ABS	valor absoluto	DINT, REAL
ACS	arco co-seno	REAL
AND	bitwise AND	DINT
ASN	arco seno	REAL
ATN	arco tangente	REAL
COS	co-seno	REAL
DEG	radianos para graus	DINT, REAL
FRD	BCD para inteiro	DINT

Operador:	Descrição:	Ótimo:
LN	natural log	REAL
LOG	logaritmo na base de 10	REAL
MOD	módulo-divisão	DINT, REAL
NOT	bitwise complemento	DINT
OR	bitwise OR	DINT
RAD	graus para radianos	DINT, REAL
SIN	seno	REAL
SQR	raiz quadrada	DINT, REAL
TAN	tangente	REAL
TOD	inteiro para BCD	DINT
TRN	truncado	DINT, REAL
XOR	bitwise exclusivo OR	DINT

Exemplo:



Quando habilitada, o CPT realiza a expressão matemática armazenada em *Expression* e, após isso, armazena o valor obtido em *resultado*.

8.2.5. Operadores Lógicos

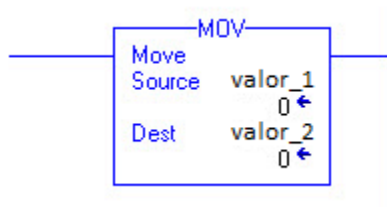
8.2.5.1. MOV

Diagrama Ladder:



Essa instrução copia a informação armazenada de *Source* para *Dest* e, portanto, essa informação continua armazenada também em *Source*.

Exemplo:



Nesse exemplo em diagrama Ladder, o valor contido em *valor_1* é copiado e armazenado em *valor_2*.

8.2.5.2. Bitwise AND (AND)

Diagrama Ladder:

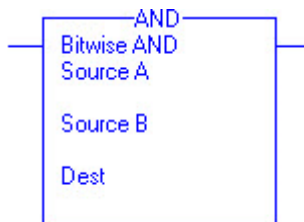


Diagrama de Blocos:



Essa instrução realiza a operação lógica binária AND usando os bits em *Source A* e os bits em *Source B*, armazenando o resultado em *Dest*. Quando habilitada, a instrução avalia a operação AND:

8.2.5.3. Bitwise OR (OR)

Diagrama Ladder:

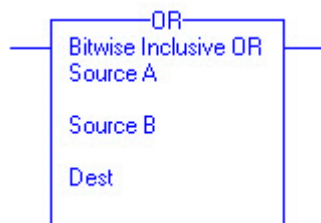
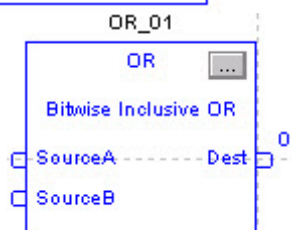


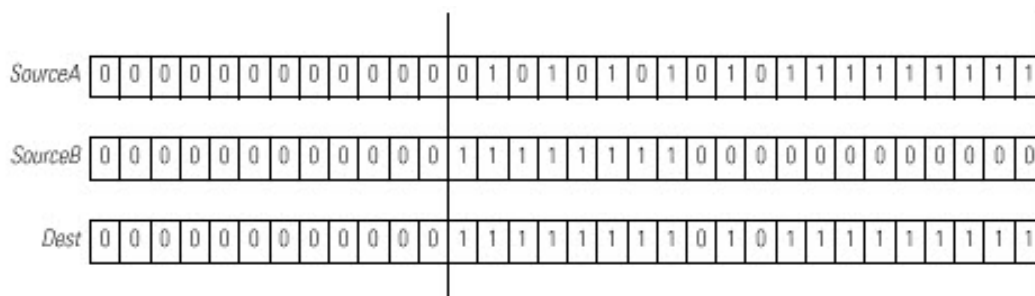
Diagrama de Blocos:

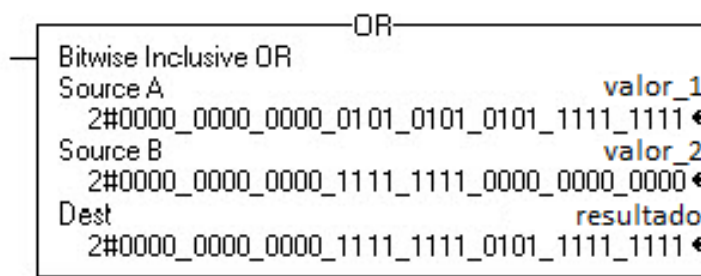


Essa instrução realiza a operação lógica binária OR usando os bits em *Source A* e os bits em *Source B*, armazenando o resultado em *Dest*. Quando habilitada, a instrução avalia a operação OR:

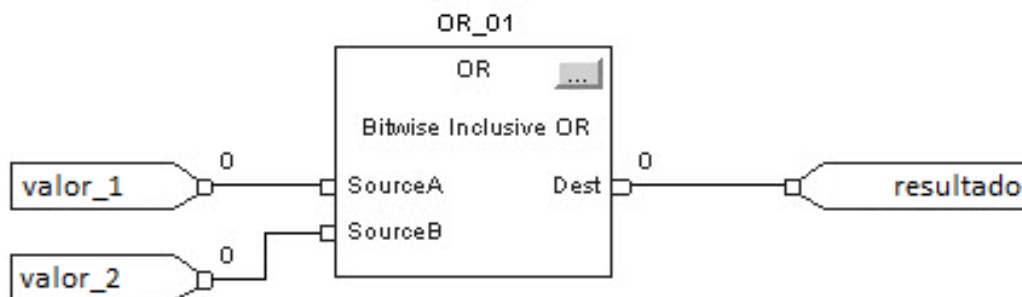
Bit em Source A	Bit em Source B	Bit de Destino
0	0	0
0	1	1
1	0	1
1	1	1

Quando habilitada, a instrução OR realiza a operação lógica em *Source A* (*valor_1*) e *Source B* (*valor_2*), e armazena o resultado obtido em *Dest*. Abaixo vemos a representação dos bits, e logo após a operação em diagrama Ladder.





O mesmo ocorre nesse exemplo em diagrama de Blocos:



8.2.5.4. XOR

Diagrama Ladder:



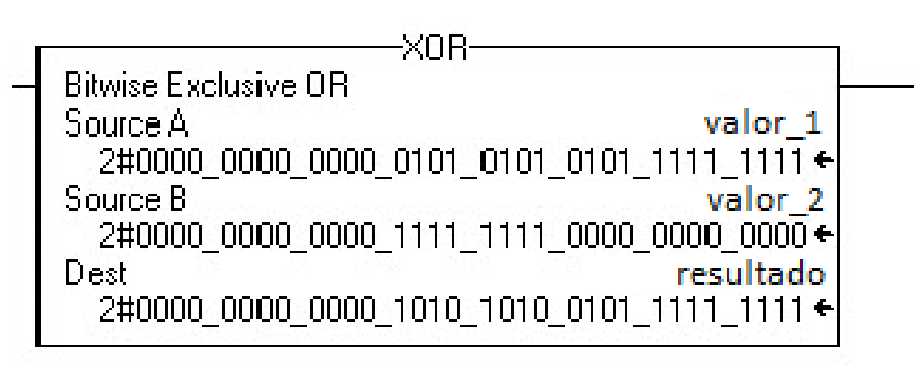
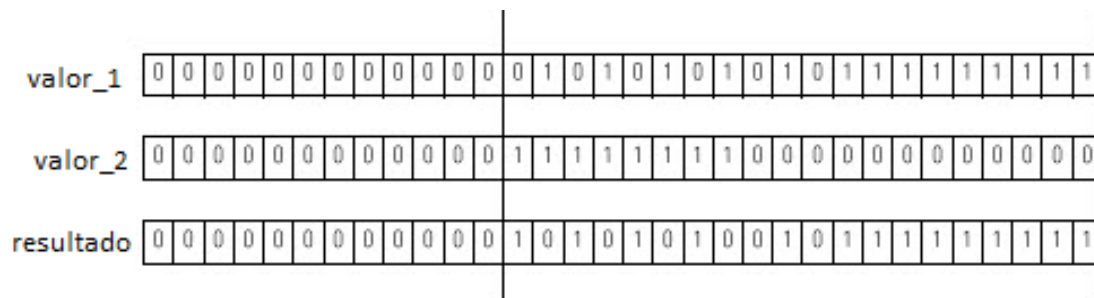
Diagrama de Blocos:



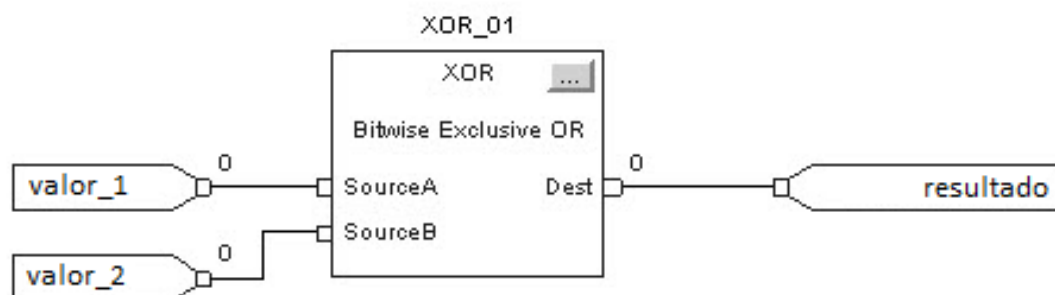
Essa instrução realiza a operação lógica binária XOR usando os bits em *Source A* e os bits em *Source B*, armazenando o resultado em *Dest*. Quando habilitada, a instrução avalia a operação XOR:

Bit em Source A	Bit em Source B	Bit de Destino
0	0	0
0	1	1
1	0	1
1	1	0

Quando habilitada, a instrução XOR realiza a operação lógica em *valor_1* e *valor_2*, e armazena o resultado obtido em *resultado*. Abaixo vemos a representação dos bits, e logo após a operação em diagrama Ladder.



O mesmo ocorre nesse exemplo em diagrama de Blocos:

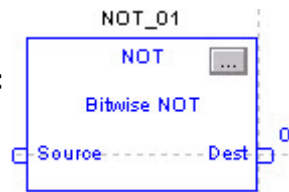


8.2.5.5. Bitwise NOT (NOT)

Diagrama Ladder:



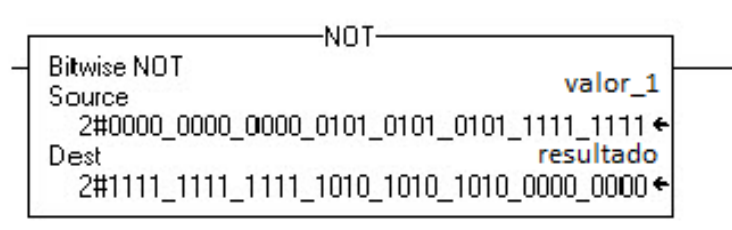
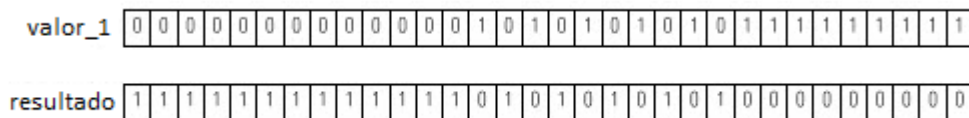
Diagrama de Blocos:



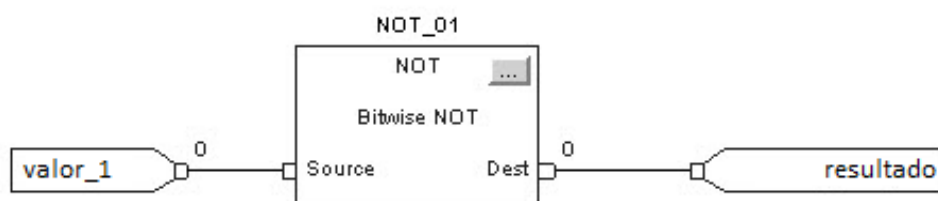
Essa instrução realiza a operação lógica binária NOT usando os bits em *Source* e armazenando o resultado em *Dest*. Quando habilitada, a instrução avalia a operação NOT:

Bit em Source	Bit de Destino
0	1
1	0

Quando habilitada, a instrução NOT realiza a operação lógica em *valor_1*, e armazena o resultado obtido em *resultado*. Abaixo vemos a representação dos bits, e logo após a operação em diagrama Ladder.



O mesmo ocorre nesse exemplo em diagrama de Blocos:



8.2.5.6. CLR



Essa instrução reseta todos os bits que estão armazenados em *Dest*. Em Texto Estruturado, essa seria equivalente à igualar à 0:

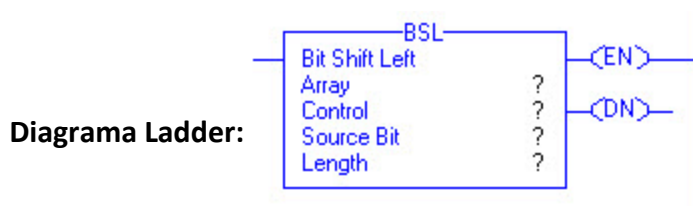
```
valor := 0;
```



Todos os bits são resetados, ou seja, igualados a 0.

8.2.6. Movimentação de Bits

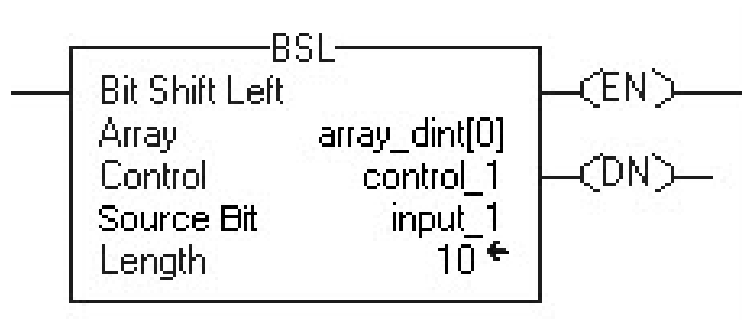
8.2.6.1. Bit Shift Left (BSL)



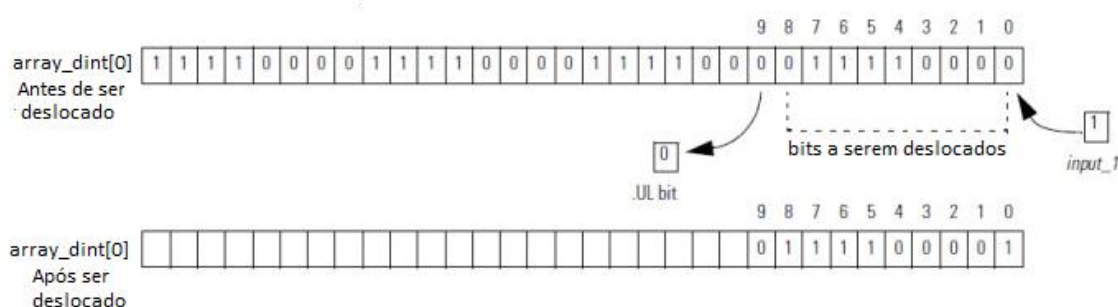
Essa instrução desloca os bits especificados em *Array* uma posição à esquerda. Quando habilitado, a instrução descarrega o bit superior do bit especificado em *Array* para o bit *.UL* (que é a saída da instrução e aloja o status do bit que foi deslocado para fora do range dos bits), e desloca o restante dos bits uma posição à esquerda.

Em *Source Bit* coloca-se o bit (0 ou 1) que será inserido na posição 0 do bit armazenado em *Array*. *Length* especifica o número de bits do *Array* a ser deslocado.

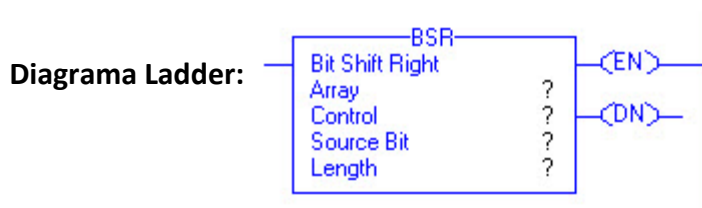
Quando a instrução está habilitada, o bit *.EN* é ativado; e após os bits serem deslocados o bit *.DN* indica que a instrução foi realizada.



Quando habilitado, a instrução BSL começa no bit 0 em *array_dint[0]*. Como vemos abaixo, a instrução descarrega o *array_dint[0].9* no bit .UL, desloca o restante dos bits uma posição à esquerda, e carrega o bit contido em *input_1* no *array_dint[0].0*. O restante dos bits (da posição 10 à 31) são inválidos, uma vez que foi determinado em *Length* o número de 10 bits a serem deslocados.



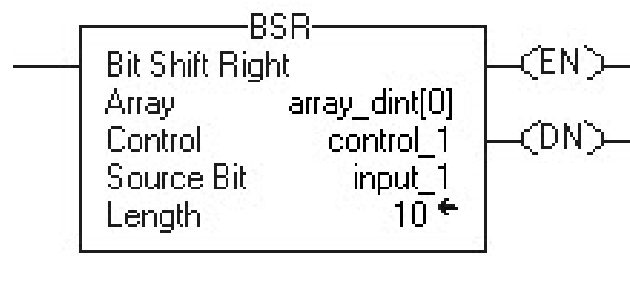
8.2.6.2. Bit Shift Right (BSR)



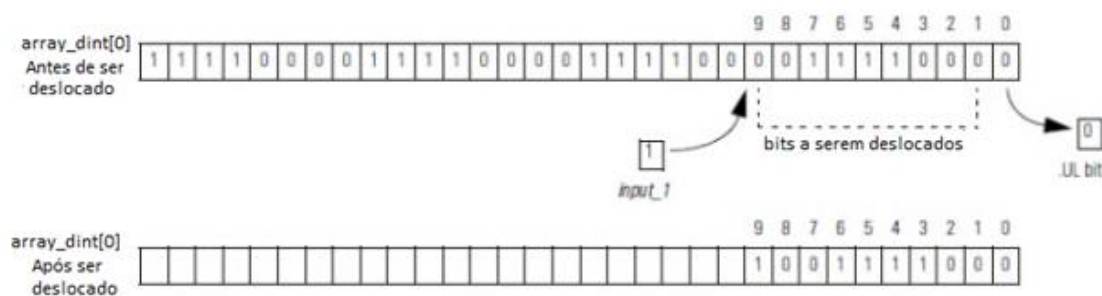
Essa instrução desloca os bits especificados em *Array* uma posição à direita. Quando habilitado, a instrução descarrega o bit superior (último bit) do bit especificado em *Array* para o bit .UL (que é a saída da instrução e aloja o status do bit que foi deslocado para fora do range dos bits), e desloca o restante dos bits uma posição à direita.

Em *Source Bit* coloca-se o bit (0 ou 1) que será inserido na posição 0 do bit armazenado em *Array*. *Length* especifica o número de bits do *Array* a ser deslocado.

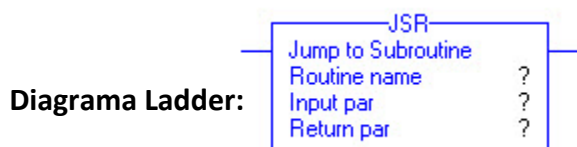
Quando a instrução está habilitada, o bit .EN é ativado; e após os bits serem deslocados o bit .DN indica que a instrução foi realizada.



Quando habilitado, a instrução BSR começa no bit 9 em *array_dint[0]*. Como vemos abaixo, a instrução descarrega o *array_dint[0]09* no bit .UL, desloca o restante dos bits uma posição à direita, e carrega o bit contido em *input_1* no *array_dint[0].9*. O restante dos bits (da posição 10 à 31) são inválidos, uma vez que foi determinado em *Length* o número de 10 bits a serem deslocados.



8.2.7. JSR



Essa instrução realiza um “salto” da rotina principal para uma diferente rotina. Em *Routine name* coloca-se a rotina que se deseja executar, que é referida como Subrotina. Essa subrotina é realizada e, após isso, volta-se a ser executada a lógica do programa principal.



Referências

- Chouliaras, V. A., & Nunez-Yanez, J. L. (2007). *An IEEE 754 floating point engine designed with an electronic system level methodology*. Acesso em 28 de 10 de 2017, disponível em <http://ieeexplore.ieee.org/document/4481066>
- EESC/USP. (s.d.). Memórias. Grupo de Sistemas Digitais.
- Faculdade de Engenharia Elétrica e de Computação (FEEC) e Universidade Estadual de Campinas (UNICAMP). (s.d.). Representação Numérica.
- FRANCHI, C. M., & CAMARGO, V. L. (2010). *Controladores Lógicos Programáveis – Sistemas Discretos* (2ª ed.). São Paulo, SP, Brasil: Érica.
- https://pt.wikipedia.org/wiki/IEC_61131-3. (s.d.).
- Hwang, Y.-M., Kim, M. G., & Rho, J.-J. (2016). Understanding Internet of Things (IoT) diffusion Focusing on value configuration of RFID and sensors in business cases (2008–2012). *Information Development*, 32(4), 0266666915578201. Acesso em 28 de 10 de 2017, disponível em <http://journals.sagepub.com/doi/abs/10.1177/0266666915578201>
- Internet of Things (IoT)*. (s.d.). Acesso em 28 de 10 de 2017, disponível em <http://www.cisco.com/web/solutions/trends/iot/overview.html>
- Lückenhaus, M. (s.d.). *Machine Vision in IIoT | 2016-05-01 | Quality Magazine*. Acesso em 28 de 10 de 2017, disponível em <http://www.qualitymag.com/articles/93296-machine-vision-in-iiot>
- Prudente, F. (2011). *Automação Industrial – PLC: Teoria e Aplicações* (2ª ed.). Rio de Janeiro, RJ, Brasil: LTC.
- Ray, T., Suresh, A., Dash, P. S., & Banerjee, P. K. (2015). Instrumentation and automation system at coal leaching pilot plant. *International Journal of Automation and Control*, 9, 158-171. Acesso em 28 de 10 de 2017, disponível em <http://inderscienceonline.com/doi/pdf/10.1504/ijaac.2015.070008>
- Rockwell Automation. (2001). *Controladores Logix5000 - Manual de Referência Geral do Conjunto de Instruções*.
- Sasajima, H., Ishikuma, T., & Hayashi, H. (2015). *Future IIOT in process automation — Latest trends of standardization in industrial automation, IEC/TC65*. Acesso em 28 de 10 de 2017, disponível em <http://ieeexplore.ieee.org/document/7285569>
- Souza, F. M. (1999). *Instrumentação – Automação Básica*. ES.
- The father of invention: Dick Morley looks back on the 40th anniversary of the PLC. (s.d.). *Manufacturing Automation*. Acesso em 28 de 10 de 2017, disponível em



<http://www.automationmag.com/programable-control/features/the-father-of-invention-dick-morley-looks-back-on-the-40th-anniversary-of-the-plc.html>

The Modbus Organization. (s.d.). Acesso em 28 de 10 de 2017, disponível em Modbus Organization, Inc.: <http://www.modbus.org/>

VIANNA, W. d. (2008). Controlador Lógico Programável. Campos dos Goytacazes, RJ, Brasil.

What is a PLC. Who uses them and what for? Learn how to. (s.d.). Acesso em 28 de 10 de 2017, disponível em <http://www.machine-information-systems.com/PLC.html>